



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIEROS AGRÓNOMOS



Optimización de la programación (*scheduling*) en Talleres de Mecanizado

TESIS DOCTORAL

D. José Eduardo Márquez Delgado

Ingeniero Mecánico

Madrid 2012



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIEROS AGRÓNOMOS



Optimización de la programación (*scheduling*) en Talleres de Mecanizado

TESIS DOCTORAL

Presentada por

José Eduardo Márquez Delgado

Ingeniero Mecánico

Dirigida por

Miguel Ángel Gómez-Elvira González

Doctor Ingeniero por la Universidad Politécnica de Madrid

Ricardo Lorenzo Ávila Rondón

Doctor en Ciencias Técnicas por la Universidad de Holguín (Cuba)

Madrid 2012



(D-15)

Tribunal nombrado por el Magfco. y Excmo. Sr. Rector de la Universidad Politécnica de Madrid, el día _____ de _____ de 2012.

Presidente: _____

Secretario: _____

Vocal: _____

Vocal: _____

Vocal: _____

Suplente: _____

Suplente: _____

Realizado el acto de defensa y lectura de Tesis el día _____ de _____ del 2012 en la E.T.S.I. Agrónomos.

EL PRESIDENTE

LOS VOCALES

EL SECRETARIO



D. Miguel Ángel Gómez-Elvira González, profesor del departamento de Ingeniería Cartográfica – Geodesia y Fotometría – Expresión Gráfica de la Escuela Técnica Superior de Ingenieros Agrónomos, de la Universidad Politécnica de Madrid.

DECLARA:

Que la Tesis Doctoral titulada “Optimización de la programación (*scheduling*) en Talleres de Mecanizado”, ha sido realizada por D. José Eduardo Márquez Delgado para aspirar al Grado de Doctor por la Universidad Politécnica de Madrid, bajo mi dirección y por ello autorizo la presentación de dicha tesis para que se proceda al trámite de su lectura.

Y para que así conste, expido la presente declaración en Madrid, a _____ de _____ de dos mil doce.

Miguel Ángel Gómez-Elvira González

GLOSARIO DE TÉRMINOS

AE	Algoritmos Evolutivos
AGS	Algoritmo Genético Simple (<i>Simple Genetic Algorithm</i>)
Cmax	Camino máximo, camino crítico (<i>makespan</i>)
CSP	<i>Constraint Satisfaction Problems</i> (problemas con satisfacción de restricciones)
E	<i>End</i> (Nodo final en un grafo)
Ec	Consumo de Energía
ECT	<i>Earliest Completion Time</i> (Tiempo de finalización adelantado)
EDD	<i>Earliest Due Date</i> (Fecha de compromiso adelantada)
FIFO	<i>First In First Out</i> (Primero en entrar primero en salir)
Forced left jump	Salto a La izquierda forzado
FSS	<i>Flow Shop Scheduling</i> (Flujo regular)
Idle	Tiempo de Ocio en las máquinas
JSS	<i>Job Shop Scheduling</i> (Flujo general)
LA	Problemas de La clase <i>Lawrence</i>
Left jump	Salto a La izquierda
Left shift	Corrimiento a La izquierda
LIFO	<i>Last In First Out</i> (Último en entrar primero en salir)
Lmax	Retraso total
LTP	<i>Longest Processing Time</i> (Tiempo de procesamiento mayor)
MBUTO	Mejora Basada en el Uso Del Tiempo Ocioso
N	Número de trabajos
NE	Número de trabajos adelantados
NP-Hard	Clase de problemas no polinomiales completo
ORB	Problemas de La clase <i>Lawrence</i>
OR-Library	<i>Operations Research Library</i> (biblioteca de investigación de operaciones)
OSS	<i>Open Shop Scheduling</i> (Flujo aleatorio)
PFSS	<i>Permutation Flow Shop Scheduling</i> (Flujo permutacional)
Pm	Máquinas en paralelo idénticas
Qm	Máquinas en paralelo uniformes
Rm	Máquinas en paralelo diferentes o no relacionadas

S	<i>Start</i> (Nodo inicial en un grafo)
SIRO	<i>Service In Random Order</i> (Servicio en orden aleatorio)
STP	<i>Shortest Processing Time</i> (Menor tiempo de procesamiento)
SWV	Problemas de La clase <i>Storer-Wu-Vaccari</i>
Tmax	Tardanza total
V	Vértices en un grafo
YN	Problemas de La clase <i>Yamada-Nakano</i>
α	Características de las máquinas
β	Características de los trabajos
γ	Criterios de optimización

RESUMEN

Los problemas de programación de tareas son muy importantes en el mundo actual. Se puede decir que se presentan en todos los fundamentos de la industria moderna, de ahí la importancia de que estos sean óptimos, de forma que se puedan ahorrar recursos que estén asociados al problema. La programación adecuada de trabajos en procesos de manufactura, constituye un importante problema que se plantea dentro de la producción en muchas empresas. El orden en que estos son procesados, no resulta indiferente, sino que determinará algún parámetro de interés, cuyos valores convendrá optimizar en la medida de lo posible. Así podrá verse afectado el coste total de ejecución de los trabajos, el tiempo necesario para concluirlos o el stock de productos en curso que será generado. Esto conduce de forma directa al problema de determinar cuál será el orden más adecuado para llevar a cabo los trabajos con vista a optimizar algunos de los anteriores parámetros u otros similares.

Debido a las limitaciones de las técnicas de optimización convencionales, en la presente tesis se presenta una metaheurística basada en un Algoritmo Genético Simple (*Simple Genetic Algorithm, SGA*), para resolver problemas de programación de tipo flujo general (*Job Shop Scheduling, JSS*) y flujo regular (*Flow Shop Scheduling, FSS*), que están presentes en un taller con tecnología de mecanizado con el objetivo de optimizar varias medidas de desempeño en un plan de trabajo.

La aportación principal de esta tesis, es un modelo matemático para medir el consumo de energía, como criterio para la optimización, de las máquinas que intervienen en la ejecución de un plan de trabajo. Se propone además, un método para mejorar el rendimiento en la búsqueda de las soluciones encontradas, por parte del Algoritmo Genético Simple, basado en el aprovechamiento del tiempo ocioso.

ABSTRACT

The scheduling problems are very important in today's world. It can be said to be present in all the basics of modern industry, hence the importance that these are optimal, so that they can save resources that are associated with the problem. The appropriate programming jobs in manufacturing processes is an important problem that arises in production in many companies. The order in which they are processed, it is immaterial, but shall determine a parameter of interest, whose values agree optimize the possible. This may be affected the total cost of execution of work, the time needed to complete them or the stock of work in progress that will be generated. This leads directly to the problem of determining what the most appropriate order to carry out the work in order to maximize some of the above parameters or other similar.

Due to the limitations of conventional optimization techniques, in this work present a metaheuristic based on a Simple Genetic Algorithm (Simple Genetic Algorithm, SGA) to solve programming problems overall flow rate (Job Shop Scheduling, JSS) and regular flow (Flow Shop Scheduling, FSS), which are present in a workshop with machining technology in order to optimize various performance measures in a plan.

The main contribution of this thesis is a mathematical model to measure the energy consumption as a criterion for the optimization of the machines involved in the implementation of a work plan. It also proposes a method to improve performance in finding the solutions, by the simple genetic algorithm, based on the use of idle time.

PENSAMIENTO

“El problema general de job shop es un reto fascinante. Si bien es fácil plantearlo y visualizarlo, es extremadamente difícil resolverlo. El problema continúa atrayendo investigadores, quienes no pueden creer que un problema tan simplemente estructurado, pueda ser tan difícil, hasta que lo intentan.”

Richard W. Conway, et al. (1967)

A toda mi familia,
A todas mis amistades.

AGRADECIMIENTO

En primer lugar me gustaría mostrar mi más sincero agradecimiento al Profesor D. Ricardo Lorenzo Ávila Rondón, uno de los directores de esta investigación y memoria, no sólo por los conocimientos que me ha transmitido durante todos estos años y el interés que han despertado en mí hacia la investigación en este apasionante mundo de la Planificación Asistida por Computadora, sino también por la confianza, la amistad y el afecto que me ha manifestado siempre. También a D. Miguel Ángel Gómez-Elvira González, el otro director de esta investigación quien me ha brindado un gran apoyo y confianza desde su elección como tutor en la UPM. A ambos Los considero mis amigos y maestros. Un especial agradecimiento a dos profesores catedráticos de la Universidad Politécnica de Madrid: D. Manuel Camps Michelena y D. Víctor Manuel Sánchez-Girón Renedo, quienes han tenido a su cargo este programa de doctorado por la parte española, así como también por la parte cubana al Dr. Rubén Jerez Pereira, coordinador del mismo en la Universidad de Granma.

A todos los miembros del departamento de Informática de la Facultad de Ciencias Técnicas de la Universidad de Granma, por brindarme todo su apoyo para poder dedicarme a tiempo completo a la culminación de esta obra.

A los directivos, personal de oficinas y otras ocupaciones de la Universidad de Granma, y en particular de la facultad de Ciencias Técnicas.

A todas mis amistades de la empresa “Manuel Fajardo Rivero”, en Manzanillo, donde adquirí los conocimientos necesarios que me han servido de cimientos en toda mi vida profesional.

A todos los integrantes del Centro de Estudios de CAD/CAM de la Universidad de Holguín, los actuales y los que ya no están, por la inmensa huella que han dejado en mi formación profesional, hasta llegar a sentirme uno más de ellos.

A los integrantes del departamento de Ingeniería Cartográfica – Geodesia y Fotometría – Expresión Gráfica, de la Universidad Politécnica de Madrid, por hacerme sentir en su departamento como en mi propia casa.

A Yoel Rodríguez Valera, por su incondicional contribución en todo momento.

En España, a Pedro A. Scudieri González por su total apoyo.

Finalmente, pero no por ello menos importante, a mis padres y a toda mi familia, por la paciencia y el cariño siempre me han demostrado, especialmente a mi sobrino Carlos Rafael, quien soportó todas mis discusiones cada vez que eran necesarias, y fueron necesarias siempre.

A mi esposa Yelaine por entenderme y complacerme cada vez que me hizo falta su ayuda, desinteresada y oportuna.

Sin la ayuda de Carlos y Yelaine, sencillamente no hubiera podido llevar a feliz término la investigación durante estos últimos años de mi vida, principalmente por mis responsabilidades en el trabajo.

Gracias.

Madrid, noviembre de 2012.

INDICE DE CONTENIDO

INTRODUCCIÓN	1
Capítulo 1	5
PROBLEMAS DE PLANIFICACIÓN DE TRABAJOS	5
1.1. Introducción	5
1.2. El problema del taller mecánico	6
1.3. Tipos de problemas de <i>scheduling</i>	7
1.4. Complejidad computacional	10
1.5. Descripción del <i>Job Shop Scheduling Problem</i>	10
1.5.1. Formulación matemática del problema	11
1.5.2. Restricciones del modelo de optimización	13
1.6. Nomenclatura	13
1.6.1. Características de las máquinas. Campo α	14
1.6.2. Características de los trabajos. Campo β	16
1.6.3. Criterios de optimalidad. Campo γ	17
1.7. Métodos de resolución	18
1.7.1. Métodos exactos	18
1.7.2. Métodos heurísticos	19
1.7.3. Métodos metaheurísticos	25
1.7.4. Métodos de aproximación	26
1.7.5. El Algoritmo <i>G&T</i>	29
1.8. Aplicación de algoritmos genéticos a la solución de problemas de <i>scheduling</i>	32
1.9 Conclusiones del capítulo	36
Capítulo 2	37
VARIANTES DE PROBLEMAS DE PLANIFICACIÓN EN EL TALLER DE MECANIZADO	37
2.1. Introducción	37
2.2. Estructura de datos para la representación del problema	38

2.3. Variante Flujo General (<i>Job Shop Scheduling, JSS</i>)	39
2.4. Variante Flujo Regular (<i>Flow Shop Scheduling, FSS</i>)	42
2.5. Variante Flujo Permutacional (<i>Permutational Flow Shop Scheduling, PFSS</i>)	46
2.6. Estudio del universo de soluciones posibles en correspondencia con la forma de obtener secuencias.....	49
2.7. Análisis de los tipos de planes de trabajo	53
2.8. Criterios tenidos en cuenta para la optimización	56
2.8.1. Criterios basados en los instantes de finalización	57
2.8.2. Criterios basados en la fecha de entrega.....	58
2.8.3. Criterio basado en el consumo de energía.....	59
2.9. Relaciones entre las medidas de desarrollo	64
2.10. Estructura utilizada para construir el cromosoma. Modelo genético	64
2.11. Mejora Basada en el Uso del Tiempo Ocioso (MBUTO).....	73
2.12. Conclusiones del capítulo	82
Capítulo 3	83
ANÁLISIS DE RESULTADOS	83
3.1. Introducción	83
3.2. Estudio experimental	83
3.3 Parámetros utilizados en el Algoritmo Genético Simple	93
3.4. Estudio experimental. Materiales y métodos	93
3.4.1. Experimento No. 1	94
3.4.2. Experimento No. 2	100
3.5. Análisis estadístico	104
3.5.1. Resultados	104
3.5. Aplicación computacional realizada.....	108
3.6. Conclusiones del capítulo	108
CONCLUSIONES GENERALES.....	109
RECOMENDACIONES	110
BIBLIOGRAFÍA	111
ANEXOS	120

INDICE DE FIGURAS

Figura 1.1. Variantes de problemas de <i>scheduling</i> según el patrón de flujo.	8
Figura 1.2. Operadores genéticos: cruzamiento y mutación.	33
Figura 1.3. Esquema de funcionamiento del algoritmo genético simple.....	35
Figura 2.1. Ejemplo de un grafo disyuntivo dirigido para un problema tipo JSS de 3 trabajos en 3 máquinas (3 x 3).	39
Figura 2.2. Gráfico de Gantt con una solución factible para el grafo anterior. El orden de ejecución de los trabajos es: 1 2 1 – 1 2 3 – 2 3 3.	41
Figura 2.3. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la figura anterior.	42
Figura 2.4. Subgrafo resultante que representa la solución del camino máximo.....	42
Figura 2.5. Ejemplo de un grafo disyuntivo dirigido para un problema de la variante FSS de 3 piezas en 3 máquinas (3 x 3).	43
Figura 2.6. Gráfico de Gantt con una solución factible para el grafo anterior. El orden de ejecución de los trabajos es: 2 1 3 – 1 3 2 – 3 2 1.	44
Figura 2.7. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la figura anterior.	45
Figura 2.8. Subgrafo resultante que representa la solución del camino máximo.....	45
Figura 2.9. Grafo disyuntivo dirigido para un problema de la variante FSS con 3 piezas en 3 máquinas (3 x 3).	47
Figura 2.10. Gráfico de Gantt con una solución factible para el grafo de la figura anterior. El orden de ejecución de los trabajos es: 1 3 2.	47
Figura 2.11. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la anterior.	48
Figura 2.12. Subgrafo que representa la solución del camino máximo.	49

Figura 2.13. Gráfico de Gantt con plan de trabajo semiactivo. Variante JSS.....	53
Figura 2.14. Gráfico de Gantt con plan de trabajo activo. Variante JSS.	54
Figura 2.15. Gráfico de Gantt con plan de trabajo sin retardo debido a máquinas no libres. Variante PFSS.....	54
Figura 2.16. Gráfico de Gantt con plan de trabajo sin retardo debido a piezas no libres. Variante PFSS.....	54
Figura 2.17. Gráfico de Gantt con plan de trabajo sin retardo debido a máquinas y piezas no libres. Variante JSS.	54
Figura 2.18. Jerarquía de las clases de planes de trabajo.....	55
Figura 2.19. Datos y variables asociados a un trabajo.....	56
Figura 2.20. Gráfico de Gantt con marcas de lectura para el consumo de energía de un plan de trabajo.....	60
Figura 2.21. Gráfico de carga que muestra el consumo de energía del plan de trabajo correspondiente al gráfico de Gantt anterior. Consumo total: 198 unidades.....	60
Figura 2.22. Gráfico de Gantt con varios objetivos calculados.....	63
Figura 2.23. Representación de un cromosoma de 6 trabajos para un problema del caso particular PFSS. No se tiene en cuenta el número de máquinas.....	65
Figura 2.24. Cromosoma inválido.....	66
Figura 2.25. Representación de un cromosoma para un problema tipo FSS de 6 piezas en 6 máquinas. Nótese la no repetición de alelos en un mismo gen.....	68
Figura 2.26. Cromosoma inválido.....	68
Figura 2.27. Representación de un cromosoma para un problema tipo JSS de 6 piezas en 6 máquinas. Nótese la repetición de alelos en un mismo gen.....	70
Figura 2.28. Cromosoma inválido.....	71

Figura 2.29. Esquema de funcionamiento del Algoritmo Genético Simple con la Mejora Basada en el Uso del Tiempo Ocioso.....	74
Figura 2.30. Gráfico de Gantt con valor de $C_{max} = 70$	75
Figura 2.31. Gráfico de Gantt con valor de $C_{max} = 65$	75
Figura 2.32. Datos Gráfico de Gantt con valor de $C_{max} = 65$	76
Figura 2.33. Datos Gráfico de Gantt con valor óptimo de $C_{max} = 55$	76
Figura 3.1. Gráfico de Gantt de un plan de trabajo tipo <i>flow shop</i> . Corresponde a la solución de la instancia de problema ta001. Valor de $C_{max} = 1278$	84
Figura 3.2. Gráfico de Gantt de un plan de trabajo tipo <i>job shop</i> . Corresponde a la solución de la instancia de problema la01. Valor óptimo de $C_{max} = 666$	87
Figura 3.3. Gráfico de Gantt con la mejor solución encontrada por AGS. Valor de $C_{max} = 1263$	98
Figura 3.4. Gráfico de Gantt con la mejor solución encontrada por AGS + MBUTO. Valor óptimo de $C_{max} = 1207$	98

INDICE DE TABLAS

Tabla 2.1. Instancia ejemplo de problema 3 x 3 variante JSS.	40
Tabla 2.3. Instancia ejemplo de problema 3 x 3 variante FSS.	44
Tabla 2.4. Ejemplos de dimensiones de problema y el tamaño del espacio de búsqueda de soluciones factibles correspondiente en dependencia de la forma de obtener las secuencias.	51
Tabla 2.5. Secuencias posibles del espacio total de búsqueda de soluciones factibles correspondiente a un problema de 3 trabajos ($N = 3$) en 3 máquinas $M = 3$	52
Tabla 2.6. Tipos de representación según la estructura del cromosoma utilizada.	64
Tabla 3.1. Resultados obtenidos para problemas de tipo <i>flow shop</i> . Instancias ta001 – ta040.	85
Tabla 3.2. Resultados obtenidos para problemas de tipo <i>flow shop</i> . Instancias ta041 – ta080.	86
Tabla 3.3. Resultados obtenidos para problemas <i>Job Shop</i> . Las instancias corresponden a S. Lawrence. (<i>Lawrence's benchmark problems</i>).	88
Tabla 3.4. Resultados de <i>makespan</i> alcanzados utilizando los problemas de tipo <i>Job shop</i> pertenecientes a la clase ORB (<i>Applegate & Cook benchmark problems</i>).	89
Tabla 3.5. Resultados de <i>makespan</i> alcanzados utilizando los problemas de tipo <i>Job shop</i> pertenecientes a la clase ABZ (<i>Adams-Balas-Zawack benchmark problems</i>).	89
Tabla 3.6. Resultados de <i>makespan</i> alcanzados utilizando los problemas de tipo <i>Job shop</i> pertenecientes a la clase YN (<i>Yamada & Nakano benchmark problems</i>).	89
Tabla 3.7. Resultados de <i>makespan</i> alcanzados utilizando los problemas de tipo <i>Job Shop</i> pertenecientes a la clase FT (<i>Fisher & Thompson benchmark problems</i>).	89

Tabla 3.8. Resultados de <i>makespan</i> alcanzados utilizando los problemas de tipo <i>Job shop</i> pertenecientes a la clase SWV (<i>Storer-Wu-Vaccari benchmark problems</i>).....	90
Tabla 3.9. Resultados de <i>makespan</i> alcanzados en la presente investigación utilizando los problemas seleccionados de mayor dificultad.....	92
Tabla 3.10. Resultados de las ejecuciones realizadas. Valores máximos y mínimos de <i>Cmax</i>	95
Tabla 3.11. Resultados de las ejecuciones realizadas: generación y tiempo.	101
Tabla 3.12. Valores descriptivos de los promedios de <i>Cmax</i>	105
Tabla 3.13. Eficacia del uso del método GA + MBUTO.....	107

INDICE DE ALGORITMOS

Algoritmo 1.1 Algoritmo Evolutivo.....	22
Algoritmo 1.2 Algoritmo de Búsqueda local.....	24
Algoritmo 1.3 Algoritmo G&T.....	30
Algoritmo 2.1. Algoritmo para Cálculo del Consumo de Energía de un plan de trabajo (<i>schedule</i>).	62
Algoritmo 2.2 Algoritmo para cromosomas inválidos de la forma: $n!$	67
Algoritmo 2.3 Algoritmo para cromosomas inválidos de la forma: $(n!)^m$	69
Algoritmo 2.4 Algoritmo para cromosomas inválidos de la forma: $\frac{(N \times M)!}{(M!)^N}$	72
Algoritmo 2.5 Algoritmo para el adelantamiento por salto a la izquierda.	78
Algoritmo 2.6 Algoritmo para el control del desplazamiento a la izquierda de una operación.....	79
Algoritmo 2.7 Algoritmo MBUTO.	81

INTRODUCCIÓN

Los problemas de planificación o programación de tareas son muy importantes en diversas aplicaciones del mundo real. Dichos problemas se utilizan en una amplia gama de diversas aplicaciones, por ejemplo, en las líneas de producción de una fábrica, en los aeropuertos para despachar los vuelos, en los hospitales para atender a los pacientes, en las escuelas para distribuir las actividades de los alumnos y profesores, en un taller para decidir qué equipo es reparado primero o bien la secuencia de la reparación de este, etc.

El *scheduling* es una tarea extremadamente difícil con una importante necesidad de cálculo. Diversas áreas de aplicación están sujetas a estos problemas, tal como operaciones de producción en una industria de manufactura, procesamiento computacional en un sistema operativo, movimiento de vehículos de transporte, etc. La gran importancia práctica convierte al *scheduling* en un área activa de investigación. Estos problemas poseen estructuras más complejas al tener que incorporarse al sistema de información de la empresa u organización, lo cual puede ser una tarea considerable, pero en situaciones reales, pueden ser relevantes diferentes restricciones.

El desarrollo actual de las computadoras, y la aparición de nuevas técnicas de simulación y optimización heurística que aprovechan plenamente las disponibilidades de cálculo intensivo que estas proporcionan, han abierto una nueva vía para abordar los problemas de secuenciación o problemas de *scheduling* (Blazewicz, *et al.*, 2007; Pinedo, 2008) como también se le conocen, y han suministrado un creciente arsenal de métodos y algoritmos (Carlier y Pinson, 1989; Brucker, *et al.*, 1994; Wenqi y Aihua, 2004) cuyo uso se extiende paulatinamente al sustituir a las antiguas reglas y algoritmos usados tradicionalmente.

La anterior situación del estado actual del problema llevó a la definición del siguiente problema científico.

Problema científico:

Insuficiencias en el proceso de planificación (programación) de secuencias para la fabricación de piezas por tecnología de mecanizado.

Hipótesis:

El realizar un modelo matemático para medir el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo, en adición a otras medidas de desempeño establecidas, así como un método para mejorar el rendimiento en la búsqueda de las soluciones encontradas, permitirá optimizar el orden de ejecución (programación) en la fabricación de piezas en talleres con tecnología de mecanizado.

El **objeto de investigación** lo constituye el proceso de programación de la producción.

Y el **campo de acción** las secuencias de fabricación de piezas en talleres de mecanizado.

Para contribuir a solucionar el problema científico y transformar el objeto de la investigación, se plantea como **objetivo general** establecer un modelo matemático para medir el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo, en adición a otras medidas de desempeño establecidas, así como un método para mejorar el rendimiento en la búsqueda de las soluciones encontradas.

Objetivo Específicos:

1. Representar a través de grafos disyuntivos las variantes de problemas de *scheduling* que se presentan en talleres con tecnología de mecanizado.

2. Determinar el universo de soluciones factibles para cada variante de problema estudiado en correspondencia con la forma de obtener las secuencias.
3. Crear un modelo matemático para determinar el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo.
4. Utilizar un algoritmo genético simple como metaheurística para la búsqueda de soluciones a problemas de programación de trabajos.
5. Mejorar el rendimiento de la metaheurística con el aprovechamiento del tiempo ocioso en las máquinas.
6. Modelar el problema de la programación de trabajos para calcular varios objetivos agrupados en tres categorías: basados en los instantes de finalización, basados en las fechas de compromisos y uno basado en el consumo de energía.
7. Comparar resultados de optimización, para el objetivo del camino máximo, con problemas clásicos de la literatura científica.
8. Realizar análisis estadístico para comprobar el rendimiento de la metaheurística con la adición de un método basado en el uso del tiempo ocioso en las máquinas.

Para realizar la investigación en sus diferentes etapas se llevaron a cabo las siguientes **tareas de investigación**:

1. Revisión bibliográfica para la caracterización del problema a investigar.
2. Diagnóstico del estado actual del problema, revelando las principales insuficiencias a resolver en la investigación.
3. Estudio de los principales métodos y técnicas de optimización empleados en la solución del problema de la programación de tareas en talleres de fabricación por tecnología de mecanizado.
4. Diseño e implementación de la aplicación informática que se propone.
5. Valoración de la efectividad de la solución a partir de la comparación de los resultados obtenidos con relación a otros investigadores, con similares y diferentes técnicas de optimización, utilizando los problemas propuestos

(*benchmarks problems*) en la la biblioteca de investigación de operaciones (*Operations Research Library, OR- Library*).

Métodos Científicos empleados para dar cumplimiento a las tareas.

- Métodos Teóricos:

Analítico - sintético: Se utilizó para procesar la información empírica y teórica sobre la caracterización del problema, del objeto y del campo de acción, así como la elaboración de las conclusiones parciales y generales de la investigación.

Histórico - lógico: este método se empleó para el estudio de los antecedentes de investigaciones relacionadas con las técnicas de optimización y los problemas de *scheduling*, en las condiciones histórico-concretas por las que han transitado estas temáticas.

Sistémico - estructural: se utilizó para lograr la estructura lógica científica y lo relativo a la modelación del producto.

- Métodos Empíricos:

Estudio de la documentación: Se utilizó para la revisión de documentos bibliográficos que aportaran información del desarrollo de las técnicas de optimización y los métodos más utilizados.

Entrevista: Se utilizó para comprender mejor el funcionamiento, ventajas y desventajas de las técnicas de optimización analizadas.

En esta investigación se presenta como **novedad** un modelo matemático para medir el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo (*schedule*).

Como **aportes** de este trabajo se encuentran:

- Un modelo matemático para medir el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo.
- Un método para mejorar el rendimiento en la búsqueda de soluciones encontradas a partir del aprovechamiento del tiempo de ocio en las máquinas.

Capítulo 1

PROBLEMAS DE PLANIFICACIÓN DE TRABAJOS

1.1. Introducción

Diversas opiniones existen con respecto al origen del *Job Shop Scheduling Problem* o *JSSP* (por sus siglas en inglés) y aunque no está claro quién debería llevarse el crédito por haber propuesto por primera vez este problema, se acepta que el libro “*Industrial Scheduling*”, editado en 1963 por Muth y Thompson (Muth, 1963), constituye la base para la mayoría de las investigaciones que siguieron.

B. Roy y B. Sussman en 1964 fueron los primeros en proponer la representación mediante el grafo disyuntivo (Roy y Sussman, 1964) y Egon Balas en 1969 (Balas, 1969), fue el primero en aplicar un acercamiento enumerativo basado en este grafo. Sin embargo existen trabajos anteriores: B. Giffler y G. L. Thompson (Giffler y Thompson, 1960) propusieron en 1960 un algoritmo de reglas de despacho de prioridad; J. R. Jackson en 1956 (Jackson, 1956) generalizó el algoritmo del *flow shop* de S. M. Johnson de 1954 (Johnson, 1954) al algoritmo del *job shop* y en 1955, S. B. Akers y J. Friedman (Akers y Friedman, 1955) aplicaron un modelo de álgebra booleana para representar secuencias de procesamiento.

1.2. El problema del taller mecánico

La programación de operaciones tiene como objetivo definir en forma concreta en que recurso de los disponibles se ejecutará cada una de las operaciones necesarias para la realización de las órdenes de trabajo emitidas y los instantes (fechas) en que tendrá lugar dicha ejecución. Se asocian a la programación de operaciones diferentes sub-funciones entre las que cabe señalar las de carga, secuenciación y temporización. En este problema se consideran únicamente dos de las subfunciones de la programación de operaciones: la secuenciación y la temporización, suponiendo que la carga haya sido realizada anteriormente. El prototipo de problema de secuenciación suele denominarse *problema del taller mecánico* (*Job Shop Scheduling Problem*). Es interesante notar como en cualquier problema del taller se pueden distinguir dos secuencias significativas: la ruta: secuencia establecida *a priori*, en que cada una de los trabajos pasa por las máquinas.

La secuencia: incógnita del problema, es la secuencia en que cada una de las máquinas recibe los trabajos.

Un taller de fabricación o *job shop* es una configuración productiva que permite fabricar una gran cantidad de productos diferentes. Este tipo de sistema se caracteriza por tener máquinas de propósito general, y porque cada trabajo tiene una ruta de fabricación única, que no necesariamente se repite entre trabajos. Un taller de maquinado constituye un buen ejemplo de una estructura *job shop*, sus máquinas (máquinas-herramientas), tales como: tornos, fresadoras, taladradoras, rectificadoras, etc., son de propósito general, ya que en ellas se pueden procesar una casi infinita gama de productos diferentes. En estos talleres cada producto puede tener una ruta diferente dentro del sistema; es decir, alguna orden irá primero al Torno para luego pasar por la Fresadora y por último a la Rectificadora, mientras que otra orden puede iniciar su proceso en la Fresadora, seguir en el torno y terminar en la rectificadora.

En otras palabras, el *JSSP* consiste en la programación temporal de las operaciones o tareas en la que se descompone un conjunto de trabajos y se tiene en cuenta que éstas deben ser ejecutadas en varias máquinas y que cada máquina solamente puede ejecutar una tarea simultáneamente. El objetivo perseguido en este problema es encontrar alguna planificación factible que

optimice alguna medida de desempeño, normalmente estas medidas para el JSSP son las siguientes: el tiempo total de producción (*makespan*), el tiempo de proceso total (*Flow time*), del tiempo de retraso total (*Lateness*), el tiempo de tardanza total (*tardiness*), entre otros.

De los anteriores, el objetivo más usado en la literatura es el de minimizar el *makespan* o C_{max} . Este objetivo es equivalente a minimizar los tiempos muertos, o a maximizar la utilización de las máquinas, y ésta es tal vez la razón por la cual ha sido abordado con mayor frecuencia por los investigadores.

1.3. Tipos de problemas de *scheduling*

Existen dos grandes familias de problemas de *scheduling*, que se dividen en cuanto a la demanda de recursos en un cierto tiempo y el suministro de dichos recursos en:

- Problemas de *scheduling* “puros”
- Problemas de asignación de recursos

En los problemas de *scheduling* “puros”, la capacidad de cada máquina (recurso) está definida para un tiempo determinado y el problema consiste en cubrir la demanda de máquinas necesarias para realizar las operaciones en el tiempo, sin exceder la capacidad disponible. En dependencia del uso de las máquinas por parte de las operaciones, se pueden distinguir tres patrones de flujo:

- Flujo aleatorio (*Open Shop Scheduling, OSS*): No existe ninguna restricción en cuanto al orden de uso de las máquinas por las operaciones de cada uno de los trabajos.
- Flujo general (*Job Shop Scheduling, JSS*): Configuración productiva donde cada trabajo es procesado en un conjunto de máquinas en un orden determinado. Un número de n trabajos deben ser procesados una sola vez por m máquinas, con un orden y durante un tiempo dado.
- Flujo regular (*Flow Shop Scheduling, FSS*): Todos los trabajos utilizan las máquinas en el mismo orden. Ninguna pieza visita una misma máquina más de una vez. Es un caso particular del *Job shop*.
- Flujo permutacional (*Permutational Flow Shop Scheduling, PFSS*): Es un caso particular del flujo regular en el cual, además, la secuencia de trabajos es la misma en todas las máquinas.

La siguiente figura (Figura 1.1) ilustra de forma general, las distintas variantes de problemas de *scheduling* que se pueden encontrar en un taller con tecnología de mecanizado. Se ha resaltado la variante de Flujo aleatorio (color rojo), debido a que no se presenta en talleres de este tipo, donde cada trabajo o pieza a fabricar posee una ruta tecnológica, la cual es conocida desde su llegada a dicho taller y además es invariable, y constituye una de las restricciones principales para las restantes variantes de problemas que si se presentan: flujo general (*Job Shop*), flujo regular (*Flow Shop*), y el flujo permutacional (*Permutational Flow Shop*).

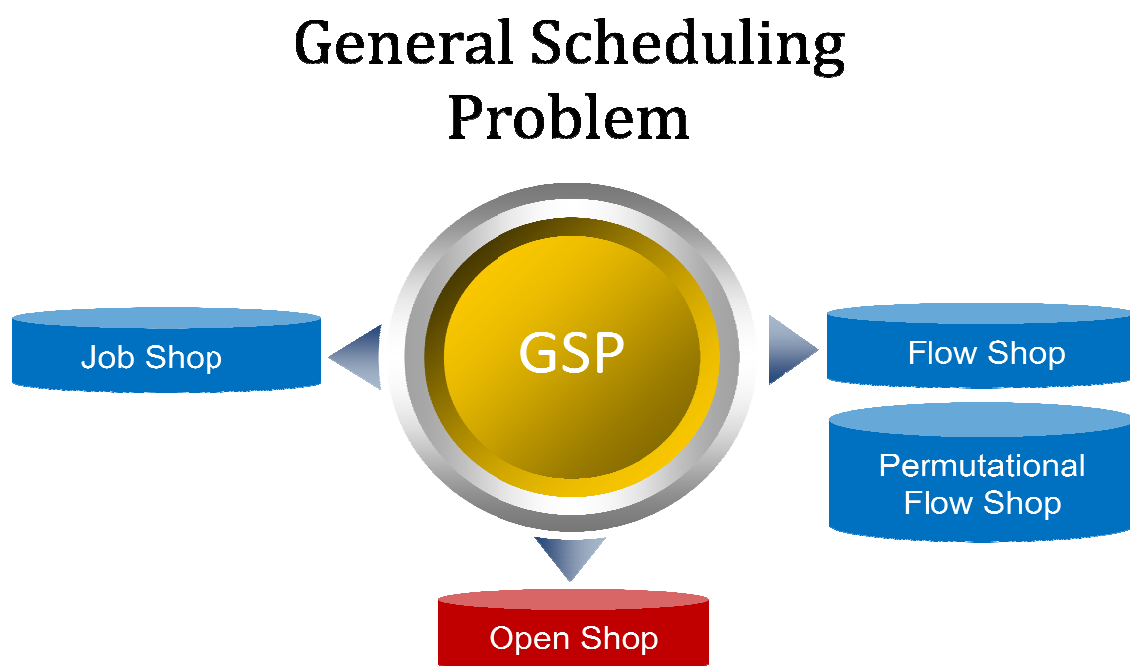


Figura 1.1. Variantes de problemas de *scheduling* según el patrón de flujo.

En los problemas de asignación de recursos, se dispone de un conjunto de operaciones, y para cada una de ellas se puede utilizar un conjunto de máquinas que realizan la misma operación, pero pueden requerir tiempos distintos o costos de procesamiento diferentes. En este caso, no se conoce con antelación qué máquina concreta va a utilizar cada operación, sino que el problema va a consistir en asignar las máquinas a tiempo para garantizar que se cumplan todas las demandas con el menor costo (tiempo, dinero, etc.).

En la solución propuesta solo se consideran los problemas de *scheduling* puros en instancias de tipo *job shop* y *flow shop*, por ser estos los que más aplicación

práctica ofrecen y los que más se presentan en la realidad, principalmente en un taller con tecnología para el maquinado de piezas.

Otra clasificación importante que tiene lugar en el taller es en problemas *estáticos* y problemas *dinámicos*.

Las características de los problemas estáticos son las siguientes:

- Los trabajos (piezas) son en número finito y determinado y deben realizarse en un taller con un número finito de máquinas.
- En el instante de realizar la programación es conocida la ruta de cada pieza, es decir, las operaciones que la componen en que máquina debe realizarse cada operación y la duración correspondiente.
- Todas las piezas están disponibles en el instante que habitualmente se adopta como instante inicial o instante cero en tiempo relativo.

Los problemas dinámicos presentan las siguientes características:

- El horizonte de funcionamiento del taller se considera ilimitado hacia el futuro, o sea, el número de máquinas se queda limitado, mientras el número de piezas es ilimitado.
- Todas las piezas que deberá tratar el taller en el futuro no estarán definidas en un momento determinado. La definición de las piezas se irá realizando a medida que va transcurriendo el tiempo.
- Progresivamente algunas piezas terminan su elaboración en el taller y lo abandonan, siendo sustituidas por otras que llegan para ser elaboradas.

Es evidente que los problemas dinámicos requieran un enfoque distinto respecto a los problemas estáticos (Fang, *et al.*, 1993; Bierwirth y Mattfeld, 1999). En esta situación no es suficiente realizar un único programa, porque esto resultaría poco eficaz al transcurrir el tiempo, por lo tanto es necesario restablecer un conjunto de programas sucesivos a lo largo del tiempo. (Lin, *et al.*, 1997).

En el presente trabajo solo se analizan los problemas estáticos, los cuales en la práctica, tienen lugar en un gran número de empresas fabricantes de piezas, conjuntos y equipos, que cuentan entre sus tecnologías, con talleres de maquinado.

1.4. Complejidad computacional

El *JSSP* es un problema de optimización catalogado como “problema no polinomial completo” (*NP-Hard*) (Papadimitriou, 1994) pues se trata de unos de los problemas de optimización combinatoria más difíciles de resolver. Los problemas *NP-Hard* son un subconjunto de la clase *NP* (problemas para los cuales no se puede tener una solución en tiempo polinomial para todas sus instancias). Un problema *NP-Hard* se presenta cuando un algoritmo que intenta solucionarlo, aumenta su tiempo de ejecución, en el peor de los casos de forma exponencial al tamaño del problema.

El problema de asignar cargas de trabajo a máquinas se cataloga como *NP-Hard*, pues se trata de unos de los problemas de optimización combinatoria más difíciles de resolver (Garey, et al., 1976). Una característica común a la mayoría de los problemas estudiados por la optimización combinatoria, es que suelen ser relativamente “fáciles” de plantear pero mucho más difíciles de modelar y, consecuentemente, mucho más difíciles de resolver. La complejidad del problema de secuenciar trabajos del tipo *job shop* radica en la cantidad abrumadora de posibles soluciones. Debido a que las operaciones a ser procesadas en una máquina forman la secuencia de operaciones para esa máquina, el plan de trabajo está formado por n secuencias de operaciones para cada máquina. Puesto que cada secuencia de operaciones puede ser permutada independientemente de la secuencia de operaciones de otra máquina, el número total de posibles soluciones para el *JSSP* es $(N!)^M$, donde N denota el número de trabajos y M el número de máquinas. Este problema no solo es del tipo *NP-Hard*, sino que de entre los que pertenecen a esta tipología, es uno de los más difíciles de resolver, de forma que no hay hasta el momento algoritmos determinísticos que lo resuelvan en forma eficiente (polinomial). Problemas de solo 10 trabajos y 10 máquinas han podido resolverse solo después de un período de 25 años (Schutten, 1998).

1.5. Descripción del *Job Shop Scheduling Problem*

El problema del *scheduling* puede ser enunciado tradicionalmente como sigue: se tiene un conjunto n de trabajos a ser procesados en m máquinas, en un orden previamente fijado, bajo ciertas restricciones asumidas. El objetivo del

job shop scheduling es organizar de manera óptima el orden de procesamiento y los tiempos de inicio de las operaciones para optimizar un determinado criterio (medida de desempeño). En general, se tienen dos tipos de restricciones en este tipo de problema. El primer tipo de restricción expresa que las relaciones de precedencia entre las operaciones de un trabajo deben ser garantizadas y se conoce como restricción de secuencia. El segundo tipo de restricción refiere que no más de un trabajo puede ser ejecutado en una máquina al mismo tiempo y se conoce como restricción de recurso. Un problema de *scheduling* se considera completamente solucionado si los tiempos de inicio de todas las operaciones son determinados y las restricciones de secuencia y recurso no son violadas.

1.5.1. Formulación matemática del problema

Las notaciones definidas por la conveniencia de la formulación del problema de *scheduling* son como sigue: denominamos $N = \{1, \dots, n\}$ y $M = \{1, \dots, m\}$ como el conjunto de trabajos y el conjunto de máquinas, donde n y m son el número de trabajos y máquinas respectivamente. Entonces n_i será el número de la operación del trabajo i . O_{ikq} representa la operación k del trabajo i a ser procesado en la máquina q , S_{ikq} y T_{ikq} representan el tiempo de inicio y el tiempo de procesamiento de O_{ikq} respectivamente. S_{ieiq} y T_{ieiq} representan el tiempo de inicio y el tiempo de procesamiento de la última operación del trabajo, respectivamente. Denotando r_i y d_i como la fecha de llegada (el tiempo de comienzo más temprano) y la fecha de compromiso (el tiempo de terminación final). P_i denota el conjunto de pares de operaciones $[O_{ikp}, O_{ilq}]$ con restricción de precedencia del trabajo i , donde la operación O_{ikp} debe preceder a la operación O_{ilq} y Q_i denota el conjunto de pares de operaciones $[O_{ikp}, O_{ilq}]$ sin restricción de precedencia del trabajo i , donde la operación O_{ikp} y la operación O_{ilq} del trabajo i pueden ser procesadas en cualquier orden. R_q es el conjunto de operaciones O_{ikq} que será procesado en la máquina q . Se asume además, que las operaciones no pueden ser interrumpidas una vez iniciadas. Tomando la minimización del *makespan* como criterio de optimización, la formulación matemática del JSSP considerada sería la siguiente:

Minimizar: $Z = \max_{i \in N} (S_{ie_{iq}} + T_{ie_{iq}})$

Sujeto a:

$$S_{ilq} - S_{ikp} \geq T_{ikp}, \quad [O_{ikp}, O_{ilq}] \in P_i$$

$$k, l \in \{1, \dots, n_i\}, \quad i \in N \quad (1)$$

$$S_{ilq} - S_{ikp} \geq T_{ikp}, \text{ o } S_{ikp} - S_{ilq} \geq T_{ilq}$$

$$[O_{ikp}, O_{ilq}] \in Q_i, \quad k, l \in \{1, \dots, n_i\}, \quad i \in N \quad (2)$$

$$S_{jlq} - S_{ikq} \geq T_{ikq}, \text{ o } S_{ikq} - S_{jlq} \geq T_{jlq}$$

$$O_{ikq}, O_{jlq} \in R_q, \quad i, j \in N, \quad q \in M \quad (3)$$

$$S_{ijq} \geq r_i$$

$$i \in N, j \in \{1, \dots, n_i\}, \quad q \in M \quad (4)$$

$$S_{ijq} \leq d_i - T_{ijq}$$

$$i \in N, j \in \{1, \dots, n_i\}, \quad q \in M \quad (5)$$

Donde:

(1) significa que dos operaciones de un mismo trabajo con restricción de precedencia no pueden ser procesadas al mismo tiempo, solo de acuerdo con dicha restricción.

(2) significa que dos operaciones sin restricción de precedencia, de manera disyuntiva, no se pueden solapar en el tiempo.

(3) significa que una máquina, de manera disyuntiva, solamente puede procesar una operación en el tiempo. Representa una restricción de recursos.

(4) significa la restricción disponibilidad.

(5) significa la restricción de fecha de cumplimiento.

El valor de la función Z es el tiempo de finalización de la última operación.

Minimizar Z significa Minimizar el *makespan*.

1.5.2. Restricciones del modelo de optimización

- Hay solo una máquina de cada tipo, no existiendo varias máquinas para realizar una misma operación.
- Una máquina puede procesar solamente un trabajo en un solo instante.
- Las restricciones tecnológicas (ruta tecnológica) son conocidas e invariables.
- Cada trabajo es una entidad, y por lo tanto, no pueden procesarse dos operaciones de un mismo trabajo simultáneamente.
- No existe interrupción, es decir, cada operación una vez comenzada debe ser completada antes de que otra operación pueda hacerlo en la misma máquina.
- Cada trabajo incluye una y solo una operación en cada máquina, por lo que todos los trabajos contienen una cantidad de operaciones no mayor al número de máquinas. No existe recirculación.
- Los tiempos de proceso son independientes de la secuencia seguida, lo que excluye tiempos de ajuste en las máquinas según la secuencia de los trabajos considerada o tiempos de transporte entre máquinas.
- Son conocidos y fijos todos los datos que intervienen: número de trabajos, número de máquinas, tiempos de proceso, entre otros.

El problema a resolver consiste en asignar tareas a recursos durante un tiempo determinado de la mejor manera posible. A ser posible, de forma óptima.

1.6. Nomenclatura

Diferentes características de los trabajos y de las máquinas, junto con distintos criterios de optimización, originan una gran variedad de modelos de planificación que necesitan ser catalogados de alguna forma. En este sentido, es posible distinguir entre varios intentos por describir la amplia gama de estos problemas. Una de las más difundidas es sin dudas la nomenclatura que tiene en cuenta tres componentes propuesta por Graham y colaboradores en 1979 (Graham, *et al.*, 1979). La clasificación triparamétrica del problema consta de tres variables $\alpha|\beta|\gamma$, las cuales representan la siguiente información. El campo α describe el ambiente de máquina y contiene una única entrada. El campo β

proporciona detalles de las características de procesamiento y restricciones; puede tener una única entrada, múltiples entradas o ninguna entrada. El campo γ identifica el objetivo a ser optimizado y usualmente contiene una única entrada. Este esquema sirve tanto para modelar los problemas como para catalogar los diferentes modelos de planificación. Dicho esquema ha sido ampliamente aceptado entre los investigadores y por la literatura especializada pues, además, tiene la ventaja de que los nuevos problemas de planificación que se han investigado a lo largo de los años y que se siguen investigando hoy en día, se han ido incorporando a las versiones extendidas y ampliadas de dicha clasificación, por lo que se considera que la clasificación triparamétrica $\alpha|\beta|\gamma$ es una clasificación abierta y así lo confirman, entre otros, los trabajos de otros investigadores (Lawler, *et al.*, 1993; Blazewicz, *et al.*, 2007; Brucker, 2007; Pinedo, 2008). Otras nomenclaturas han sido utilizadas para describir problemas de secuenciamiento, tales como (Conway, *et al.*, 1967; Lawler, *et al.*, 1982; Vignier, *et al.*, 1999).

En la presente investigación se asume la clasificación triparamétrica propuesta por Graham.

1.6.1. Características de las máquinas. Campo α

Se puede distinguir los problemas de planificación en los que se dispone de una única máquina ($\alpha = 1$) de aquellos problemas en los que se puede utilizar varias máquinas. Desde el momento en que se dispone de dos o más máquinas cabe la posibilidad de que varias de ellas ejecuten trabajos distintos u operaciones de diferentes trabajos simultáneamente, y entonces es posible hablar de *paralelismo*. También conviene distinguir los problemas en los que las máquinas ejecutan las mismas funciones (máquinas *no especializadas*) de aquellos problemas en los que ciertas máquinas están especializadas en ciertas tareas y no pueden realizar otras (máquinas *especializadas*).

Los posibles ambientes de máquina que se pueden especificar en el campo α son:

- *Máquina única (1)*: Es el ambiente de máquina más simple y es el caso especial de todos los demás ambientes de máquina.
- *Máquinas en paralelo idénticas (P_m)*: Hay m máquinas idénticas en paralelo. Una operación de un trabajo j se puede procesar en cualquiera de las m

máquinas. Si no se permite el procesamiento del trabajo j sobre cualquier máquina o sobre alguna perteneciente a un subconjunto dado, entonces en el campo β aparece la entrada M_j .

- *Máquinas en paralelo uniformes (Q_m)*: Hay m máquinas en paralelo con diferentes velocidades; v_i indica la velocidad de la máquina i . El tiempo de permanencia p_{ij} del trabajo j en la máquina i es p_j/v_i , asumiendo que solo se procesa en la máquina i . Si todas las máquinas tienen la misma velocidad, es decir, $v_i = 1$ para todo i y $p_{ij} = p_j$, entonces este ambiente es idéntico al previo.
- *Máquinas en paralelo diferentes o no relacionadas (R_m)*: Este ambiente es una generalización del anterior. Hay m máquinas diferentes en paralelo. La máquina i puede procesar el trabajo j a una velocidad v_{ij} . El tiempo de permanencia p_{ij} del trabajo j sobre la máquina i es p_j/v_{ij} , asumiendo que solo se procesa sobre la máquina i . Si la velocidad de las máquinas es independiente de los trabajos, es decir, $v_i = v_{ij}$ para todo i y j , entonces el ambiente es idéntico al anterior.
- *Flow shop (F_m)*: Hay m máquinas en serie, cada trabajo se procesa en cada una de ellas. Todos los trabajos tienen la misma trayectoria, es decir, primero se procesan sobre la máquina 1, luego sobre la máquina 2, y así sucesivamente. Cuando un trabajo deja de usar una máquina se agrega a la cola de la próxima máquina. Usualmente se asume que todas las colas trabajan bajo la disciplina *primero en entrar primero en salir (FIFO)*. Si se aplica la disciplina FIFO al *flow shop*, se le denomina *flow shop* con permutación y el campo β incluye la entrada pmu .
- *Flexible Flow shop (FF_s)*: Es una generalización del *flow shop* y del ambiente de máquinas paralelas. En lugar de m máquinas en serie, hay s etapas en serie con una determinada cantidad de máquinas en paralelo cada una de ellas. Cada trabajo se procesa primero en la etapa 1, luego en la etapa 2, y así sucesivamente. Cada etapa funciona como un banco de máquinas paralelas; en cada etapa el trabajo j necesita solo una máquina y cualquier máquina puede procesar cualquier trabajo. Las colas entre las distintas etapas trabajan bajo una disciplina *FIFO*.

- *Open shop* (O_m). Hay m máquinas y n trabajos. Cada trabajo se debe procesar en cada una de las m máquinas. No hay restricción en relación a la ruta de cada trabajo a través del ambiente de máquinas. El plan determina la ruta de cada trabajo, diferentes trabajos pueden tener distintas rutas.
- *Job shop* (J_m). Hay m máquinas y n trabajos. Cada trabajo tiene predeterminada su ruta. Se hace una distinción cuando los trabajos visitan las máquinas solo una vez y cuando los trabajos visitan las máquinas más de una vez. En el último caso, el campo β contiene la entrada *recrc* para indicar recirculación.

1.6.2. Características de los trabajos. Campo β

Las restricciones de procesamiento especificadas en el campo β pueden incluir múltiples entradas. Las entradas posibles son:

- *Release time* (r_i). Si este símbolo está presente en el campo β , el trabajo j no puede empezar su procesamiento antes de su fecha de disponibilidad (*release date* r_j), lo cual indica un *scheduling* dinámico. Si r_j no aparece en el campo β , el procesamiento del trabajo j puede comenzar en cualquier momento, indicando un *scheduling* estático. En contraste con el *release date*, los tiempos de entrega no se especifican en este campo. El tipo de función objetivo da información suficiente si se consideran tiempos de entrega o no.
- *Setup times*. Tiempos de dependientes de la secuencia (s_{jk}). s_{jk} representa el tiempo de configuración (*setup time*) dependiente de la secuencia de los trabajos j y k ; s_{0k} indica el tiempo de setup para el trabajo k si es el primero en la secuencia y s_{j0} hace referencia al tiempo de limpieza (*clean-up*) luego del trabajo j si éste es el último de la secuencia (tanto s_{0k} como s_{j0} pueden ser igual a 0).
- *Preemptions*. Significa que los trabajos no pueden ser interrumpidos durante el proceso. Se indica con la entrada *prmp*.
- *Precedence constraints*. Uno o más trabajos pudieran tener que ser completados antes que a otro trabajo se le permita iniciar su proceso. Puede ser representado por un grafo acíclico dirigido $G = (V, A)$, $V = \{1, \dots, n\}$ corresponde a los trabajos (vértices), $(j, k) \in A$ (aristas). Si el j th trabajo debe ser completado antes que k th, las cadenas de cada trabajo tienen un

predecesor y un sucesor a lo sumo. El indicativo *intree* significa que cada trabajo tiene por lo menos un sucesor. El indicativo *outtree* significa que cada trabajo tiene por lo menos un predecesor.

- *Breakdowns*. Las máquinas no están disponibles de forma continua. Se indica con la entrada *brkdown*.
- *Permutation*. En el ambiente *flow shop* las colas delante de cada máquina operan según la disciplina FIFO. Se indica con la entrada *prmu*.
- *Blocking*. En el *flow shop* hay un *buffer* limitado entre dos máquinas sucesivas. Cuando el *buffer* está lleno, la máquina anterior es bloqueada y no se le permite liberar un trabajo completado hasta tanto no se descongestiona la máquina siguiente. Se indica con la entrada *block*.
- *No wait*. No se permite a los trabajos esperar entre dos máquinas sucesivas. Se indica con la entrada *no-wait*.
- *Recirculation*. En el ambiente *job shop* un trabajo puede visitar una máquina en más de una ocasión. Se indica con la entrada *recrc*.

1.6.3. Criterios de optimalidad. Campo y

El tercer y último campo expresa el número de funciones objetivo a considerar, sus características y, en caso de más de un criterio, el tipo de optimización en el que se está interesado, es decir, si se buscan puntos eficientes, puntos extremos, realizar una optimización simultánea o una optimización jerárquica.

El objetivo puede ser minimizar una función de tiempos de finalización de los trabajos, los cuales dependen de la planificación. El tiempo de finalización de la operación de un trabajo j sobre la máquina i se denota como C_{ij} . El tiempo en el que el trabajo j sale del sistema (*completion time*) se indica como C_j . La demora (*lateness*) de un trabajo j se define como:

$$L_j = C_j - d_j \quad (6)$$

La cual es positiva cuando el trabajo j finaliza tarde y negativo cuando se completa en forma temprana. La tardanza (*tardiness*) de un trabajo j se define como:

$$T_j = \max(C_j - d_j, 0) = \max(L_j, 0) \quad (7)$$

La diferencia entre la tardanza y la demora (retraso) es que la primera nunca es negativa. La unidad de penalidad del trabajo j se define como:

$$U_j = \begin{cases} 1 & \text{si } C_j > d_j \\ 0 & \text{en otro caso} \end{cases} \quad (8)$$

1.7. Métodos de resolución

Los problemas de *scheduling* son una subclase de los problemas con satisfacción de restricciones (*Constraint Satisfaction Problems, CSP*) que aparecen con frecuencia en la realidad. Al clasificar como NP-duros se han aplicado todas las técnicas de Inteligencia Artificial, con mayor o menor éxito. En estos problemas existe un conjunto finito de soluciones, por lo tanto en teoría, bastaría evaluar cada una de ellas con respecto al objetivo perseguido, para encontrar la solución óptima. Sin embargo, en la realidad este procedimiento no es viable, ya que la complejidad de la mayoría de los problemas de las empresas requeriría tiempos de resolución excesivamente largos. En las secciones siguientes se revisan los métodos más utilizados y se tiene en cuenta sus ventajas y desventajas.

1.7.1. Métodos exactos

Estos métodos son los que permiten obtener la solución óptima de un problema. Una solución es óptima si es la mejor de todas las soluciones posibles, por lo tanto se podría pensar, que los métodos exactos representan la mejor opción para la resolución de los problemas de *scheduling*, pero en la realidad solo son aplicables a instancias de tamaño muy reducido, ya que el tiempo de cómputo requerido aumenta exponencialmente con su dimensión. También hay que añadir que algunos procedimientos desarrollados son más apropiados para determinados criterios, como es el caso de los que se fundamentan en el concepto de camino crítico.

Con el fin de encontrar el máximo o mínimo de una función objetivo, el acercamiento más obvio sería de naturaleza analítica, utilizando sus derivadas.

Es

importante que la función y su primera derivada sean continuas. Estos métodos pueden ser tan sólo utilizados para pequeños problemas que contengan numerosas variables continuas con funciones objetivo simples. Por este motivo, este método no es aplicable a los problemas de secuenciación (*job shop scheduling*).

Algunos de estos métodos son los siguientes:

Enumeración total: es el método más sencillo para solucionar el problema del flujo de línea del taller mecánico es el de enumerar todas las posibles soluciones y evaluarlas. Este método no puede considerarse muy eficiente como se ha descrito anteriormente dado que su complejidad computacional asciende vertiginosamente.

Ramificación y poda: el método exacto más relevante es el algoritmo de ramificación y poda (*Branch and Bound*) propuesto por Bruker (Brucker, *et al.*, 1994), desarrollado a partir de conceptos y técnicas propuestas también por otros investigadores como Carlier y Pinson (Carlier y Pinson, 1989), y forma parte de los algoritmos de enumeración implícita, ya que “enumera” todas las soluciones posibles hasta llegar a la óptima, aunque por el camino se van descartando soluciones, demostrando *a priori* su no optimización. En este tipo de algoritmo el espacio de las soluciones se divide en unas ramas (fase de ramificación), cada rama se evalúa, y se continúa investigando solo sobre las ramas susceptibles de llegar a soluciones óptimas (fase de poda).

Programación entera y entera mixta: encuentran su aplicación en problemas que consideran que todas o algunas de sus variables han de tomar obligatoriamente valores enteros. Los métodos más conocidos para resolver los problemas de programación entera son las derivadas y el *branch and bound*. Cotas más bajas, en el caso de la minimización, se pueden conseguir mediante la relajación del problema de programación lineal. Constituye otro enfoque posible ya que casi la totalidad de los problemas combinatorios admiten una formulación como programas lineales enteros o binarios, y especialmente como mixtos. El inconveniente de este método reside en la gran cantidad de variables que surgen en la mayoría de los problemas.

1.7.2. Métodos heurísticos

Debido a la imposibilidad de utilizar algoritmos exactos para la resolución de problemas combinatorios comunes, ya que hasta el momento no ha sido posible encontrar formulaciones que, acotadas en un tiempo polinomial de cálculo, brinden una solución determinística que sea óptima global, las investigaciones se han dirigido hacia el desarrollo de procedimientos heurísticos (MacCarthy y Liu, 1993; Sadeh y Fox, 1996; Tsai y Lin, 2003; Wang y Yu, 2010) y metaheurísticos, que sacrifican la optimización de la solución a

cambio de tiempos de cálculo aceptables. Los problemas de planificación han encontrado en ellos un vasto campo de aplicación por cumplir una o más de las siguientes circunstancias:

- Cuando no se puede aplicar un método exacto para la resolución de un problema, por no existir o porque requiera demasiados recursos (tiempo de cálculo, memoria, presupuesto, entre otros.)
- Cuando el problema o la solución planteada no requieren una solución exacta o precisa, sino solo aproximada.
- Cuando los datos disponibles son poco fiables.
- Como un paso intermedio para la aplicación de otros algoritmos. Muchas veces la solución aportada por un método heurístico es tomada como punto de partida de otros procedimientos (principalmente metaheurísticos).
- Cuando las limitaciones de tiempo, espacio, entre otros, conducen a la utilización de métodos de respuesta rápida aunque sea a costa de la precisión.

Existen diversos algoritmos heurísticos que se pueden clasificar en:

Métodos de descomposición: consisten en dividir el problema en subproblemas más pequeños, siendo la salida de uno la entrada de otro, de forma que al resolver ambos, se obtiene la solución para el problema global.

Métodos de reducción: estos métodos identifican alguna característica que deba poseer la solución óptima y de este modo simplifican el problema.

Métodos de manipulación del modelo: son aquellos que modifican las estructuras del modelo con el fin de hacerlo más sencillo de resolver, deduciendo a partir de la solución del problema modificado, la solución del original.

Métodos de búsqueda local: parten de una solución factible inicial, probablemente obtenida a partir de otra heurística, y mediante alteraciones a la solución, van iterando hacia otras factibles de su entorno, almacenando la mejor solución encontrada hasta que se cumpla una determinada condición de parada.

Algoritmos como los que siguen son ejemplos del funcionamiento de algunos procedimientos. El algoritmo 1.1 muestra la estructura de Algoritmo Evolutivo (AE) el cual tiene un carácter probabilista y mantiene una población de

individuos o cromosomas, $P(t)$, en cada iteración t . Cada uno de los individuos constituye una solución potencial del problema, representada comúnmente a través de una cadena de símbolos. Los individuos se evalúan mediante la función de actitud (*fitness function*) que determina la calidad de la solución o grado de adaptación del individuo al medio. Posteriormente, se construye una nueva población ($t + 1$) comenzando con un proceso de selección a partir de la población $P(t)$, de manera que los mejores individuos presentan mayor probabilidad de ser seleccionados. A continuación se modifica esta nueva población mediante la aplicación sobre algunos individuos de los operadores genéticos (cruce y mutación). Un caso particular de los AEs son los Algoritmos Genéticos (AGs), los cuales son analizados más en detalles en la sección 1.18 por constituir la solución aplicada en esta investigación.

Algoritmo 1.1 Algoritmo Evolutivo.

```
1  Leer_parámetros(N_Gen, T_pob, Fc, Fm)
2  t = 0;
3  Iniciar(P(t));
4  Evaluar(P(t));
5  mientras (N_Gen)
6      t = t + 1;
7      P'(t) = Selección(P(t - 1));
8      Método_Selección();
9      P''(t) = Modificar(P(t'));
10     Método_Cruzamiento();
11     Método_Mutación();
12     P(t) = Aceptar(P'(t), P''(t));
13     Método_Aceptación();
14 fin mientras
FIN
```

En el caso de la búsqueda local, como muestra el algoritmo siguiente (Algoritmo 1.2) se parte de una solución inicial y en cada iteración se calcula un conjunto de soluciones vecinas mediante una regla de vecindad. Cada una de las soluciones es necesario que se evalúe, siendo ésta una de las acciones más críticas del algoritmo por el elevado tiempo de ejecución que presupone. A continuación se selecciona una de las soluciones vecinas con un determinado criterio. Si esta solución cumple el criterio de aceptación, la solución seleccionada reemplaza a la solución actual S , y el proceso continúa hasta que se cumpla la condición que indica la finalización. Generalmente ocurre que la condición se agote para un determinado número de iteraciones, o bien que no se produzcan mejoras en los últimos intentos. Normalmente la búsqueda local no mantiene traza de todo el proceso realizado hasta el momento, sino que solamente utiliza información sobre el estado actual y sus vecinos. A pesar de esto, la búsqueda local es una estrategia muy eficaz en muchos casos, ya que realizan una búsqueda en un espacio de soluciones potenciales del problema tratando de encontrar aquella que maximice, o minimice, una determinada función objetivo. Además se puede combinar con otras estrategias, como por ejemplo los algoritmos evolutivos descritos anteriormente.

Algoritmo 1.2 Algoritmo de Búsqueda local.

```
1  S = Solución inicial
2  mientras (Condición_terminación)
3      V = Soluciones_Vecinas(S);
4  Evaluar_Soluciones(V);
5      S' = Seleccion(V);
6      Si (Condición_aceptación)
7          S = S';
8      fin si
9  fin mientras
10 Devuelve S;
FIN
```

El *JSSP* aparece con profusión en la literatura científica como ejemplo de prueba de muchas técnicas de resolución de problemas. Hay también disponibles numerosos bancos de uso común entre los investigadores, lo que facilita el contraste de resultados experimentales. En (Jain y Meeran, 1998), Jain y Meeran ofrecen una clasificación de las principales técnicas que han sido aplicadas para resolver el *JSSP*.

1.7.3. Métodos metaheurísticos

Las metaheurísticas son estrategias inteligentes para diseñar o mejorar procedimientos heurísticos muy generales con un alto rendimiento. El término metaheurística fue dado a conocer por primera vez por Fred Glover en 1986 (Glover, 1986), y desde entonces han surgido multitud de propuestas para diseñar buenos procedimientos que den solución a determinados problemas, adoptando la denominación de metaheurística. La base de los procedimientos metaheurísticos la constituyen los métodos constructivos y los de búsqueda local. Algunos de los tipos fundamentales son: las de relajación, las constructivas, las de búsqueda y las evolutivas.

- Las metaheurísticas de relajación se refieren a procedimientos de resolución de problemas que utilizan relajaciones del modelo original, mediante modificaciones que lo hacen más fácil de resolver, cuya solución facilita la solución del problema original. Entre estas se encuentran los métodos de relajación lagrangiana (Beasley, 1993; Guinard, 2002).
- Las metaheurísticas constructivas son procedimientos que consiguen una solución a partir del análisis y selección paulatina de las componentes de una estructura, inicialmente vacía, que representa la solución.
- Las metaheurísticas de búsqueda guían los procedimientos que usan transformaciones o movimientos para recorrer el espacio de soluciones alternativas y explotar las estructuras de entornos asociadas.
- Las metaheurísticas evolutivas están enfocadas a los procedimientos basados en conjuntos de soluciones que evolucionan sobre el espacio de dichos conjuntos.

Algunas metaheurísticas surgen de combinaciones de distintos tipo, tal es el caso de la metaheurística *GRASP* (*Greedy Randomized Adaptive Search*

Procedure) (Resende y González-Velarde, 2003; Resende y Ribeiro, 2003), que combina una fase constructiva con una fase de búsqueda de mejora.

De forma general, todas las metaheurísticas se pueden considerar como estrategias aplicadas a procesos de búsqueda, donde todas las soluciones intermedias en el proceso de resolución del problema, se interpretan como elementos de un espacio de búsqueda, que se van modificando a medida que se aplican las distintas operaciones diseñadas para llegar a la solución definitiva. Por ello, y porque los procesos de búsqueda heurística constituyen el paradigma central de las metaheurísticas, es frecuente interpretar que el término metaheurística es aplicable esencialmente a los procedimientos de búsqueda sobre un espacio de soluciones alternativas. La evaluación del rendimiento de una metaheurística debe atender tanto a la eficiencia como a la eficacia de los procedimientos heurísticos obtenidos. Para medir su eficacia, éstas deben afrontar con éxito problemas de un banco de casos para los que se conozcan las soluciones. Si no se dispone de estos casos, se deben construir recurriendo a procesos de simulación que se aproximen a tales circunstancias. La eficiencia del método se comprueba experimentalmente en el empleo de un tiempo computacional moderado (razonable) para alcanzar éxito en los problemas tratados.

1.7.4. Métodos de aproximación

Los métodos de aproximación no garantizan una solución exacta, pero son capaces de obtener soluciones muy cercanas al óptimo, con unos tiempos de cálculo moderados, y por ello son útiles para problemas grandes. Su importancia radica en que las heurísticas inspiradas en fenómenos naturales y resolución inteligente, resultan muy apropiadas para problemas combinatorios difíciles, proporcionando un adecuado enlace bilateral entre la investigación de operaciones y la inteligencia artificial. Entre los más representativos se encuentran los siguientes:

Reglas de prioridad de despacho (*Priority dispatch rules*): una de las estrategias más comunes para resolver problemas de *scheduling* es el uso de reglas de despacho, las cuales permiten definir prioridades entre los trabajos que se encuentran en un taller. Pueden ser sencillas, basadas en un dato del

producto, como el tiempo de procesamiento o la fecha de entrega; también se pueden obtener a través de cálculos entre diferentes variables, como en el caso de la holgura. Se trata en general de algoritmos voraces que toman decisiones basadas en algún criterio de optimización local. En la literatura (Rajendran y Zeigler, 2001; Pinedo, 2008) se puede encontrar una amplia variedad de este tipo de estrategias heurísticas para problemas de *scheduling*, destacándose, entre otras, las siguientes:

- **FIFO** (*First In First Out*): basada en atender primero al primer trabajo en entrar en el proceso. Se emplea a menudo y, especialmente, con productos perecederos, donde adquiere el nombre **FEFO** (*First Expiration First Out*).
- **LIFO** (*Last In First Out*): basada en atender primero al último trabajo en entrar en el proceso. No es muy común, pero en ocasiones, cuando el material ocupa grandes superficies y su movimiento por las distintas áreas del taller produce interrupciones adicionales en otras máquinas, suele ser útil esta regla.
- **SIRO** (*Service In Random Order*): bajo esta regla no se intenta realizar ninguna optimización.
- **STP** (*Shortest Processing Time*): Ordena los trabajos de menor a mayor tiempo de procesamiento, es decir, conforme a la duración. Corresponde a la regla “ejecutar primero la operación más corta” y permite minimizar el índice de eficiencia F_{med} y el stock medio. Es una de las más utilizadas. Los trabajos se procesan en orden creciente de tiempos de proceso p_i .

$$p_1 \leq p_2 \leq \dots \leq p_n$$

- **LTP** (*Longest Processing Time*): Ordena los trabajos de mayor a menor tiempo de procesamiento, es decir, conforme a la duración. Corresponde a la regla “ejecutar primero la operación más larga”. Los trabajos se procesan en orden decreciente de tiempos de proceso p_i .

$$p_1 \geq p_2 \geq \dots \geq p_n$$

- **EDD (Earliest Due Date)**: Ordena los trabajos en función de la fecha de vencimiento, de forma creciente, es decir, el primer trabajo de la lista es el que tiene menor fecha de entrega. Corresponde a la regla “ejecutar primero lo más urgente” y permite minimizar T_{max} . Los trabajos se procesan en orden creciente a la fecha de vencimiento d_i .

$$d_1 \leq d_2 \leq \dots \leq d_n$$

- **ECT (Earliest Completion Time)**: Ordena los trabajos en función de la fecha de terminación, de forma creciente, es decir, el primer trabajo de la lista es el que tiene menor fecha de terminación. Corresponde a la regla “ejecutar primero lo que acaba antes”. Los trabajos se procesan en orden creciente a la fecha de terminación C_i .

$$C_1 \leq C_2 \leq \dots \leq C_n$$

- **Holgura mínima**: Considera el tiempo restante total hasta la finalización del trabajo. De esta forma se programan antes los trabajos con mayores posibilidades de retrasarse.

Heurísticas basadas en cuellos de botella (Bottleneck based heuristics): un ejemplo de esta aproximación es el *Shifting Bottleneck Procedure (SBP)* (Adams, et al., 1988), el cual tiene como estrategia relajar el problema de *scheduling* $n \times m$, en m problemas de 1 máquina y resolver cada subproblema cada vez de forma iterativa. Cada una de las soluciones se compara con las demás y se ordenan las máquinas según su solución. La máquina no secuenciada que tiene la solución mayor se identifica como la máquina que provoca el cuello de botella. *SBP* secuencia dicha máquina basándose en las que ya han sido calculadas, ignorando el resto de las máquinas no secuenciadas. La selección de la máquina que constituye el cuello de botella viene motivada por la suposición de que el *scheduling* en etapas posteriores podría deteriorar el tiempo de realización del *scheduling* (*makespan*).

Métodos de búsqueda local (Local Search Methods): este procedimiento está basado (Vaessens, et al., 1994) en la suposición de que es posible encontrar varias soluciones entre la solución inicial y la final tal que cada una

de ellas es ligeramente diferente a la inmediata anterior. Un concepto asociado a la búsqueda local es la descripción del conjunto de soluciones parecidas, denominado “vecindario”. Dos soluciones son “vecinas” si una puede ser obtenida a partir de la otra intercambiando el orden entre dos tareas consecutivas en un mismo recurso, perteneciendo ambas al camino crítico (*critical path*). Este procedimiento puede ofrecer una medida de la bondad de la solución encontrada, pero no garantiza que la solución obtenida sea el óptimo global del problema considerado. Tiene la ventaja de que en poco tiempo puede encontrar soluciones suficientemente buenas para un conjunto amplio de problemas de naturaleza combinatoria.

Clausura y satisfacción de restricciones (CSP): debido a que los problemas de *scheduling* se caracterizan por un conjunto de restricciones temporales, las técnicas desarrolladas para los problemas de satisfacción de restricciones, y más concretamente, sobre restricciones temporales, pueden aplicarse para resolver problemas de *scheduling*. En este sentido, se utilizan dos procesos típicos: la *clausura*, como técnica deductiva de nuevas restricciones derivadas, y técnicas de búsqueda CSP, guiadas por heurísticas.

1.7.5. El Algoritmo G&T

El algoritmo de Giffler y Thompson (Giffler y Thompson, 1960), conocido como algoritmo G&T, se puede considerar como la base de todas las heurísticas basadas en reglas de prioridad. Giffler y Thompson en su propuesta crearon procedimientos para generar planes de trabajo (*schedules*) activos y sin retardo (en la sección 2.10 se describen los tipos de planes de trabajo). Este es un algoritmo voraz que produce planificaciones activas en $N \times M$ pasos, es al mismo tiempo no determinista y se puede particularizar de muchas formas. Su estrategia se muestra en el siguiente algoritmo.

Algoritmo 1.3 Algoritmo G&T.

```
1 A = Conjunto inicial de tareas no planificadas
2 mientras ( $A \neq \emptyset$ )
3   Determinar la tarea  $\theta \in A$  con el menor tiempo de fin;
4   Construir el conjunto  $B$  con las tareas de  $A$  que requieren el recurso  $R_\theta$ , y pueden comenzar antes
   del tiempo de fin de la tarea  $\theta$ ;
5   (se obtiene una planificación activa)
6   Seleccionar  $\theta^*$  de  $B$  con algún criterio y planificarla;
7   Seleccionar  $\theta^*$  de  $A$  y adicionar a  $A$  la sucesora de  $\theta^*$  si no es la última tarea del trabajo;
8 fin mientras
FIN
```

Debido a que en problemas de gran tamaño el espacio de búsqueda que forman todas las planificaciones activas es excesivamente amplio, la búsqueda debe restringirse a una parte de este espacio por razones de eficiencia. En este sentido, una posibilidad es emplear una variante del algoritmo G&T denominada *G&T Híbrido*. (Storer y Talbot, 1992; Bierwirth y Mattfeld, 1999)

Un resumen que recoge los procedimientos más establecidos (se ha mantenido su acepción en inglés, siguiendo el uso habitual) y una representación de investigadores que han hecho uso de los mismos, es el siguiente:

- *Logic Programming* (Figlali, et al.) (Hentenryck y Simonis, 1992).
- *Automatic Learning* (AL) (Zweben, et al., 1992).
- *Branch and Bound* (B&B) (Brucker, et al., 1994; Allahverdi y Al-Anzi, 2006; Blazewicz, et al., 2007; Artigues y Feillet, 2008; Artigues, et al., 2009).
- *Constraint Programming* (CP) (Dorndorf, et al., 2002; Farber, et al., 2007).
- *Ant Colony Optimization* (ACO) (Dorigo y Stützle, 2003; Xing, et al., 2010).
- *Evolutionary Algorithms* (EA) (Kacem, et al., 2002).
- *Estimation Distribution Algorithms* (Montoya Torres, et al.) (Larrañaga y Lozano, 2002; Larrañaga, et al., 2003).
- *Fuzzy Adaptive Neighborhood Search* (FANS) (Yun, 2002).
- *Genetic Algorithms* (GA) (Goldberg, 1989; Syswerda, 1991; Fang, et al., 1993; Bierwirth, 1995; Yamada y Nakano, 1995; Chen y Gen, 1996; Michalewicz, 1996; Yamada y Nakano, 1997; Yamada, 2003).
- *Local Search* (LS) (Vaessens, et al., 1994; Yamada y Reeves, 1997).
- *Genetic Programming* (GP) (Howard y D'Angelo, 1995; Banzhaf, et al., 1998).
- *Greedy Randomized Adaptive Search Procedure* (GRASP) (Resende y González-Velarde, 2003; Resende y Ribeiro, 2003).
- *Memetic Algorithms* (MA) (Yang, et al., 2008).
- *Artificial Neural Networks* (ANN) (Looi, 1992; Akyol, 2004; Rondón, et al., 2008).
- *Simulated Annealing* (SA) (Kirkpatrick, 1983; Davis, 1987; Laarhoven, et al., 1992; Yamada, et al., 1994; Russell y Norvig, 1995; Sadeh y Nakakuki, 1996; Yamada y Nakano, 1996; Kolonko, 1999; Ponnambalam, et al., 1999; Zhang y Wu, 2010).
- *Scatter Search* (SS) (Laguna, et al., 2003; Martí y Laguna, 2003)

- *Tabu Search (TS)* (Dell'Amico y Trubian, 1993; Nowicki y Smutnicki, 1996; Vilcot y Billaut, 2008; Zhang, *et al.*, 2008).
- *Shifting Bottleneck (SB)* (Adams, *et al.*, 1988; Pezzella y Merelli, 2000; Wenqi y Aihua, 2004; Brito, *et al.*, 2006; Gao, *et al.*, 2007).
- *Variable Neighborhood Search (VNS)* (Gao, *et al.*, 2008; Roshanaei, *et al.*, 2009; Adibi, *et al.*, 2010; Yazdani, *et al.*, 2010).
- *Critical Path Method (CPM)* (Pinedo, 2005).
- *Program Evaluation and Review Technique (PERT)* (Pinedo, 2005).

1.8. Aplicación de algoritmos genéticos a la solución de problemas de scheduling

Los algoritmos genéticos (AGs) son técnicas de búsqueda estocástica e iterativa basada en los mecanismos de evolución natural, que pueden usarse para resolver problemas de búsqueda y optimización. Los principios básicos fueron establecidos por el profesor J. Holland (Holland, 1975) y se encuentran bien descritos en varios textos (Goldberg, 1989; Davis, 1990; Reeves, 1993). Estos usan una estrategia de selección y operadores –esencialmente recombinación y mutación–, para generar y explorar nuevos puntos en el espacio de soluciones que originen una nueva población, la cual estará constituida por un conjunto de individuos denominados cromosomas que están compuestos por elementos menores llamados genes, los cuales codifican las instrucciones, parámetros y demás elementos usados para construir una solución del problema a resolver (Figura 1.2). Los algoritmos genéticos utilizan un conjunto de individuos sobre los cuales se aplican operadores y estrategias para encontrar soluciones mejoradas del problema. El conjunto de individuos que usa el algoritmo genético se denomina población. La codificación más común de los cromosomas que conforman las soluciones es a través de cadenas binarias (Nakano y Yamada, 1991), aunque se han utilizado también números reales y letras (Bean, 1994). El primero de estos esquemas ha gozado de mucha popularidad debido a que es el que propuso originalmente Holland, y además porque resulta muy sencillo de implementar en los sistemas de cómputo.

Una función de aptitud (*fitness function*) debe ser diseñada para cada problema de manera específica. Una característica que esta debe tener es que debe ser

capaz de “castigar” a las malas soluciones y de “premiar” a las buenas, de forma que sean estas últimas las que se propaguen con mayor rapidez. La selección de padres se efectúa al azar usando un procedimiento que favorezca a los individuos mejor adaptados, ya que a cada individuo se le asigna una probabilidad de ser seleccionado que es proporcional a su función de aptitud.

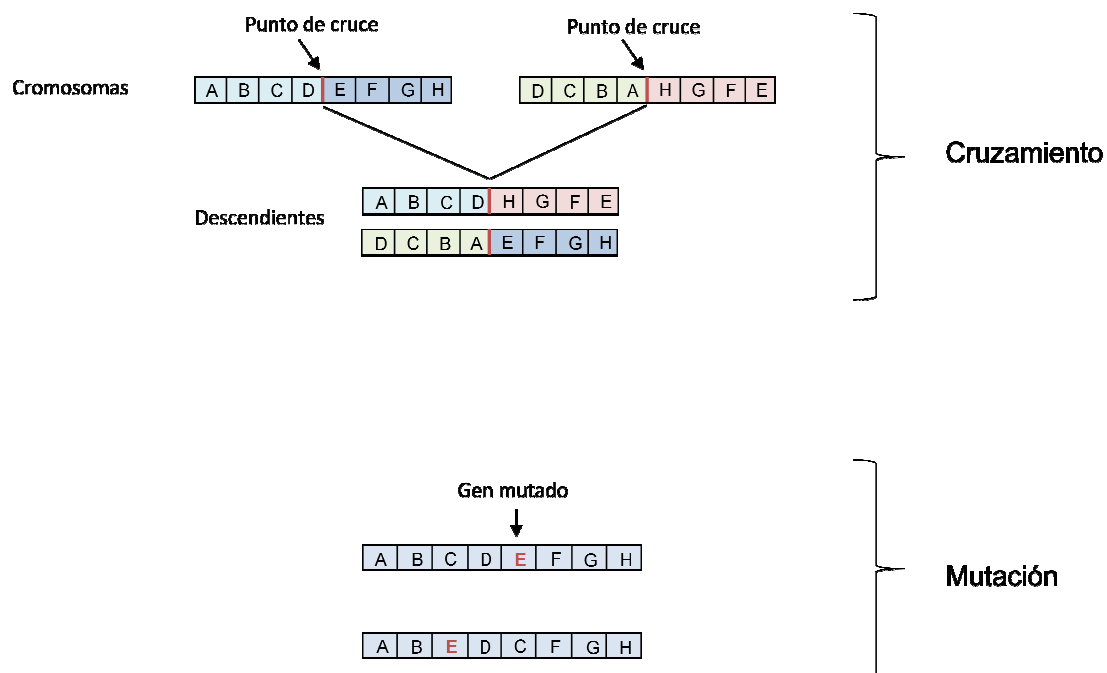


Figura 1.2. Operadores genéticos: cruzamiento y mutación.

Los algoritmos genéticos no buscan una única solución, sino que están orientados a que, en promedio, los individuos de la población busquen una mejora en la función objetivo de las soluciones, de manera que en posteriores generaciones se enriquezcan (evolucionen), preservando sólo las mejores características de los individuos de generaciones anteriores.

Estos algoritmos han demostrado ser un método global y robusto de búsqueda de soluciones de problemas, unido a esto, la aplicación más común de estos ha sido la solución de problemas de optimización donde se ha comprobado que son muy eficientes y confiables. Precisamente la principal ventaja de estas características es el equilibrio alcanzado entre la eficiencia y la eficacia para resolver diferentes y complejos problemas de grandes dimensiones. Si bien no se garantiza que el AG encuentre la solución óptima del problema, existe evidencia empírica de que se encuentran soluciones de un nivel aceptable, en

un tiempo competitivo con el resto de los algoritmos de optimización combinatoria. Cabe destacar, que en el caso de que existan técnicas especializadas para resolver un determinado problema, lo más probable es que superen al AG tanto en rapidez como en eficacia. El gran campo de aplicación de los estos algoritmos se relaciona con aquellos problemas para los cuales no existen técnicas especializadas. Incluso en el caso en que dichas técnicas existan y funcionen, pueden efectuarse mejoras de las mismas hibridándolas con los algoritmos genéticos.

Entre las decisiones que hay que tomar para implementar un algoritmo genético se encuentran:

- Criterio de codificación: se refiere a cómo se va a almacenar la información del genoma.
- Criterio de tratamiento de individuos no factibles: se refiere a cómo se van a tratar los individuos que no cumplan las restricciones.
- Criterio de inicialización: se refiere a cómo se va a construir la población inicial del algoritmo genético.
- Criterio de parada: determina cuándo el algoritmo ha llegado a una solución aceptable.
- Función de adaptación: se corresponde con la medida de desempeño, el objetivo que se evalúa.
- Operadores genéticos: se emplean para determinar cómo va a ser la nueva generación. Básicamente son los operadores de cruce y mutación aunque pueden ser empleados otros adicionales –aborto, envejecimiento.
- Criterios de reemplazo: determinan quienes se van a cruzar.

En la solución al *Job Shop Scheduling* en esta investigación, se utilizó un Algoritmo Genético Simple (*Simple Genetic Algorithm*, SAG). Un esquema básico de su funcionamiento es el que se muestra a continuación (Figura 1.3).

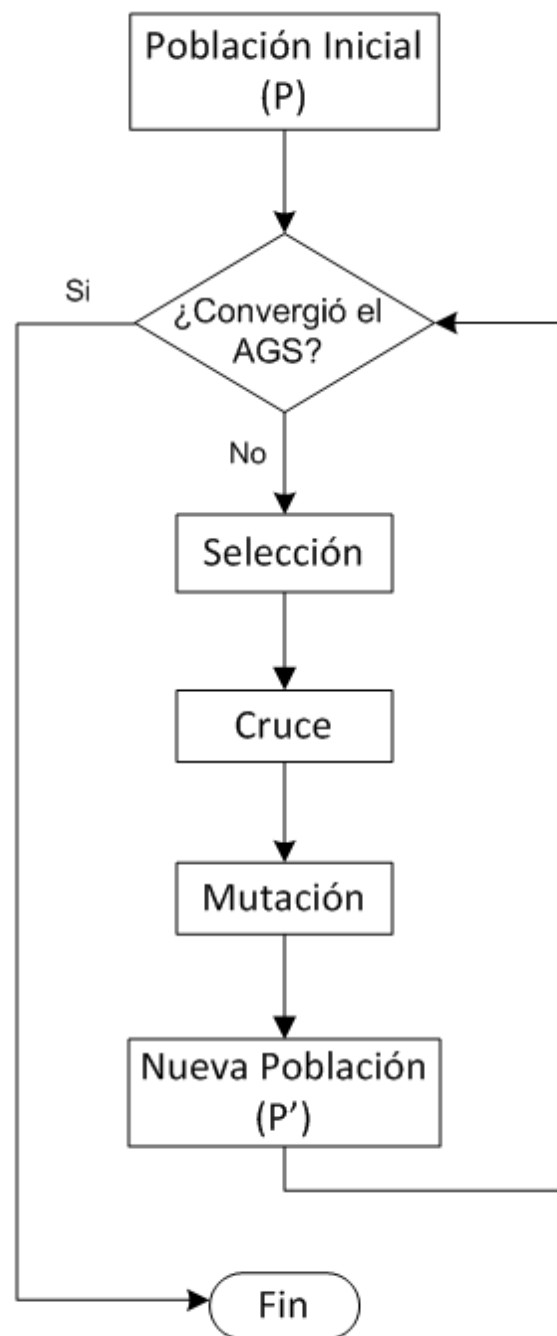


Figura 1.3. Esquema de funcionamiento del algoritmo genético simple.

1.9 Conclusiones del capítulo

En este capítulo se han analizado los problemas de planificación de trabajos, su nomenclatura, así como diversas aproximaciones de resolución de los mismos y se ha concluido que:

1. Existen diferentes sistemas de producción de acuerdo con el flujo de los productos o los procesos. Entre ellos podemos encontrar los sistemas *flow shop* o fabricación en línea, y los *job shop* o taller de tareas.
2. El problema de secuenciamiento de tareas es considerado de difícil solución y es típico de la optimización combinatoria. Está clasificado técnicamente como de solución en un tiempo no polinomial (NP-completo).
3. Para su solución se utilizan técnicas de optimización combinatoria que permiten encontrar soluciones de buena calidad en tiempos de ejecución computacionalmente aceptables.
4. La clasificación triparamétrica del problema que consta de tres variables $\alpha|\beta|\gamma$ sirve tanto para modelar los problemas como para catalogar los diferentes modelos de planificación y permite la incorporación de nuevos problemas de planificación que se investigan en la actualidad.
5. Los algoritmos genéticos constituyen una solución factible para la optimización de problemas de programación (*job shop scheduling*)

Capítulo 2

VARIANTES DE PROBLEMAS DE PLANIFICACIÓN EN EL TALLER DE MECANIZADO

2.1. Introducción

Al tener el *scheduling* la misión de asignar recursos escasos a tareas de forma temporal, con el objetivo de optimizar alguna medida de desempeño, se puede establecer la correspondencia entre la definición del problema y su formulación como un problema JSS en el taller con tecnología de maquinado. Es así, que entonces los recursos pasarán a ser las máquinas o puestos de trabajo, en sentido general, los trabajos serán las piezas a fabricar y las tareas van a constituirse en las operaciones a realizar sobre los trabajos en las distintas máquinas, las cuales adquieren el nombre de la máquina utilizada, como por ejemplo: en la máquina-herramienta nombrada Torno, la operación recibirá el nombre de torneado. De manera similar ocurrirá con las demás máquinas del taller.

En la presente investigación se analizan dos variantes de gran interés dentro de los problemas de *scheduling*, tal y como se anticipó anteriormente (sección 1.3), las cuales se identifican en los procesos de manufactura con tecnología de mecanizado. Estas son: el *Job Shop Scheduling* (JSS) y el *Flow Shop Scheduling* (FSS), tal y como se conocen en la terminología anglosajona.

También se incluye el *Permutation Flow Shop Scheduling (PFSS)*, el cual constituye un caso particular de la variante *FSS*.

2.2. Estructura de datos para la representación del problema

Como en todos los problemas que se enfrentan en el mundo real, para poder resolverlos se tiene que encontrar una forma de abstraerlos y poder representar sus posibles soluciones. Existen varias formas de representar el *JSS* para su solución. Entre las más conocidas están, la representación con Grafos Disyuntivos (Yamada y Nakano, 1997) y la representación con Redes de Petri (Murata, 1989; Desrochers y Al-Jaar, 1995). Para el desarrollo de este trabajo se utilizó una representación basada en un grafo disyuntivo dirigido, al tener en cuenta que esta representación garantiza una mayor claridad de las soluciones a obtener debido a las restricciones que se tienen en el problema. Esta forma de representación ha encontrado mayor aceptación entre los investigadores de esta temática, lo cual no le quita el mérito ni la fortaleza a la representación basada en redes de Petri.

Un grafo (ver Figura 2.1) es una pareja de conjuntos $G = (V, A)$, donde V es el conjunto finito de vértices, y A es el conjunto de aristas, este último es un conjunto de pares de la forma (u, v) tal que $u, v \in V$. En este caso se añade que $u \neq v$. Al tener esto en cuenta se asume que: cada nodo del grafo representa la relación (máquina i , trabajo j). Existen dos nodos especiales: S y E que representan el inicio (*Start*) y el final (*End*) de todas las operaciones. Para cada dos operaciones consecutivas en el mismo trabajo $(i, j) \in A$ existe un arco dirigido; los nodos S y E son el primero y el último nodo de todos los trabajos respectivamente. Para cada par de trabajos que emplean la misma máquina $\{i, j\} \in V$ existen dos arcos (i, j) y (j, i) en sentidos opuestos, que indican cuál es la operación que se debe ejecutar antes. Cada nodo i tiene asociado un peso p_i que indica el tiempo que se necesita para completar la operación i . Una vez que se ha construido el grafo como se definió anteriormente, y se tiene el valor de p_i como el peso de cada rama (i, j) , entonces el *makespan* es igual a la longitud del camino más largo entre S y E también conocido como camino máximo o camino crítico (*critical path*).

2.3. Variante Flujo General (*Job Shop Scheduling, JSS*)

La variante *JSS* presenta como característica principal que cada trabajo j_j consiste en una cadena de m_j operaciones $O_{1j}, \dots, O_{m_jj}$. No tiene por que ser $m_j = m$. Incluso puede ser $m_j > m$, con lo que puede presentarse la situación en la que alguna máquina procese dos o más operaciones de un mismo trabajo, lo que se conoce como recirculación, la cual no ha sido incluida en el presente trabajo. La trayectoria de las máquinas de cada trabajo está dada pero no tiene que ser la misma para todos los trabajos. Es decir, O_{ij} se procesa en la máquina μ_{ij} en un tiempo p_{ij} . Nótese que los talleres que no se dedican en masa a la producción de un único producto, presentan problemas de planificación similares al *job shop*. Solo se ha desarrollado un algoritmo que minimiza el intervalo de fabricación para el caso del *job shop* en dos máquinas: el algoritmo de Jackson (Jackson, 1955).

La figura siguiente muestra un grafo disyuntivo dirigido con la representación de esta variante del problema.

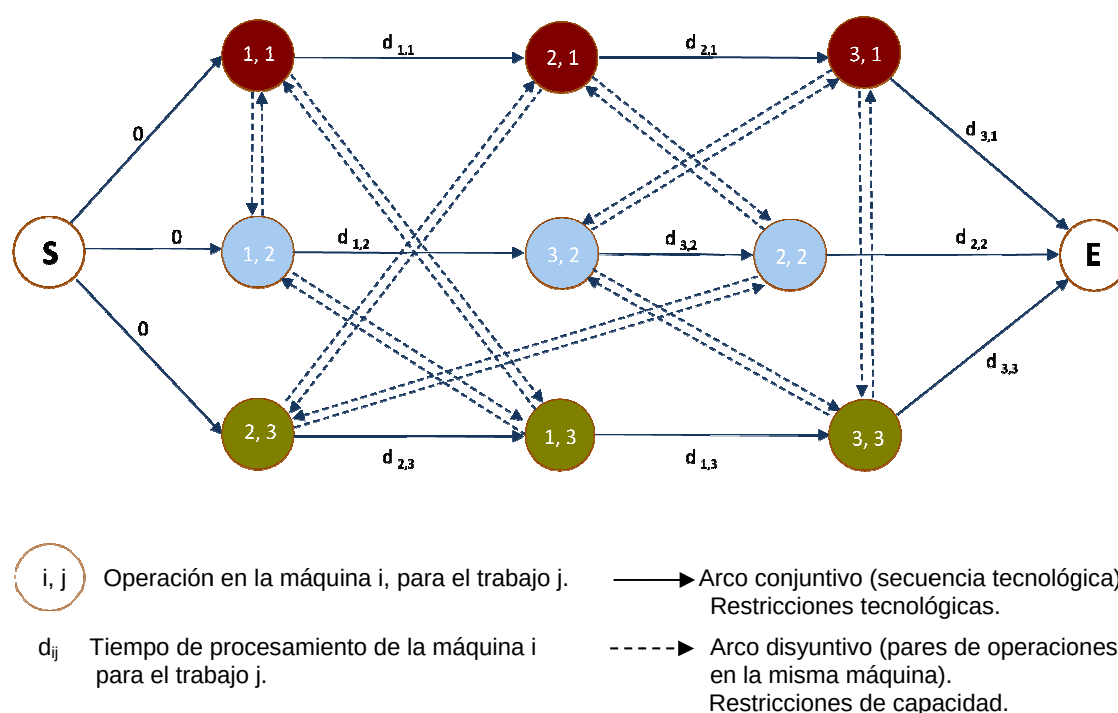


Figura 2.1. Ejemplo de un grafo disyuntivo dirigido para un problema tipo *JSS* de 3 trabajos en 3 máquinas (3×3).

Las relaciones de competencia (disyunciones) existen entre las diferentes operaciones de los distintos trabajos, es decir, una operación cualquiera de una

pieza puede necesitar la misma máquina que otra operación cualquiera de otra pieza, lo cual hace aún más complejo el problema.

El objetivo perseguido en este problema es encontrar alguna planificación factible que optimice alguna medida de desempeño. Normalmente el más usado en la literatura es el de minimizar el *makespan* (9). Este objetivo es equivalente a minimizar los tiempos muertos, o a maximizar la utilización de las máquinas, y ésta es tal vez la razón por la cual ha sido abordado con mayor frecuencia por los investigadores.

Makespan: es el tiempo mínimo para completar todos los trabajos. Esta versión del problema es conocida en la literatura como $J||C_{máx}$ y se obtiene de la forma:

$$C_{máx} = \max_{j \in \{1 \dots n\}} C_j \quad (9)$$

La siguiente tabla (Tabla 2.1) contiene datos de un típico problema JSS de 3 trabajos (piezas) en 3 máquinas, para un total de 9 operaciones sin la posibilidad de recirculación, es decir, ninguna operación puede retornar a una máquina ya visitada. Los trabajos están representados por los números 1, 2 y 3. Se puede observar, como los trabajos 1 y 2 comienzan compitiendo en su primera operación en la máquina número 1, sin embargo el trabajo 3 comienza en la máquina número 2 y de forma general todos siguen trayectorias (ruta tecnológica) distintas.

Tabla 2.1. Instancia ejemplo de problema 3 x 3 variante JSS.

Trabajos	Máquinas (Tiempo de procesamiento)		
1	1 (2)	2 (3)	3 (5)
2	1 (6)	3 (5)	2 (3)
3	2 (4)	1 (3)	3 (4)

A través de un gráfico de Gantt, como el que muestra la figura (Figura 2.2) se visualiza mejor un plan de trabajo con una solución factible, en este caso óptima, en cuanto a la minimización del *makespan*. Es posible distinguir en la secuencia del ejemplo, la existencia de tantas subsecuencias como máquinas existen, y que en cada una de estas subsecuencias se permite la repetición de

un mismo trabajo, significando el orden de sus operaciones en las máquinas correspondientes. Se debe interpretar de la siguiente forma: la primera vez que aparezca un trabajo se refiere a su primera operación, la segunda vez, se refiere a la segunda, y así sucesivamente se cumple con todos. Continuando con el ejemplo, se pueden presentar secuencias de la forma: 1 1 1 – 2 2 2 – 3 3 3.

El número total de alternativas diferentes que existen para el ejemplo de la tabla anterior, de planificar 3 piezas en 3 máquinas, es de 1 680.

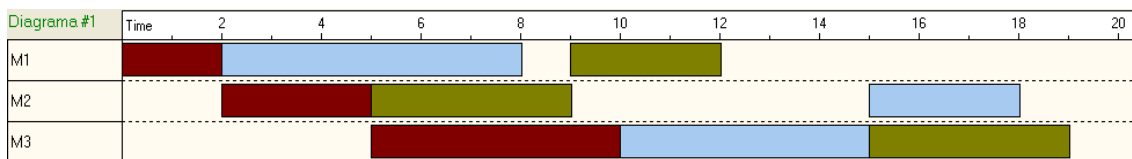


Figura 2.2. Gráfico de Gantt con una solución factible para el grafo anterior. El orden de ejecución de los trabajos es: 1 2 1 – 1 2 3 – 2 3 3.

La siguiente figura (Figura 2.3) muestra el mismo grafo anterior, con la particularidad de haber resuelto las disyunciones necesarias entre las diferentes operaciones de los distintos trabajos, para encontrar el camino máximo. Es posible apreciar como los arcos disyuntivos (con líneas discontinuas), terminan adquiriendo un sentido definitivo en los pares de operaciones que demandan la utilización de una misma máquina, con lo cual se establece la dirección y el sentido completo desde el principio hasta el final de la secuencia. Su recorrido cumple con las restricciones del modelo, por lo tanto constituye una solución factible al problema. Se corresponde con el gráfico de Gantt de la figura anterior (Figura 2.2).

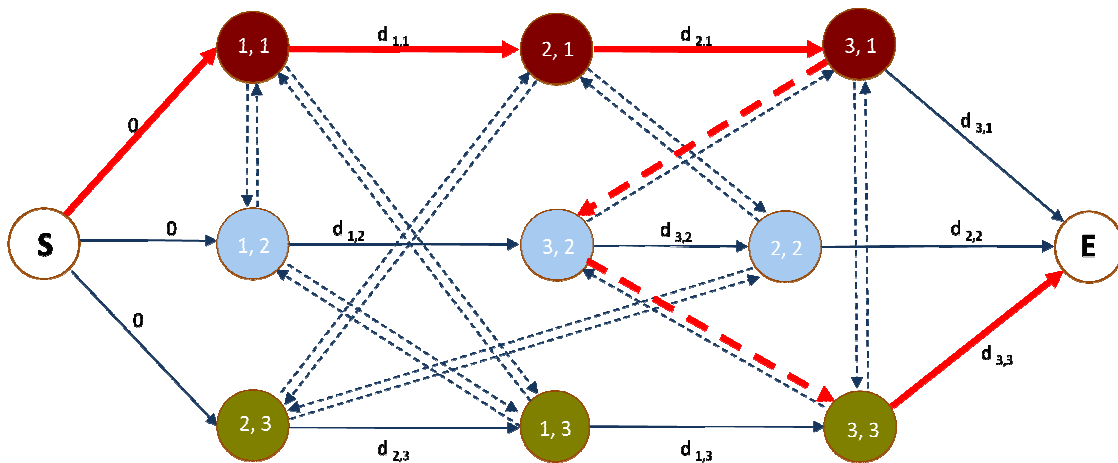


Figura 2.3. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la figura anterior.

El subgrafo correspondiente a esta la solución es el que se muestra a continuación (Figura 2.4).

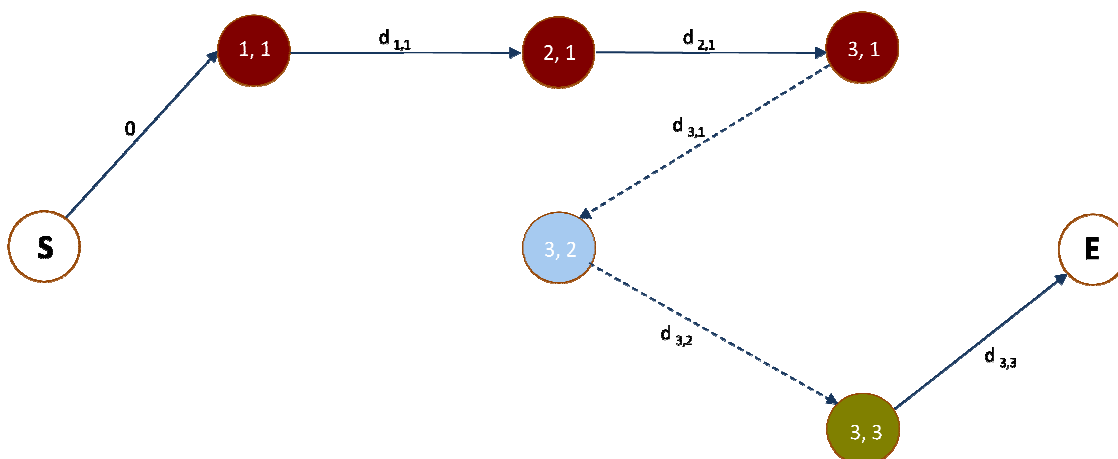


Figura 2.4. Subgrafo resultante que representa la solución del camino máximo.

2.4. Variante Flujo Regular (*Flow Shop Scheduling, FSS*)

En la variante *FSS* o fabricación en línea, cada pieza se procesa por todas o algunas de las máquinas (puestos de trabajo) en el taller siguiendo un orden prefijado por un patrón común. Dicho orden es relevante y siempre es el mismo, es decir, todas las piezas deben seguir un mismo patrón de flujo, esto es, la misma trayectoria de máquinas. No es necesario que un trabajo j pase por todas las máquinas. Si el trabajo j no se procesa por la máquina i , se

considera el tiempo de procesamiento $p_{ij} = 0$. De esta manera se engloba también los problemas *flow shop* en los que no todos los trabajos tengan que ejecutarse en todas las máquinas. Esta variante se parece a una cadena de montaje, aunque existen algunas diferencias. En la figura siguiente (Figura 2.5) se muestra un grafo disyuntivo dirigido que sirve para representar esta variante del problema (Márquez, et al., 2012). Se puede apreciar como existen relaciones de competencia (disyunciones) entre las mismas operaciones de los distintos trabajos, es decir, los tres trabajos deben hacer uso de la misma máquina en su primera operación, así también en su segunda y de esa misma manera terminan en la tercera. Expresado de otra forma, las operaciones de un mismo orden O_{ij} necesitan de la misma máquina i .

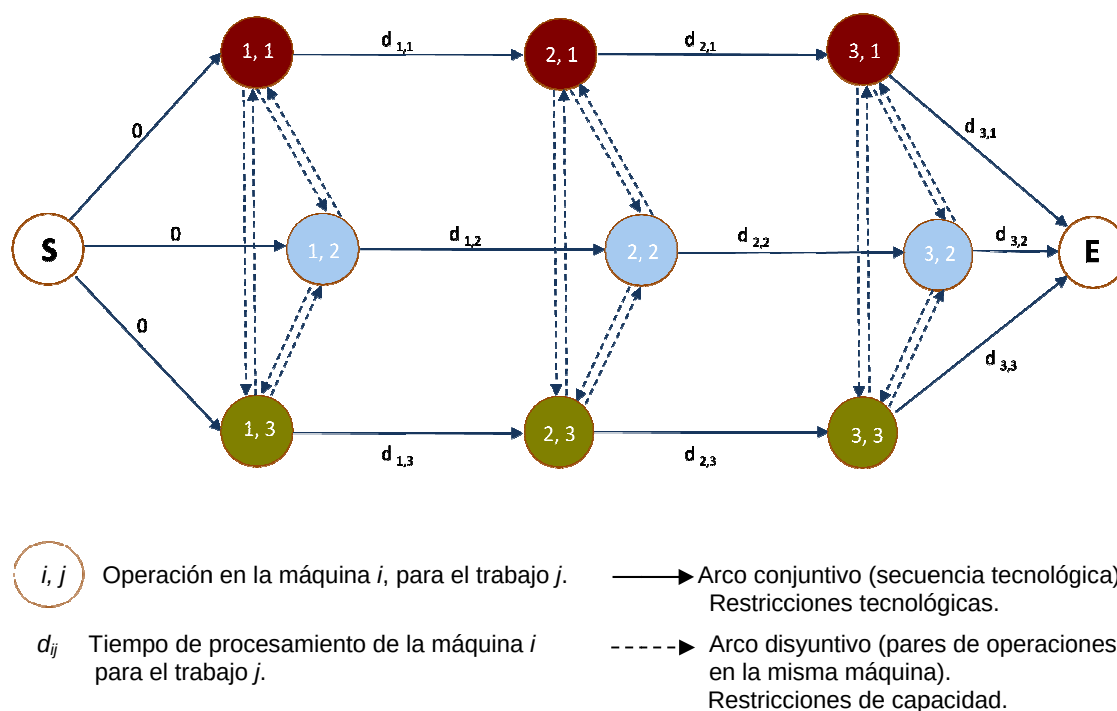


Figura 2.5. Ejemplo de un grafo disyuntivo dirigido para un problema de la variante FSS de 3 piezas en 3 máquinas (3 x 3).

Este problema es muy complejo debido a su carácter combinatorio. Si los N trabajos se procesan en las M máquinas existen $(N!)^M$ alternativas para la programación.

La siguiente tabla (Tabla 2.2) contiene datos de un típico problema FSS de 3 trabajos (piezas) en 3 máquinas, para un total de 9 operaciones sin la

posibilidad de recirculación. Los trabajos están representados por los números 1, 2 y 3. Se puede observar, como los tres trabajos comienzan compitiendo en su primera operación en la máquina número 1, lo mismo sucede en la máquina número 2 y en la máquina número 3, con las respectivas operaciones de cada trabajo. En esta variante todos siguen la misma trayectoria (ruta tecnológica) y solo experimentan distintos tiempos de permanencia en cada máquina.

Siguiendo este ejemplo, si hay que planificar 3 piezas en 3 máquinas, se dispone de 216 alternativas diferentes. Hasta el momento solo está resuelto de forma óptima el caso estático del *flow shop* en dos máquinas. El algoritmo que minimiza el intervalo de fabricación es el algoritmo de Johnson (Johnson, 1954).

Tabla 2.2. Instancia ejemplo de problema 3 x 3 variante FSS.

Trabajos	Máquinas (Tiempo de procesamiento)		
1	1 (2)	2 (3)	3 (5)
2	1 (6)	2 (5)	3 (3)
3	1 (4)	2 (3)	3 (4)

El gráfico de Gantt de la figura siguiente (Figura 2.6) muestra un plan de trabajo con una solución factible. Es posible distinguir nuevamente en la secuencia del ejemplo, la existencia de tantas subsecuencias como máquinas existen, con la diferencia de que en cada una de ellas no se permite la repetición de un mismo trabajo. Su interpretación es similar a la variante anterior, es decir, la primera vez que aparezca un trabajo se refiere a su primea operación, la segunda vez, se refiere a la segunda, y así sucesivamente se cumple con todos. No podrán existir secuencias de la forma: **1 1 1 - 2 2 2 - 3 3 3**.

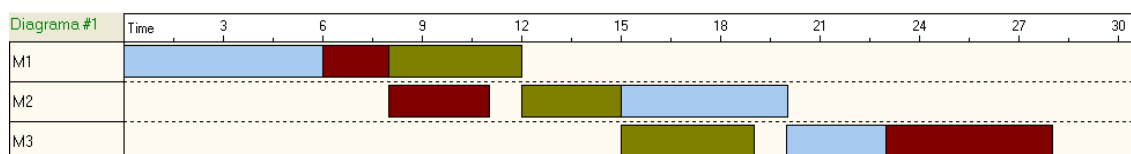


Figura 2.6. Gráfico de Gantt con una solución factible para el grafo anterior. El orden de ejecución de los trabajos es: 2 1 3 – 1 3 2 – 3 2 1.

La siguiente figura (Figura 2.7) muestra el mismo grafo anterior, con la determinación del sentido definitivo para las disyunciones formadas entre las diferentes operaciones de los distintos trabajos, para encontrar el camino máximo. Los arcos disyuntivos (con líneas discontinuas), denotan el sentido definitivo en los pares de operaciones que demandan la utilización de una misma máquina. Su recorrido cumple con las restricciones del modelo, por lo tanto constituye una solución factible al problema. Se corresponde con el gráfico de Gantt de la figura anterior (Figura 2.6).

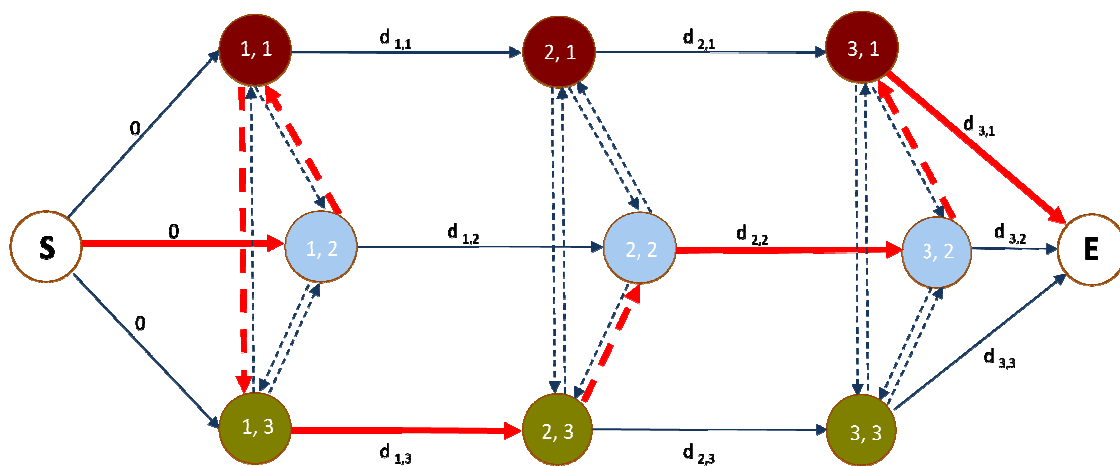


Figura 2.7. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la figura anterior.

El subgrafo siguiente (Figura 2.8) muestra de otra forma esta solución.

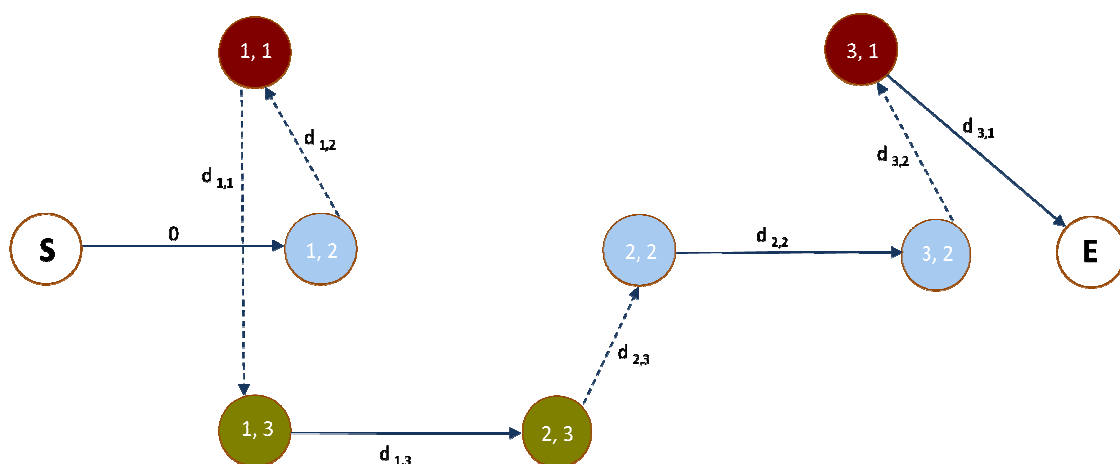


Figura 2.8. Subgrafo resultante que representa la solución del camino máximo.

2.5. Variante Flujo Permutacional (*Permutational Flow Shop Scheduling, PFSS*)

Por último, se presenta la variante *PFSS* que constituye un caso particular de la anterior, en la cual hay m máquinas en serie y cada trabajo se procesa en cada una de ellas siguiendo una misma trayectoria. En esta versión del *FSS*, se exige además, que las “colas” que forman los trabajos para visitar las máquinas, lo hagan bajo una disciplina de *primero en entrar primero en salir* (*First In First Out, FIFO*), quedando de esta manera también visitadas las máquinas en el mismo orden.

En la figura siguiente (Figura 2.9) se muestra un grafo disyuntivo dirigido similar al de la variante anterior. Nuevamente existen relaciones de competencia (disyunciones) entre las mismas operaciones de los distintos trabajos, es decir, los tres trabajos deben hacer uso de la misma máquina en su primera operación, así también en su segunda y de esa misma manera terminan en la tercera.

Este problema también resulta muy complejo por su carácter combinatorio. Como los n trabajos se procesan en las m máquinas en el mismo orden, y además estas máquinas deben ser visitadas también en un mismo orden, existen entonces $n!$ alternativas para la programación. Siguiendo el ejemplo anterior, al planificar 3 piezas en 3 máquinas, se dispone de 6 alternativas diferentes, o sea, basta con las permutaciones de todos los trabajos.

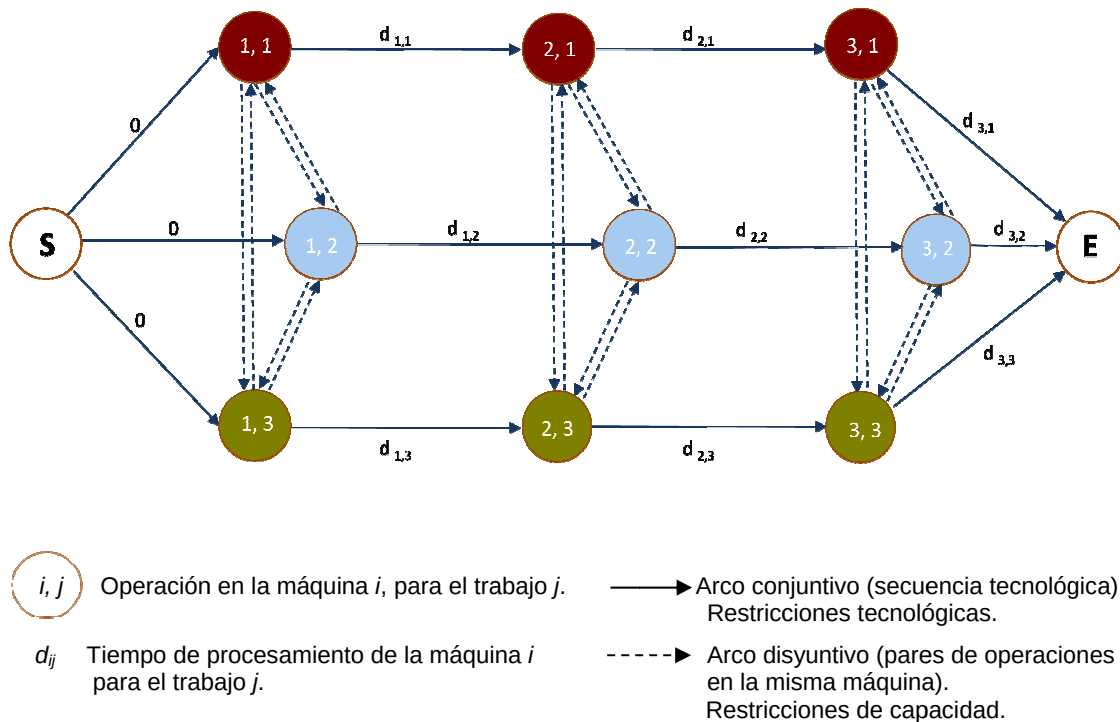


Figura 2.9. Grafo disyuntivo dirigido para un problema de la variante *FSS* con 3 piezas en 3 máquinas (3 x 3).

Una solución factible para un plan de trabajo se muestra mediante un gráfico de Gantt (Figura 2.10). En esta variante no es necesaria la existencia de sub-secuencias en correspondencia con el número de máquinas, ya que como las mismas son visitadas en idéntico orden, basta con establecer secuencias a base de los distintos trabajos. Para el ejemplo, los datos han sido tomados de la tabla anterior (Tabla 2.2) la cual presenta un problema *FSS* de 3 trabajos (piezas) en 3 máquinas, para un total de 9 operaciones sin la posibilidad de recirculación. Al añadir la condición de que las máquinas sean visitadas en el mismo orden, el camino máximo siempre finalizará en la última operación del último trabajo que visita la primera máquina.

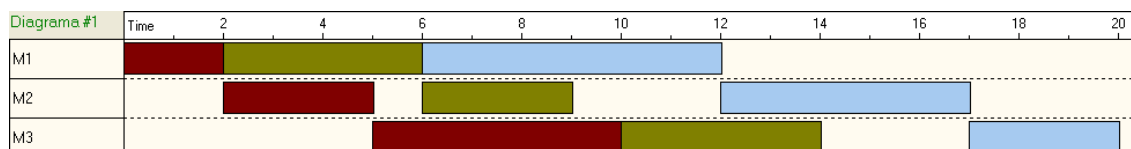


Figura 2.10. Gráfico de Gantt con una solución factible para el grafo de la figura anterior. El orden de ejecución de los trabajos es: 1 3 2.

Similar a la variante *FSS*, la siguiente figura (Figura 2.11) muestra el mismo grafo anterior (Figura 2.9), con la determinación del sentido definitivo para las disyunciones formadas entre las diferentes operaciones de los distintos trabajos, para encontrar el camino máximo. Nuevamente los arcos disyuntivos (con líneas discontinuas), denotan el sentido definitivo en los pares de operaciones que demandan la utilización de una misma máquina. Su recorrido cumple con las restricciones del modelo, por lo tanto constituye una solución factible al problema. Se corresponde con el gráfico de Gantt de la figura anterior (Figura 2.10).

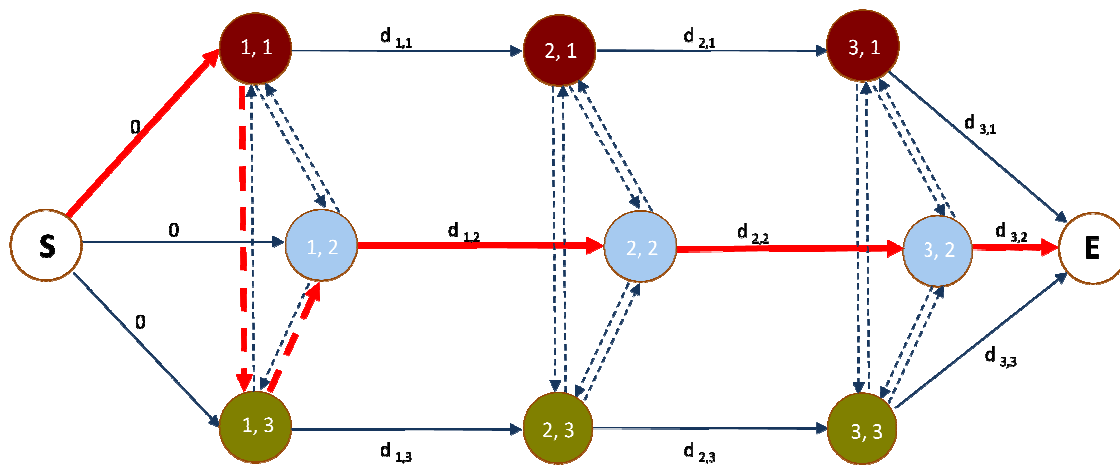


Figura 2.11. Grafo que representa la solución del camino máximo mostrado en el gráfico de Gantt de la anterior.

Un subgrafo correspondiente al plan mostrado en el gráfico de Gantt (Figura 2.10), es el que se muestra a continuación (Figura 2.12). Corresponde a la solución óptima del camino máximo (*makespan*).

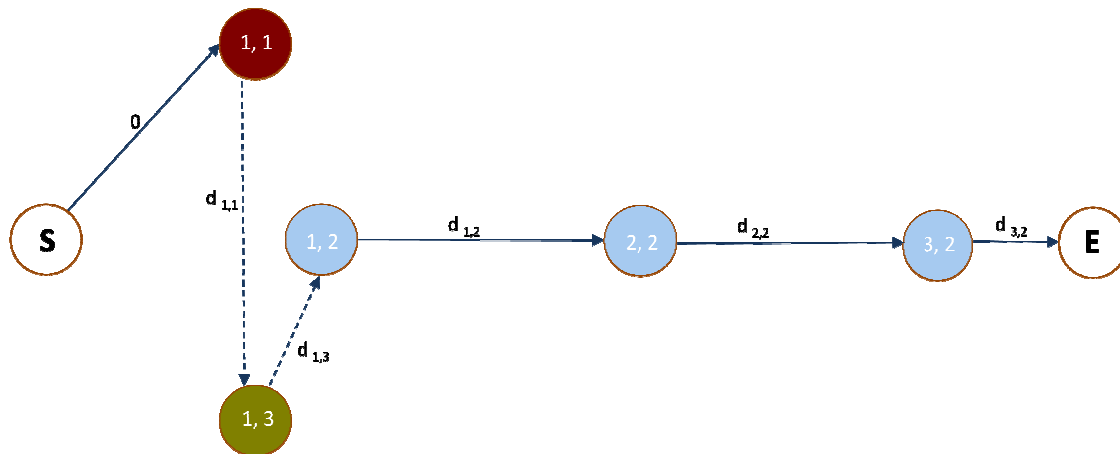


Figura 2.12. Subgrafo que representa la solución del camino máximo.

En la siguiente sección, se resume el análisis del universo de soluciones de cada una de las variantes de problema, que se presentan en un taller con tecnología de maquinado en dependencia del tamaño del espacio de búsqueda de soluciones, derivado de las alternativas antes mencionadas.

2.6. Estudio del universo de soluciones posibles en correspondencia con la forma de obtener secuencias

En esta investigación se realizó un estudio basado en tres formas distintas de secuencias posibles, en correspondencia con cada variante. Estas son:

- Permutación de trabajos: las secuencias se obtienen solo por permutación de los trabajos N por lo tanto representan un orden fijo en que serán visitadas todas las máquinas. Este cálculo permite obtener el total de secuencias válidas para el caso flujo permutacional (*PFSS*), dentro de la variante de flujo regular (*FSS*). Las mismas se obtienen de la expresión siguiente (10):

$$N! \quad (10)$$

- Permutación de operaciones (partición) sin repetición: las secuencias se pueden obtener diferentes para cada máquina por permutación de las operaciones, las cuales no se pueden repetir para un mismo trabajo. Este cálculo permite obtener el total de secuencias válidas para la variante *FSS*, en la cual, aunque todos los trabajos presentan un mismo orden, cada máquina puede experimentar un orden distinto en la visita de los trabajos.

Las mismas se obtienen de la expresión siguiente (11):

$$(N!)^M \quad (11)$$

- Permutación de operaciones (partición) con repetición: las secuencias se pueden obtener diferentes para cada máquina por permutación de las operaciones, las cuales se pueden repetir para un mismo trabajo. Este cálculo permite obtener el total de secuencias válidas para la variante JSS, en la cual los trabajos tienen distinto orden y por lo tanto visitan desigual las máquinas. Las mismas se obtienen de la expresión siguiente (12):

$$\frac{(N * M)!}{(M!)^N} \quad (12)$$

La siguiente tabla (Tabla 2.3) muestra algunos ejemplos del tamaño del espacio de búsqueda de soluciones factibles, en dependencia de las formas de cálculo anteriores, para obtener las secuencias a partir del número de trabajos y del número de máquinas.

Como se puede observar en la medida en que aumenta el número de trabajos y el número de máquinas, la dimensión del espacio de búsqueda de soluciones se incrementa y produce un aumento en la complejidad del problema.

A continuación (Tabla 2.4) se muestra una porción del desarrollo de las secuencias para un problema de 3 trabajos en 3 máquinas. En la representación han sido utilizadas letras como símbolos, y las columnas con permutación de operaciones, no muestran su desarrollo de forma completa debido al espacio que ocuparían.

Tabla 2.3. Ejemplos de dimensiones de problema y el tamaño del espacio de búsqueda de soluciones factibles correspondiente en dependencia de la forma de obtener las secuencias.

N	M	Permutación de trabajos $N!$	Permutación de operaciones (partición)	
			Sin repetición $(N!)^M$	Con repetición $\frac{(N * M)!}{(M!)^N}$
3	3	6	216	1 680
4	4	24	331 776	63 063 000
5	5	120	24 883 200 000	623 360 743 125 120
6	6	720	139 314 069 504 000 000	2 670 177 736 637 149 247 308 800
7	7	5 040	82 606 411 253 903 523 840 000 000	7 363 615 666 157 189 603 982 585 462 030 336 000

Tabla 2.4. Secuencias posibles del espacio total de búsqueda de soluciones factibles correspondiente a un problema de 3 trabajos ($N = 3$) en 3 máquinas ($M = 3$).

Permutación de trabajos $N!$	Permutación de operaciones (partición)	
	Sin repetición $(N!)^M$	Con repetición $\frac{(N * M)!}{(M!)^N}$
ABC	ABC-ABC-ABC	AAA-BBB-CCC
ACB	ABC-ABC-ACB	AAA-BBC-BCC
BAC	ABC-ABC-BAC	AAA-BBC-CBC
BCA	ABC-ABC-BCA	AAA-BBC-CCB
CAB	ABC-ABC-CAB	AAA-BCB-BCC
CBA	ABC-ABC-CBA	AAA-BCB-CBC
	ABC-ACB-ABC	AAA-BCB-CCB
	ABC-ACB-ACB	AAA-BCC-BBC
	ABC-ACB-BAC	AAA-BCC-BCB
	ABC-ACB-BCA	AAA-BCC-CBB
	ABC-ACB-CAB	AAA-CBB-BCC
	ABC-ACB-CBA	AAA-CBB-CBC
	ABC-BAC-ABC	AAA-CBB-CCB
	ABC-BAC-ACB	AAA-CBC-BBC
	ABC-BAC-BAC	AAA-CBC-BCB
	ABC-BAC-BCA	AAA-CBC-CBB
	ABC-BAC-CAB	AAA-CCB-BBC
	ABC-BAC-CBA	AAA-CCB-BCB
	ABC-BCA-ABC	AAA-CCB-CBB
	ABC-BCA-ACB	AAA-CCC-BBB
	ABC-BCA-BAC	AAB-ABB-CCC
	ABC-BCA-BCA	AAB-ABC-BCC
	ABC-BCA-CAB	AAB-ABC-CBC
	ABC-BCA-CBA	AAB-ABC-CCB
	ABC-CAB-ABC	AAB-ACB-BCC
	ABC-CAB-ACB	AAB-ACB-CBC
	ABC-CAB-BAC	AAB-ACC-BBC
	ABC-CAB-BCA	AAB-ACC-BCB
	ABC-CAB-CAB	AAB-ACC-CBB
	ABC-CAB-CBA	AAB-BAB-CCC
	ABC-CBA-ABC	AAB-BAC-BCC
	ABC-CBA-ACB	AAB-BAC-CBC
	ABC-CBA-BAC	AAB-BAC-CCB
	ABC-CBA-BCA	AAB-BBA-CCC
	ABC-CBA-CAB	AAB-BBC-ACC
	ABC-CBA-CBA	AAB-BBC-CAC
	ACB-ABC-ABC	AAB-BBC-CCA
	ACB-ABC-ACB	AAB-BCA-BCC
	ACB-ABC-BAC	AAB-BCA-CBC
	ACB-ABC-BCA	AAB-BCA-CCB
	...	...
Total de secuencias	6	216
		1 680

2.7. Análisis de los tipos de planes de trabajo

El espacio total de planificaciones factibles es demasiado grande y contiene además muchas soluciones que no son realmente interesantes, ya que a partir de ellas es posible obtener otras mejores o iguales. En la búsqueda de soluciones para problemas de *scheduling* se consideran tres tipos de planificaciones más reducidas, nombradas: semiactiva (*semi-active*), activa (*active*) y sin retardo o densas (*non-delay*). Estas últimas son un subconjunto de las planificaciones activas, las cuales son a su vez un subconjunto de las planificaciones semiactivas. Cada tipo se puede describir de la siguiente manera de acuerdo a sus características:

- Plan semiactivo: Es un plan de trabajo en el que existe una o varias máquinas con tiempo ocioso mayor o igual que el tiempo de duración de alguna operación que puede adelantar su procesamiento y mantener el orden de precedencia (ruta tecnológica). Se cambia el orden en la secuencia de dos o más operaciones de diferentes trabajos sin violar ninguna restricción (Figura 2.13). Estos desplazamientos de operaciones quedan limitados porque se llega a un punto en que la máquina o la pieza no están libres.

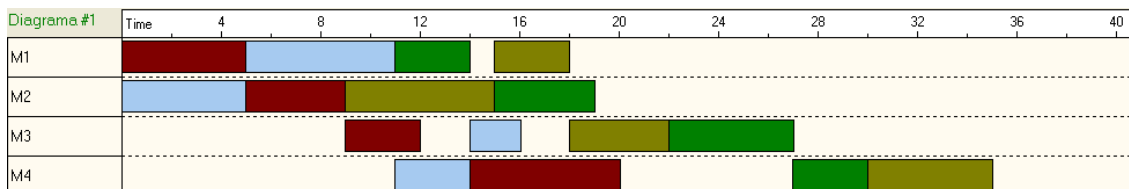


Figura 2.13. Gráfico de Gantt con plan de trabajo semiactivo. Variante JSS.

- Plan activo: Es un plan de trabajo en el que existe una o varias máquinas con tiempo ocioso menor que el tiempo de duración de alguna operación que puede adelantar su procesamiento y mantener el orden de precedencia (ruta tecnológica). Se cambia el orden en la secuencia de dos o más operaciones de diferentes trabajos sin violar ninguna restricción (Figura 2.14). Estos desplazamientos de operaciones quedan limitados porque se llega a un punto en que la máquina o la pieza no están libres.

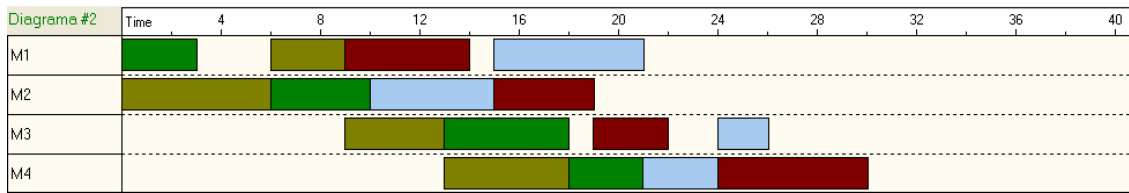


Figura 2.14. Gráfico de Gantt con plan de trabajo activo. Variante JSS.

- Plan sin retardo: Es un plan de trabajo en el cual no existe tiempo ocioso en las máquinas porque ninguna operación está lista para comenzar adelantado su procesamiento. Nunca están libres las máquinas o las piezas. (Figuras 2.15, 2.16 y 2.17).

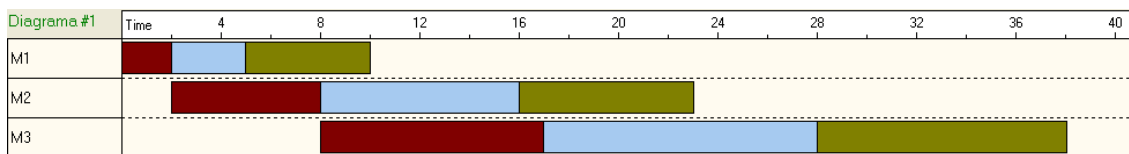


Figura 2.15. Gráfico de Gantt con plan de trabajo sin retardo debido a máquinas no libres. Variante PFSS.

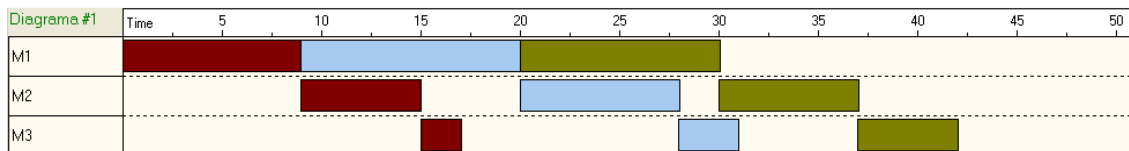


Figura 2.16. Gráfico de Gantt con plan de trabajo sin retardo debido a piezas no libres. Variante PFSS.

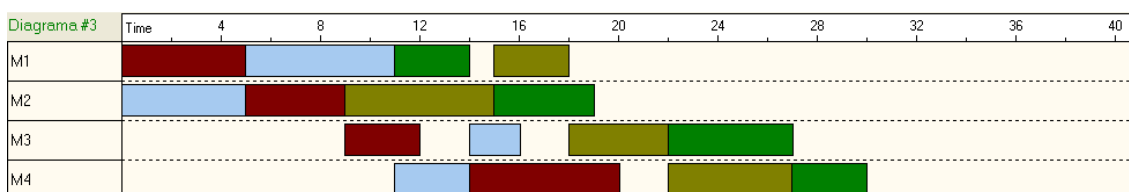


Figura 2.17. Gráfico de Gantt con plan de trabajo sin retardo debido a máquinas y piezas no libres. Variante JSS.

A continuación (Figura 2.18) se muestra la jerarquía de las clases de planes de trabajo. Es fácil comprobar que una planificación sin retardo es activa, y que una planificación activa, es también semiactiva. Además, la intersección del conjunto de planificaciones óptimas con el conjunto de las planificaciones

activas, es siempre no vacío, mientras que en general, esto mismo no se puede garantizar en el caso de las planificaciones sin retardo y semiactivas. En consecuencia, si queremos tener la seguridad de que el espacio de búsqueda contiene al menos una solución óptima debemos restringir la búsqueda, como mucho, al espacio de planificaciones activas. En la presente investigación, se reconoce como región o zona donde es posible encontrar los valores óptimos de solución a problemas de *scheduling*, al espacio de búsqueda de planificaciones activas y en dependencia de la instancia del problema, a los espacios de búsqueda de planificaciones semiactivas y sin retardo. Es decir, no siempre se encuentran valores óptimos en estos dos últimos espacios de planificaciones para las medidas de desempeño nombradas regulares, que son aquellas en las que el índice puede formularse en función de los instantes de salida de las piezas:

$$\gamma = f(C_1, C_2, \dots, C_i) \quad (13)$$

Por lo tanto una medida regular solo empeora (aumenta) si uno por lo menos de los valores C_i aumenta y solo mejora (disminuye) si alguno o todos los valores de C_i disminuyen.

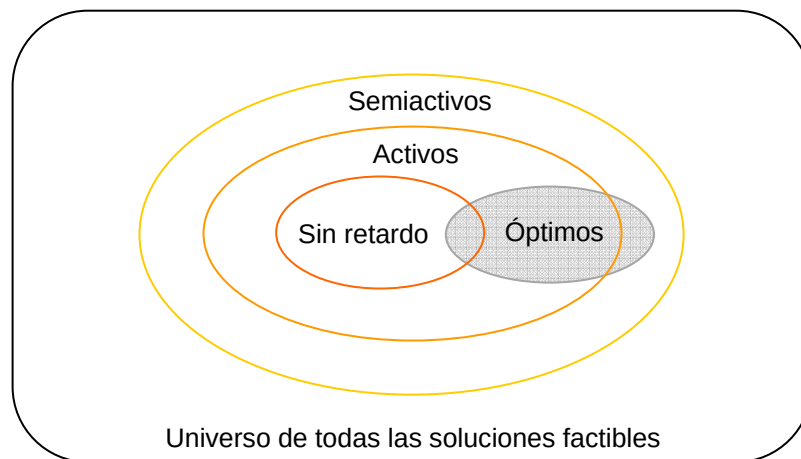


Figura 2.18. Jerarquía de las clases de planes de trabajo.

En la sección de anexos (Anexo 1) se encuentra un ejemplo que muestra la existencia del valor óptimo de camino máximo en cada una de las zonas antes

mencionadas. El ejemplo se obtuvo con la instancia de problema la33, cuyo valor óptimo es: $C_{max} = 1719$.

2.8. Criterios tenidos en cuenta para la optimización

En general, no es fácil definir el objetivo en un problema de secuenciación y planificación ya que suelen ser numerosos y a menudo se encuentran en conflictos, a lo que se añade la dificultad de modelar matemáticamente estos objetivos para encontrar una solución que los optimice. Estos permiten clasificar los programas según su finalidad y por otro lado cuantificar el valor del programa estableciendo si un programa es mejor que otro. A continuación se describen los criterios utilizados para dirigir la búsqueda de una planificación tenidos en cuenta en la presente investigación. Estos criterios están agrupados según el tipo de medida que afectan. La figura que continúa (Figura 2.19) permite una mejor interpretación basada en los datos y variables asociados a un trabajo, conjuntamente con las definiciones y notaciones que se pueden hacer sobre la planificación antes de enunciar los objetivos a plantear.

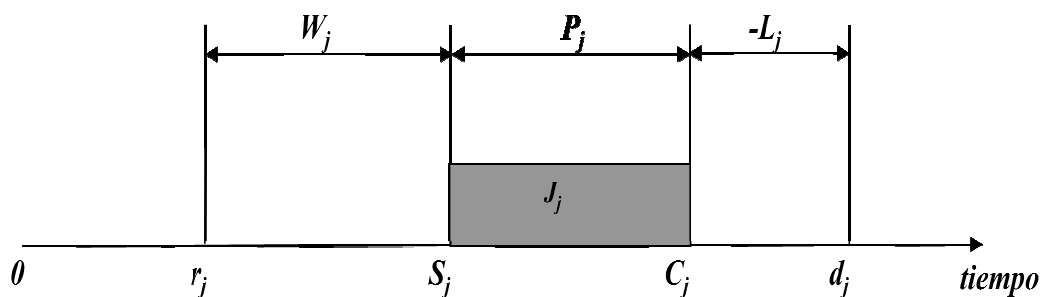


Figura 2.19. Datos y variables asociados a un trabajo.

donde:

r_j : tiempo de disponibilidad del trabajo J_j (*release date*)

S_j : tiempo de comienzo (*start time*)

C_j : tiempo de finalización (*completion time*)

d_j : tiempo de vencimiento (*due date*)

W_j : peso o ponderación (*weight*)

P_j : tiempo de proceso (*process time*)

L_j : retraso (*Lateness*)

2.8.1. Criterios basados en los instantes de finalización

- finalización de todos los trabajos (*makespan*).

Se trata del tiempo de finalización de la última tarea. Esta versión del problema es conocida en la literatura como $J||C_{max}$ y es sin dudas, la versión más tratada en la literatura, por lo que existen numerosas referencias a propuestas que resuelven el problema con diferentes métodos, apoyados la mayoría en el camino crítico.

$$C_{max} = \max_{j \in \{1 \dots n\}} C_j \quad (14)$$

- flujo total (*flow time*).

El flujo total es la suma de los tiempos de finalización de todos los trabajos. Cuando se considera esta función objetivo, el problema JSS se denota como $J||\sum C_i$. Aunque se trata de una versión muy parecida al clásico problema $J||C_{max}$, en realidad es más difícil de resolver. Probablemente esto sea debido a que, en general, una instancia del problema $J||\sum C_i$ tiene menos soluciones óptimas que la correspondiente instancia del problema $J||C_{max}$, por lo que para una metaheurística resulta más difícil de encontrar una de estas soluciones.

$$F_{max} = \sum_{j \in \{1 \dots n\}} F_j \quad (15)$$

- ocio de las máquinas (*idle time*).

El ocio total es la suma de los tiempos de inactividad, debido al ocio, de las máquinas que intervienen en la ejecución del plan de trabajo. Minimizar este objetivo equivale a un máximo aprovechamiento de las máquinas.

$$Idle = \sum_{j=1}^m I_j \quad (16)$$

donde:

I_j : tiempo total de ocio de la máquina j .

m : número de máquinas.

2.8.2. Criterios basados en la fecha de entrega

- retraso (*lateness*).

Minimizar este objetivo es apropiado cuando existe una recompensa por entregar un trabajo antes de tiempo. Una demora positiva indica la tardanza en la finalización del trabajo, mientras que, la conclusión del trabajo anticipadamente a su fecha de compromiso d_j se indica por una demora negativa o adelanto, cuyo valor absoluto es la cantidad de tiempo anticipada.

$$L_j = C_j - d_j \quad (17)$$

- tardanza (*tardiness*).

La minimización de este objetivo se hace importante cuando existe una penalización en caso de demora por terminar un trabajo fuera de plazo. Mide la peor violación de los tiempos de entrega.

$$T_j = \max\{C_j - d_j, 0\} \quad (18)$$

Indica el retraso en la ejecución del trabajo J_j

- número de trabajos adelantados (*early Jobs*).

El número de trabajos adelantados se desea maximizar con el propósito de cumplir con los compromisos pactados. Se hace útil, además, cuando se premia por la entrega de los trabajo a tiempo.

$$NE_j = \sum_{j=1}^n F_j \quad (19)$$

donde:

F_j : trabajo adelantado j .

n : número de trabajos.

– número de trabajos retrasados (*tardy Jobs*).

Se corresponde con la regla de decisión EDD (*Earliest Due Date*) al ordenar los trabajos en función de la fecha de entrega o vencimiento d_i . Los trabajos se procesan en orden creciente a esta fecha.

$$NU_j = \sum_{j=1}^n G_j \quad (20)$$

donde:

G_j : trabajo retrasado j .

n : número de trabajos.

Estas medidas de desarrollo son importantes debido a que hay casos en los que la penalización en que se incurre por terminar un trabajo fuera de plazo no es proporcional a la cantidad, de modo que es lo mismo entregarlo un minuto o un siglo después. Una situación de este tipo puede darse por ejemplo si un vuelo es programado para aterrizar después del tiempo máximo para el que dispone de combustible, las consecuencias son igual de catastróficas tanto si es un minuto o una hora lo que se pasa de ese instante. Por lo tanto, en ocasiones más que plantear un objetivo que sea función de la cantidad en que se sobre pasa la fecha de entrega, se busca minimizar el número de trabajos fuera de plazo, siendo entonces preferible, por ejemplo, retrasar un trabajo mucho tiempo y mantener los demás en plazo que retrasar todos menos tiempo y que se salgan muchos de plazo.

2.8.3. Criterio basado en el consumo de energía

La energía utilizada por un plan de trabajo (*schedule*) es la suma total de los valores de energía consumido por cada máquina durante el tiempo de proceso, es decir, desde el inicio de la primera operación hasta la finalización de la última, que coincide con el valor del camino máximo o *makespan* (C_{max}), por lo cual no se realizan mediciones fuera de este último valor. La figura siguiente (Figura 2.22) muestra un ejemplo de gráfico de Gantt, al cual le han sido añadidas las marcas de lectura (mediciones) realizadas (líneas discontinuas verticales), para obtener el consumo de energía en diferentes instantes de tiempo a lo largo del plan de trabajo. La figura siguiente (Figura 2.23) muestra

un gráfico de carga que representa el consumo de energía resultado de las mediciones realizadas, así como el valor total y el promedio de dicho consumo (línea horizontal).

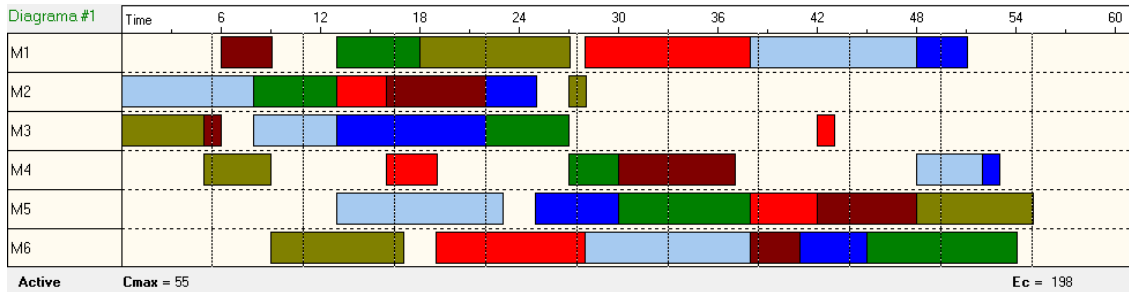


Figura 2.20. Gráfico de Gantt con marcas de lectura para el consumo de energía de un plan de trabajo.

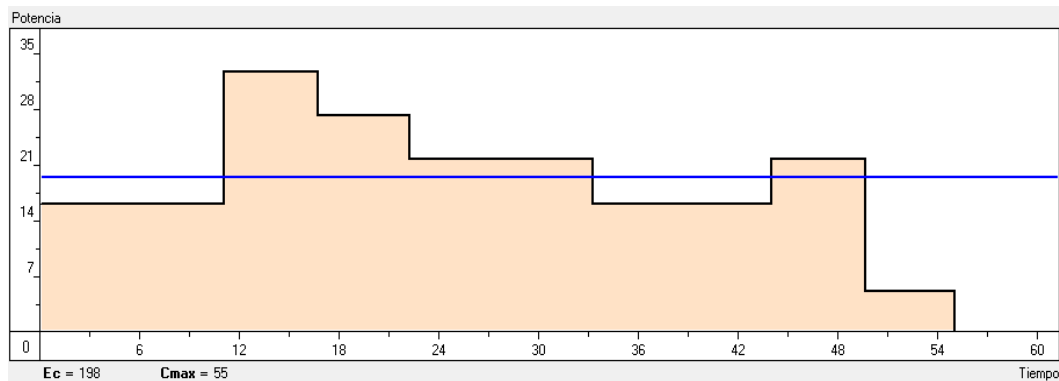


Figura 2.21. Gráfico de carga que muestra el consumo de energía del plan de trabajo correspondiente al gráfico de Gantt anterior. Consumo total: 198 unidades.

En el que el eje vertical contiene los valores de potencia de las máquinas, y en el eje horizontal el tiempo transcurrido desde el inicio de la secuencia hasta su final.

Un aumento en el número de mediciones k , o lo que es lo mismo, una reducción en los intervalos de tiempo de dichas mediciones, trae consigo una mayor precisión en la detección de máquinas encendidas o activadas (funcionando) y de máquinas apagadas o desactivadas (no funcionando). En la sección de anexos (Anexo 2), se muestra este mismo plan de trabajo, al cual se le aplica un mayor número de mediciones, con el objetivo de observar la influencia de este incremento en los resultados que se obtienen en el valor del consumo de energía. Para este ejemplo, se utilizó como dato un valor de potencia nominal en las máquinas igual a 1 unidad de potencia.

La siguiente expresión ha sido desarrollada para el cálculo del consumo de energía total de un plan de trabajo (*schedule*) a partir del conocimiento del tiempo de finalización de la última operación.

$$E_c = \sum_{i=1}^k \sum_{j=1}^m \left(Pot_j * \frac{C_{max}}{k} \right)_i \quad (21)$$

donde:

k : número total de mediciones del consumo de energía. ($k \geq 1$)

m : número total de máquinas.

Pot_j : potencia nominal de la máquina j .

C_{max} : tiempo de finalización del último trabajo (*makespan*).

E_c : energía total consumida por el plan de trabajo.

El algoritmo siguiente (Algoritmo 2.1) es el encargado de realizar este cálculo, teniendo en cuenta la secuencia o plan de trabajo, el número de mediciones y el valor de la potencia nominal de las máquinas que intervienen en el proceso. El algoritmo finaliza una vez que se ha alcanzado el valor de la última medición a realizar.

Algoritmo 2.1. Algoritmo para Cálculo del Consumo de Energía de un plan de trabajo (*schedule*).

```
CALCULO-CONSUMO-ENERGIA (secuencia, número_mediciones, potencia_maquinas)
1  Hacer consumo = 0
2  p = secuencia.GetCantDePiezas()
3  m = secuencia.GetCantDeMaquinas()
4  k = número_mediciones
5  cmax = secuencia.GetMakespan()
6  Maquinas_ON[1...m][1...k] = 0
7  Hacer Crear-Tabla-Maquinas-ON(secuencia, &Maquinas_ON, k)
8  Repetir desde c = 1 hasta k con incremento (1) hacer
9      Repetir desde f = 1 hasta m con incremento (1) hacer
10         ¿Es Maquinas_ON[f][c] == 1?
11         True: consumo = (potencia_maquinas[f] * cmax / k)
12     Fin Repetir
13 Fin Repetir
14 Salir-Retornando(consumo)
FIN
```

El proceso de optimización ha sido concebido para un único objetivo, a la vez que para obtener información de los demás objetivos calculados. Debido a la cantidad de criterios que se tienen en cuenta (8 en total), algunos se deben minimizar mientras otros se deben maximizar, incluyendo el hecho de que presentan diferentes unidades de medidas, por lo cual es posible que en ocasiones estos criterios se encuentren en conflicto y requieran ser analizados adecuadamente por criterio de expertos. Esto significa que para una mejor toma de decisiones en el proceso de optimización, se pueden calcular los valores de cada uno de los criterios anteriores que se han tenido en cuenta en esta investigación. De esta manera, se ofrece al técnico encargado de realizar la programación del taller de mecanizado, una mayor información sobre el proceso de fabricación de piezas.

La figura siguiente (Figura 2.22) muestra un plan de trabajo en forma de gráfico de Gantt con todos los objetivos que aquí se proponen calculados. Es así que se ofrecen los resultados de:

1. Camino máximo
2. Flujo total
3. Ocio
4. Adelanto
5. Tardanza
6. Número total de trabajos adelantados
7. Número total de trabajos tardados
8. Consumo de energía

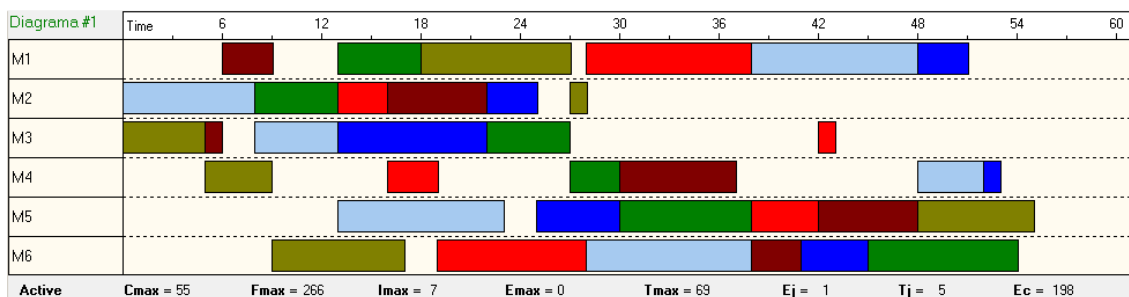


Figura 2.22. Gráfico de Gantt con varios objetivos calculados.

2.9. Relaciones entre las medidas de desarrollo

Entre las diferentes medidas de desarrollo anteriores existen relaciones por las cuales unas son equivalentes a otras, es decir, que las secuencias óptimas son las mismas si se plantea un objetivo o se plantea otro equivalente. Estas equivalencias permiten reformular un problema pudiendo cambiar un objetivo por otro más sencillo o cuya formulación matemática tenga mejores propiedades a la hora de resolverlo. Algunos ejemplos son los siguientes:

- si los instantes de posible inicio (*release date*) son igual a 0 para todos los trabajos, son equivalentes C_{max} y F_{max} .
- Si todos los trabajos tienen una misma fecha de entrega son equivalentes C_{max} y L_{max} .
- Una programación (*schedule*) que es óptima según el criterio L_{max} también lo es para T_{max} , sin embargo lo contrario no tiene por qué ser cierto. En cualquier caso nos permite resolver el problema con el criterio L_{max} , cuya formulación y resolución son más sencillas que las del otro criterio.

2.10. Estructura utilizada para construir el cromosoma. Modelo genético

Varias han sido las formas de codificar las soluciones para el problema del *Job Shop Scheduling* usando Algoritmos Genéticos. En la siguiente tabla (Tabla 2.6) se hace un resumen de algunas de las representaciones más empleadas por otros investigadores, atendiendo al tipo de representación que emplean.

Tabla 2.5. Tipos de representación según la estructura del cromosoma utilizada.

Representación	Autor
Binaria basada en pares de trabajo	Nakano y Yamada (1991)
Secuencia de operaciones con particiones basadas en listas de preferencias	Falkenauer y Bouffouix (1991), Croce <i>et al.</i> (1995)
Secuencia de operaciones con particiones y operadores especializados	Yamada y Nakano (1992)
Secuencia de trabajos	Holsapple <i>et al.</i> (1993)
Secuencia de operaciones sin particiones	Fang <i>et al.</i> (1993), Bierwirth (1995)
Secuencia de operaciones basadas en números aleatorios	Bean (1994)
Basadas en reglas de prioridad	Dorndorf y Pesch (1995)

Para el desarrollo de esta investigación se utilizó la secuencia de trabajos según el caso particular *PFSS*, debido a que los datos de los problemas

propuestos, permiten aplicar la disciplina *FIFO*, y resulta suficiente para generar el universo completo de soluciones factibles donde encontrar valores óptimos de planificaciones, para el objetivo propuesto de minimizar el camino máximo. La longitud del cromosoma queda determinada por la cantidad de trabajos N , por lo tanto utiliza menos memoria para su almacenamiento. Adicionalmente, esta forma directa de representar el cromosoma aprovecha el conocimiento del problema a modelar lo que se traduce en menos intentos por violentar el orden del proceso o ruta tecnológica de fabricación de las piezas.

A continuación (Figura 2.23), se muestra un cromosoma conformado por una secuencia de caracteres que representa los distintos trabajos y donde coinciden genes y alelos. Con esta representación se le dio solución a los problemas del caso particular *PFSS*, cuyos resultados se ofrecen en la sección de estudios experimentales. Hay que destacar, que esta representación en la que solo intervienen los trabajos, es suficiente para encontrar el valor óptimo global en el objetivo de minimizar el *makespan*.

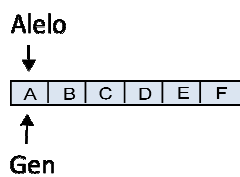


Figura 2.23. Representación de un cromosoma de 6 trabajos para un problema del caso particular *PFSS*. No se tiene en cuenta el número de máquinas.

El funcionamiento del algoritmo genético mediante sus operadores (métodos) de cruzamiento y mutación, provoca que se construyan cromosomas que resultan inválidos para esta representación. Tal es el caso de cuando se intenta repetir un alelo, que significa repetir un trabajo (Figura 2.24), lo cual no está permitido en este tipo de representación. Una alternativa posible es continuar con la ejecución del algoritmo genético y solo evaluar aquellos cromosomas que resulten válidos. El resultado puede ser que se consuma un gran número de generaciones sin que se pueda evaluar un cromosoma. Otra alternativa, sería corregir la invalidez de aquellos cromosomas que resultan no aceptados, luego de la aplicación de los diferentes operadores de cruzamiento y mutación.

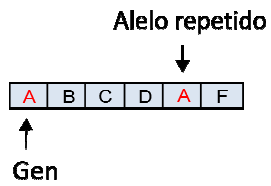


Figura 2.24. Cromosoma inválido.

Un ejemplo de esto es el algoritmo siguiente (Algoritmo 2.2), en el cual se verifica de izquierda a derecha, elemento por elemento (alelos), la conformación de la cadena completa del cromosoma hasta encontrar si existen algunos repetidos y proceder entonces a completar con los que faltan. De esta manera se garantiza que en todas las generaciones de la evolución del algoritmo genético, siempre se evalúen cromosomas válidos.

Algoritmo 2.2 Algoritmo para cromosomas inválidos de la forma: $N!$

```
ARREGLAR-CROMOSOMA-DE-TRABAJOS (&secuencia, gen_invalido)
1  Hacer len = secuencia.Longitud()
2  Repetir desde i = 1 hasta len con incremento(1) hacer
3      imagen.AgregarElemento(i)
4  Fin Repetir
5  Repetir desde j = 0 hasta len con incremento(1) hacer
6      imagen.EliminarElemento(secuencia[j])
7  Fin Repetir
8  repetir desde k = len - 1 hasta 0 con decremento(1) hacer
9      ¿Es secuencia[k] == gen_invalido?
10     True: secuencia[k] = imagen[0]
11     Ir al paso 10
12 Fin Repetir
13 ¿Es secuencia.Valida(&tmp_gen_invalido) == false?
14     True: ARREGLAR_SECUENCIA_DE_TRABAJOS(&secuencia, tmp_gen_invalido)
FIN
```

En la variante *FSS* ocurre otro tanto igual. La siguiente figura (Figura 2.25), muestra un cromosoma conformado por una secuencia de caracteres agrupados en subcadenas (genes), su longitud total es el producto de la cantidad de trabajos (piezas) por la cantidad de máquinas, y se producen tantos genes como máquinas existan en el problema, de esta forma se cumple con la forma de cálculo a partir de la cual se obtienen todas las combinaciones posibles (ver Tabla 2.4),

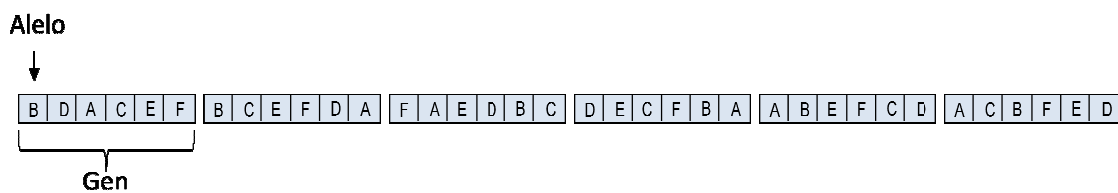


Figura 2.25. Representación de un cromosoma para un problema tipo *FSS* de 6 piezas en 6 máquinas. Nótese la no repetición de alelos en un mismo gen.

El algoritmo genético mediante sus operadores de cruzamiento y mutación, provoca que se construyan cromosomas que resultan inválidos para esta representación (Figura 2.26). En este caso la invalidez consiste, en la aparición de alelos repetidos en un mismo gen, ya que en esta forma de representar el cromosoma, es necesario verificar entonces que en el interior de cada gen, no se produzcan estas repeticiones indeseadas.

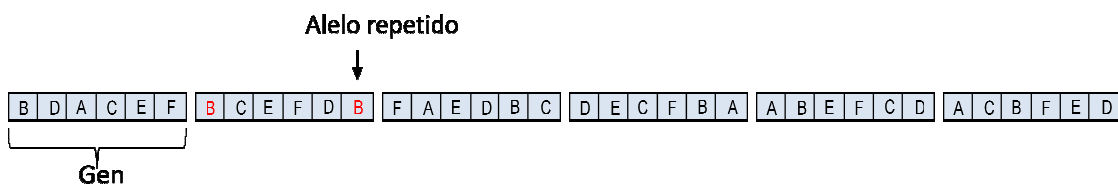


Figura 2.26. Cromosoma inválido.

Se acude a un algoritmo como el que se muestra a continuación (Algoritmo 2.3), capaz de convertir a válidos los cromosomas que resultan inválidos en el proceso continuo de la evolución del algoritmo genético.

Algoritmo 2.3 Algoritmo para cromosomas inválidos de la forma: $(N!)^M$

```
ARREGLAR-CROMOSOMA-DE-OERACIONES-SIN-REPETICION(&secuencia, gen_invalido, alelo_invalido)
1  Hacer npieza = secuencia.GetCantDePiezas()
2  posA = gen_invalido * npieza
3  posB = (gen_invalido + 1) * npieza - 1
4  Repetir desde i = 1 hasta npieza con incremento(1) hacer
5      imagen.AgregarElemento(i)
6  Fin Repetir
7  Repetir desde j = posA hasta posB con incremento(1) hacer
8      imagen.EliminarElemento(secuencia[j])
9  Fin Repetir
10 Repetir desde k = posB hasta posA con decremento(1) hacer
11     ¿Es secuencia[k] == alelo_invalido?
12         True: secuencia[k] = imagen[0]
13         Ir al paso 12
14     ¿Es secuencia.Valida(&tmp_gen_invalido, &tmp_alelo_invalido) == false?
15         True: ARREGLAR-CROMOSOMA-DE-OERACIONES-SIN-REPETICION(&secuencia, tmp_gen_invalido,
                                                                    tmp_alelo_invalido)
16 Fin Repetir
FIN
```

Para la correcta modelación de la variante JSS, en la presente investigación se propone una representación basada en una secuencia con particiones según la cantidad de máquinas, y contiene cada una de estas subsecuencias (genes) el orden en que son ejecutadas las operaciones de cada trabajo, de modo que cada plan (*schedule*) pueda ser representado por un único cromosoma. Estas secuencias con particiones tienen la particularidad de ser con la posibilidad de repetición de elementos individuales (alelos) que conforman cada subsecuencia dentro de la estructura del cromosoma. Así por ejemplo, la siguiente secuencia de caracteres (Figura 2.27) representa la estructura de un cromosoma para un problema de 6 trabajos (piezas) en 6 máquinas (6 x 6). Con esta representación se le dio solución a los problemas de de tipo JSS, cuyos resultados se ofrecen en la sección 3.2 (Análisis de Resultados). Se puede apreciar que existen tantas sub-secuencias como máquinas en el problema, y que en cada una de ellas es posible encontrar algunas repeticiones, significando que se realizarían las operaciones consecutivas para un mismo trabajo. Hay que considerar, que esta forma de representar las planificaciones supone un mayor número de soluciones factibles dentro del universo total, pero garantiza la posibilidad de generar todas las combinaciones posibles, donde encontrar el valor óptimo global para una medida de desempeño determinada en un plan de trabajo, al poder ser representadas todas las soluciones posibles de problemas de esta variante de cálculo (ver Tabla 2.4).

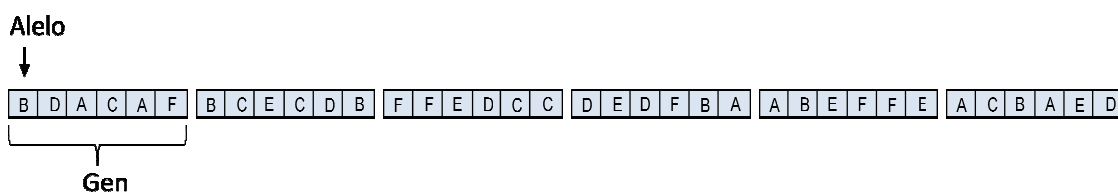


Figura 2.27. Representación de un cromosoma para un problema tipo JSS de 6 piezas en 6 máquinas. Nótese la repetición de alelos en un mismo gen.

Nuevamente, el algoritmo genético mediante sus operadores de cruzamiento y mutación, provoca que se construyan cromosomas que resultan inválidos para esta representación (Figura 2.28). En este caso, en que está permitida la repetición de alelos en un mismo gen, la invalidez consiste, en el intento de formarse cromosomas con un número de alelos mayor que el número de

máquinas (genes), ya que una de las restricciones tenidas en cuenta a la hora de modelar estos problemas, plantea que no se permite la recirculación, es decir, que un trabajo retorne a una máquina ya visitada.

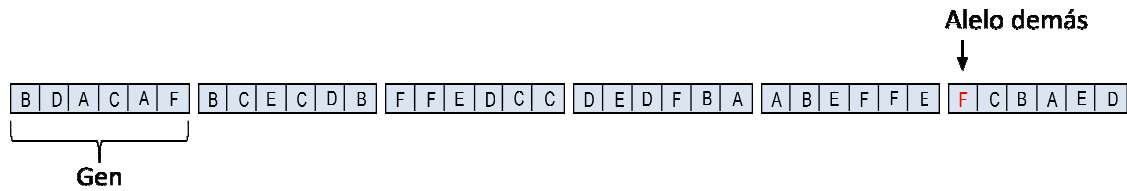


Figura 2.28. Cromosoma inválido.

Es necesario crear un algoritmo como el que se muestra (Algoritmo 2.4), capaz de convertir a válidos los cromosomas que resultan inválidos en el proceso continuo de la evolución del algoritmo genético.

Algoritmo 2.4 Algoritmo para cromosomas inválidos de la forma: $\frac{(N \times M)!}{(M!)^N}$

[illegible]

2.11. Mejora Basada en el Uso del Tiempo Ocioso (MBUTO)

La mejora basada en el uso del tiempo ocioso es un método desarrollado para ayudar a la metaheurística utilizada, en este caso algoritmos genéticos, en su proceso de búsqueda de mejores soluciones para optimizar uno o varios objetivos en un plan de trabajo. En cada evolución del AG se procede a probar si es posible mejorar las soluciones encontradas. La MBUTO, como su nombre lo indica, está basada en el aprovechamiento del tiempo ocioso en las máquinas, que tiene lugar en determinados planes de trabajo, tal es el caso de los planes semiactivos y activos. Como se conoce, en los planes sin retardo no existe tiempo ocioso alguno, es por ello que este método no es aplicable en el caso particular *PFSS*, y si lo es para las variantes *FSS* y *JSS*. En ambas variantes, al existir planes semiactivos y activos, es posible adelantar el inicio de determinadas operaciones.

Un esquema básico del funcionamiento de este método con el algoritmo genético es el que se muestra a continuación (Figura 2.29). En el mismo se puede apreciar los momentos en que se verifica si un plan de trabajo factible puede ser mejorado mediante el aprovechamiento de tiempos de ocio existentes y de esta forma mejorar las soluciones encontradas en cada iteración. En principio, este método se puede aplicar a de conjunto con otras metaheurísticas y para otros criterios de optimización en problemas de secuenciamiento.

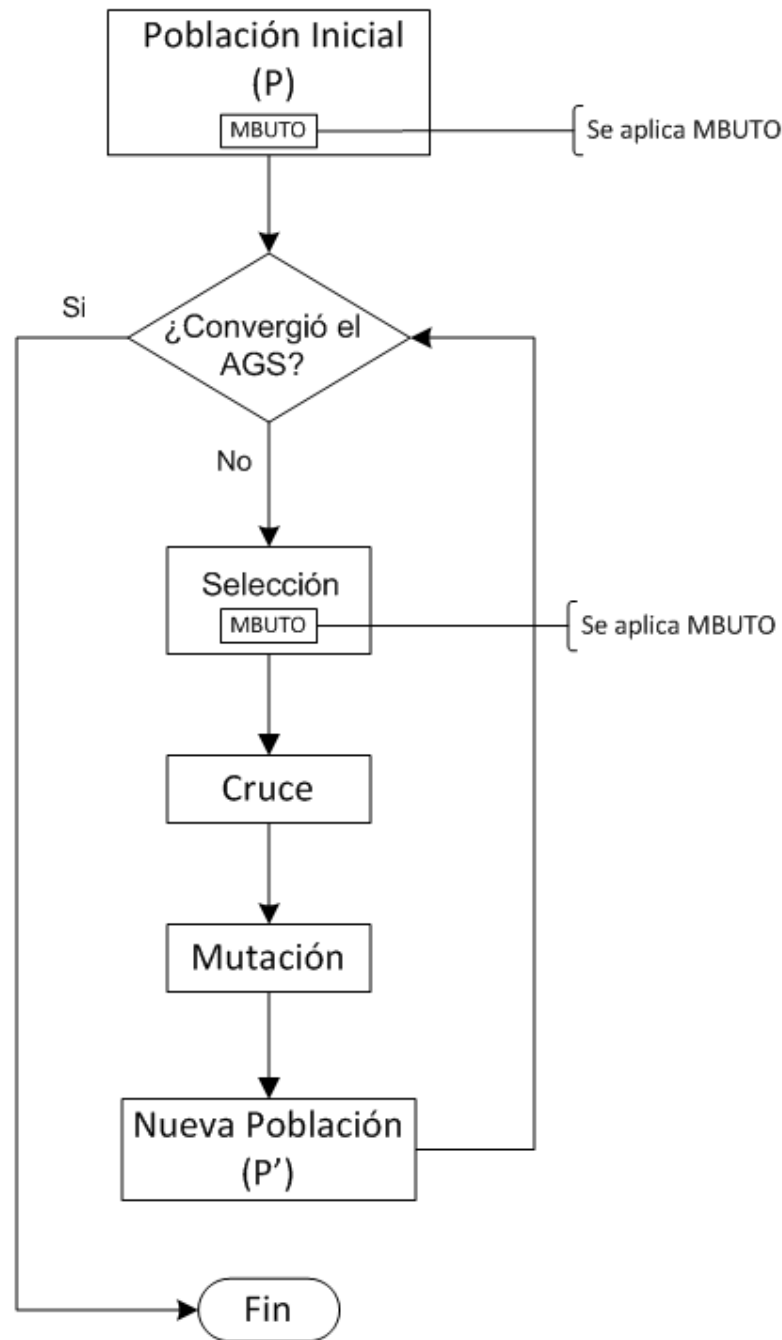


Figura 2.29. Esquema de funcionamiento del Algoritmo Genético Simple con la Mejora Basada en el Uso del Tiempo Ocioso.

En la siguiente figura (Figura 2.30) se muestra un plan de trabajo de variante *job shop*. Corresponde a una solución de la instancia de problema ft06 (Fisher y Thompson, 1963), cuyo valor óptimo de *makespan* es $C_{max} = 55$. Al aplicarle la MBUTO pueden ocurrir adelantamientos de operaciones mediante “salto a la izquierda” (*left jump*) y correspondientes “corrimientos a la izquierda” (*left shift*).

En la figura ocurre el salto en la máquina denominada M3 y el corrimiento en la máquina denominada M5.

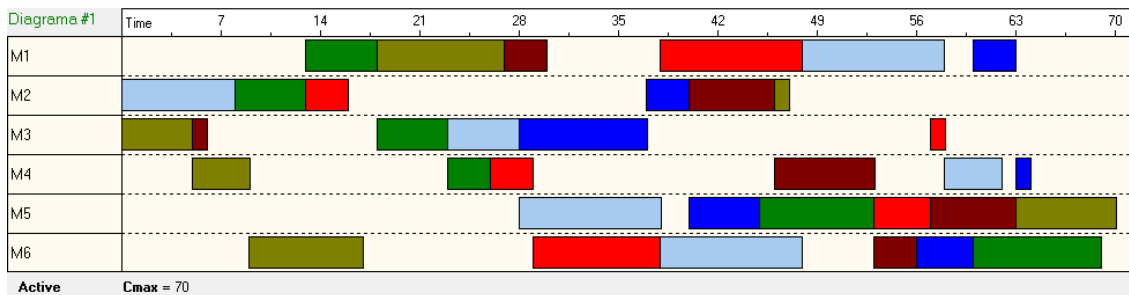


Figura 2.30. Gráfico de Gantt con valor de $C_{max} = 70$.

El adelantamiento (salto), se produce sin provocar el retraso de la operación o las operaciones adelantadas, por lo cual, el tiempo de la operación que se adelanta en este ejemplo, es menor que el tiempo de ocio que se aprovecha (Figura 2.31).

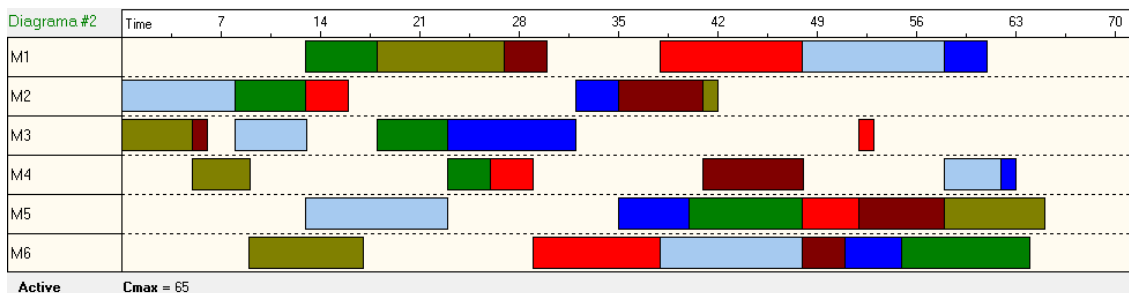
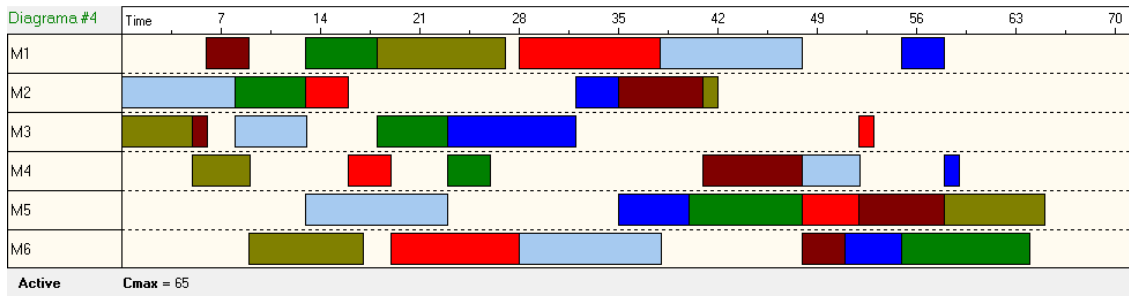
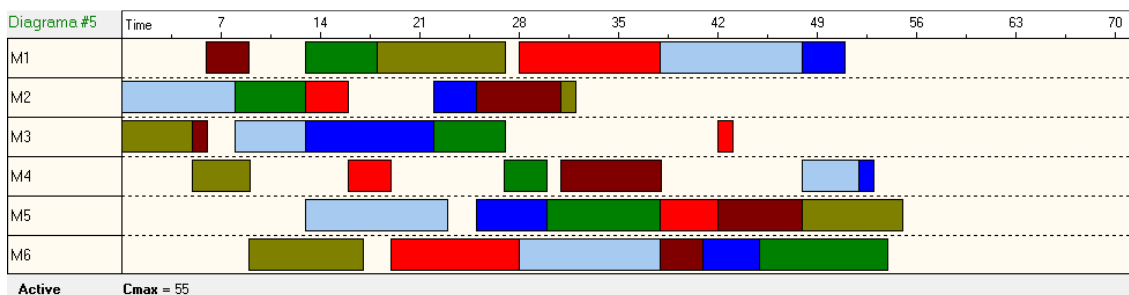


Figura 2.31. Gráfico de Gantt con valor de $C_{max} = 65$.

A continuación (Figura 2.32) se muestra un plan de trabajo en el cual, aún existe tiempo ocioso en algunas de sus máquinas, y se realiza el adelantamiento mediante lo que ha sido denominado en la presente investigación como “salto a la izquierda forzado” (*forced left jump*) de la operación que es posible adelantar y continúa con los correspondientes corrimientos a la izquierda. En la figura siguiente (Figura 2.33) se muestra el salto que ocurre en la máquina M3 y el corrimiento en las máquinas M2 y M5.


 Figura 2.32. Datos Gráfico de Gantt con valor de $C_{max} = 65$.

Esta forma de realizar el adelantamiento, a diferencia del anterior, trae consigo el retraso de una o más operaciones adelantadas, por lo cual, el tiempo de la operación que se adelanta en este ejemplo, es mayor que el tiempo de ocio que se aprovecha.


 Figura 2.33. Datos Gráfico de Gantt con valor óptimo de $C_{max} = 55$.

El resultado final ha sido que un plan de trabajo con una solución factible y un valor de camino máximo mayor, ha sido transformado en otro plan de trabajo factible y un valor de camino máximo menor.

En los anexos (Anexo 3) se encuentra el desarrollo completo de las transiciones realizadas a este plan de trabajo, cuyo valor inicial de *makespan* es $C_{máx} = 70$ y finaliza con un valor de $C_{máx} = 55$, el cual es el valor óptimo para esta instancia de problema. Como se puede apreciar, la MBUTO permite obtener mejoras en el proceso de búsqueda que en este caso algoritmo genético va realizando, y soluciones como la anterior, permanecían ocultas, a la espera de ser encontradas. De esta manera se cumple que la MBUTO ayuda a la metaheurística (AGS) a encontrar mejores soluciones durante su proceso de búsqueda. Esta mejora se puede ser aplicar, en principio, a otras

metaheurísticas que se utilizan en procesos de optimización similares al que aquí se presenta.

El algoritmo siguiente (Algoritmo 2.5) representa los pasos a seguir para que se ejecute el adelantamiento, ya sea por un típico salto a la izquierda o por un salto a la izquierda forzado. Se trata de localizar cuál máquina presenta tiempo ocioso y la operación que puede utilizar ese tiempo. Una vez que se conoce dicho tiempo se compara con el ocioso y el algoritmo determina la forma en que se realizará el adelantamiento. En el salto a la izquierda forzado “se fuerza” la reubicación de la operación adelantada y se realiza el correspondiente recalcule de las nuevas posiciones para las operaciones que resulten retrasadas en su inicio.

Como parte del proceso de desplazamiento hacia la izquierda, es necesario controlar que se realice sin ocasionar permutaciones indeseadas entre las operaciones de un mismo trabajo (pieza), lo cual constituiría una violación de la ruta tecnológica de obtención de la pieza. Una de las restricciones del problema plantea que cada trabajo tiene una ruta tecnológica, la cual es conocida e invariable, por lo tanto debe ser garantizada durante el proceso de desplazamiento a la izquierda con la finalidad de aprovechar tiempos de ocio en las máquinas.

Para garantizar lo anterior, se realizó un algoritmo (Algoritmo 2.6) capaz de controlar estos desplazamientos, sin permitir “saltos” en los que se intente violar el orden de ejecución de cada trabajo.

Algoritmo 2.5 Algoritmo para el adelantamiento por salto a la izquierda.

```
LEFTJUMP-FORCEDLEFTJUMP (secuencia, Opactual, Opanterior)
1  Hacer jblock[1...Jobs] = 0
2  Hacer mblock[1...Machines] = -1
3  posC = Opactual
4  posB = Opanterior
5  mientras posC > posB Y posC <> -1 hacer
6      jblock[0] = Opactual.Job
7      mblock[0] = Opanterior.Machine
8      Hacer Desplazamiento-Izquierdo(secuencia, posC, posB, jblock, mblock, 1, 1)
9      posC = secuencia.BuscarPosicion(posC)
10     posB = Buscar-posicion-en-secuencia()
11     Hacer jblock[1...Jobs] = 0
12     Hacer mblock[1...Machines] = -1
13     Ir al paso 5
14 Fin mientras
FIN
```

Algoritmo 2.6 Algoritmo para el control del desplazamiento a la izquierda de una operación.

```
DESPLAZAMIENTO-IZQUIERDO(secuencia, posC, posB, jblock, mblock, jcount, mcount)
1  ¿Es posC == 0?
2    True: Salir-Retornando(-1)
3  Hacer jfound = false
4  Hacer mfound = false
5  Repetir desde j = 0 hasta jcount con incremento(1) hacer
6    ¿Es [posC - 1].Job == jblock[j]?
7    True: Hacer jfound = true
8  Fin Repetir
9  Repetir desde m = 0 hasta mcount con incremento(1) hacer
10   ¿Es [posC - 1].Machine == mblock?
11   True: Hacer mfound = true
12 Fin Repetir
13 ¿Es jfound == false Y mfound == true?
14 Si: Hacer jblock[jcount] = [posC - 1].Job
15   jcount++
16 Fin Repetir
17 ¿Es mfound == false Y jfound == true?
18 True: Hacer mblock[mcount] = [posC - 1].Machine
19   mcount++
20 Fin Repetir
21 ¿Es (jfound == true O mfound == true) Y posC - 1 <> posB?
22 Si: Hacer k = DESPLAZAMIENTO-IZQUIERDO(secuencia, posC, posB, jblock, mblock, jcount, mcount)
23 ¿Es k <> -1?
24 True: Hacer Intercambiar(posB, posC)
25   Salir-Retornando(posC - 1)
26 Salir-Retornando(-1)
FIN
```

El método completo finalmente es el encargado de determinar si el adelantamiento de una operación será realizado por un salto a la izquierda, o es necesario aplicar un salto a la izquierda forzado. También, incluye la evaluación del objetivo que se persigue. En el algoritmo que continúa (Algoritmo 2.7) se realiza la comparación del *makespan* antes de producirse el adelantamiento, por cualquiera de las formas antes descritas, y luego de realizado, para que finalmente se conserve la mejor solución obtenida. En este ejemplo, la evaluación del *makespan* constituye la función objetivo del problema.

Algoritmo 2.7 Algoritmo MBUTO.

```
MBUTO (secuencia)
1  Recorrer secuencia desde  $Op_{1,0}$  hasta  $Op_{n,m}$  hacer
2  ¿Existe tiempo de ocio para  $Op_{i,j}$ ?
3      True:  $Op'_{a,b}$  = secuencia.Obtener-Operación-que-admite-adelantamiento()
4          Cmax = secuencia.GetMakespan()
5          Hacer LeftJump-ForcedLeftJump(secuencia,  $Op_{i,j}$ ,  $Op'_{a,b}$ )
6          Cmax' = secuencia.GetMakespan()
7      ¿Es Cmax' <= Cmax?
8          True: Hacer  $Op_{i,j} = Op_{1,0}$ 
9              Ir al paso 1
10         False: Deshacer LeftJump-ForcedLeftJump(secuencia,  $Op_{i,j}$ ,  $Op'_{a,b}$ )
11             Ir al paso 1
12 Fin Recorrer
FIN
```

2.12. Conclusiones del capítulo

En este capítulo se han analizado algunas de las representaciones utilizadas para especificar problemas de *scheduling*, así como las principales variantes que se presentan en el taller de mecanizado. Se estudió además el universo de soluciones posibles en correspondencia con la forma de obtener las secuencias y se establecieron varios criterios para la optimización de planes de trabajo. Se presentó a Algoritmos Genéticos como la metaheurística utilizada para la solución del problema, conjuntamente con un método para mejorar el rendimiento en el proceso de búsqueda de soluciones óptimas.

De todo lo anterior se concluye que:

1. La representación del problema del *scheduling* mediante un grafo disyuntivo dirigido es apropiada al permitir cumplir con sus restricciones.
2. Se estableció para cada variante la forma de cálculo de secuencias para obtener el universo total de soluciones factibles.
3. Se estableció un modelo matemático para medir el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo, como criterio para la optimización.
4. La representación genética utilizada para cada variante resultó apropiada según la forma de obtener las secuencias y para el funcionamiento del Algoritmo Genético.
5. Los algoritmos realizados para la recuperación de cromosomas inválidos permitió un mejor rendimiento en el funcionamiento del Algoritmo Genético.
6. El método para la Mejora Basada en el Uso del Tiempo Ocioso (MBUTO), fue posible utilizarlo con el Algoritmo Genético con el propósito de ayudar en el proceso de búsqueda de mejores soluciones.

Capítulo 3

ANÁLISIS DE RESULTADOS

3.1. Introducción

Dado que los algoritmos genéticos tienen un gran basamento probalístico y no exacto, su validez como método de búsqueda de soluciones debe ser realizada de forma experimental. En general se deben evaluar no solamente la eficacia y la eficiencia, como en cualquier otro método de búsqueda, sino también la estabilidad.

En el caso de los problemas de *scheduling* existen bancos de prueba (*benchmark problems*) de uso común entre los investigadores, lo cual facilita la comparación de distintos métodos de resolución.

3.2. Estudio experimental

En los resultados que forman parte de la presente investigación se encuentra la solución a un total de 162 problemas de scheduling, de ellos 80 de la variante de flujo regular y 82 problemas de la variante de flujo general. En las tablas que se muestran a continuación se encuentran los resultados obtenidos para estos problemas, los cuales forman parte de la biblioteca de investigación de operaciones *OR- Library* (Anexo 4), donde además existen otros tipos de problemas para otras aplicaciones.

La siguiente tabla (Tabla 3.1) muestra una serie de resultados obtenidos

Tabla 3.1. Resultados obtenidos para problemas de tipo *flow shop*. Instancias ta001 – ta040.

Instancia	N	M	CI	CS	AGS
ta001	20	5	1232	1278	1278
ta002	20	5	1290	1359	1359
ta003	20	5	1073	1081	1081
ta004	20	5	1268	1293	1293
ta005	20	5	1198	1236	1235
ta006	20	5	1180	1195	1195
ta007	20	5	1226	1239	1239
ta008	20	5	1170	1206	1206
ta009	20	5	1206	1230	1230
ta010	20	5	1082	1108	1108
ta011	20	10	1448	1582	1594
ta012	20	10	1479	1659	1678
ta013	20	10	1407	1496	1509
ta014	20	10	1308	1378	1395
ta015	20	10	1325	1419	1426
ta016	20	10	1290	1397	1411
ta017	20	10	1388	1484	1496
ta018	20	10	1363	1538	1554
ta019	20	10	1472	1593	1608
ta020	20	10	1356	1591	1613
ta021	20	20	1911	2297	2315
ta022	20	20	1711	2100	2099
ta023	20	20	1844	2326	2349
ta024	20	20	1810	2223	2229
ta025	20	20	1899	2291	2314
ta026	20	20	1875	2226	2247
ta027	20	20	1875	2273	2296
ta028	20	20	1880	2200	2217
ta029	20	20	1840	2237	2255
ta030	20	20	1900	2178	2196
ta031	50	5	2712	2724	2724
ta032	50	5	2808	2834	2840
ta033	50	5	2596	2621	2622
ta034	50	5	2740	2751	2753
ta035	50	5	2837	2863	2864
ta036	50	5	2793	2829	2832
ta037	50	5	2689	2725	2732
ta038	50	5	2667	2683	2704
ta039	50	5	2527	2552	2561
ta040	50	5	2776	2782	2782

Tabla 3.2. Resultados obtenidos para problemas de tipo *flow shop*. Instancias ta041 – ta080.

Instancia	N	M	CI	CS	AGS
ta041	50	10	2907	3025	3073
ta042	50	10	2821	2892	2964
ta043	50	10	2801	2864	2927
ta044	50	10	2968	3064	3071
ta045	50	10	2908	2986	3043
ta046	50	10	2941	3006	3057
ta047	50	10	3062	3107	3127
ta048	50	10	2959	3039	3074
ta049	50	10	2795	2902	2956
ta050	50	10	3046	3091	3132
ta051	50	20	3480	3875	3958
ta052	50	20	3424	3715	3888
ta053	50	20	3351	3668	3801
ta054	50	20	3336	3752	3826
ta055	50	20	3313	3635	3795
ta056	50	20	3460	3698	3797
ta057	50	20	3427	3716	3874
ta058	50	20	3383	3709	3839
ta059	50	20	3457	3765	3880
ta060	50	20	3438	3777	3875
ta061	100	5	5437	5493	5493
ta062	100	5	5208	5268	5284
ta063	100	5	5130	5175	5205
ta064	100	5	4963	5014	5023
ta065	100	5	5195	5250	5255
ta066	100	5	5063	5135	5146
ta067	100	5	5198	5246	5246
ta068	100	5	5038	5106	5106
ta069	100	5	5385	5454	5467
ta070	100	5	5272	5328	5342
ta071	100	10	5759	5770	5835
ta072	100	10	5345	5349	5397
ta073	100	10	5623	5677	5691
ta074	100	10	5732	5791	5843
ta075	100	10	5431	5468	5508
ta076	100	10	5246	5303	5313
ta077	100	10	5523	5599	5667
ta078	100	10	5556	5623	5683
ta079	100	10	5779	5875	5922
ta080	100	10	5830	5845	5903

Los resultados obtenidos para problemas de la variante *Job Shop Scheduling*, han sido avalados, mediante la solución de 82 problemas propuestos por otros investigadores. Un ejemplo lo constituye el conjunto (40 en total) de instancias de problemas propuestos por S. Lawrence (Beasley, 1990), conocidas como *Lawrence's benchmark problems* de las cuales se muestran los resultados en la siguiente tabla (Tabla 3.3).

La primera columna ofrece el nombre dado a cada instancia. Las siguientes columnas indican el tamaño: número de trabajos N , número de máquinas M . La que continúa se refiere al valor de la cota inferior (*Lower Bound, LB*). Las demás corresponden a resultados obtenidos por otros investigadores, reportado en (Yamada, 2003). Por ese orden, T. Yamada, D. Applegate, P. van Laarhoven, H. Matsuo, J. Adams y P. Brucker. La última columna ha sido adicionada con el propósito de mostrar los resultados alcanzados por nuestra propuesta. El asterisco simple indica el valor óptimo o mejor valor mínimo conocido. En negritas se ha querido destacar los resultados que han sido superados o igualados en la presente investigación.

A continuación otras tablas (Tabla 3.4, a 3.8) muestran los resultados alcanzados en la solución del resto de los problemas resueltos para esta variante.

En la siguiente figura (Figura 3.2) se muestra en forma de gráfico de Gantt una de las soluciones obtenidas. Cada trabajo tiene un color diferente. De nuevo un mismo color significa las distintas operaciones de un trabajo en cada máquina visitada. Constituye un típico problema JSS en el cual cada trabajo tiene un orden (ruta tecnológica) diferente. En la sección de anexos (Anexo 6) se muestran otras soluciones de problemas de esta variante, en forma de gráfico de Gantt.

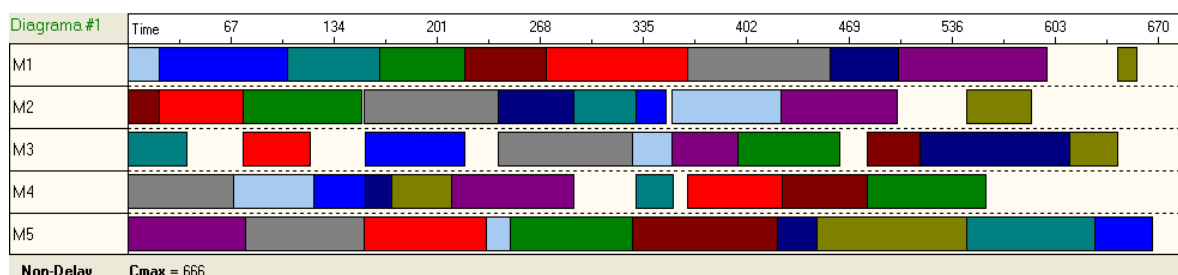


Figura 3.2. Gráfico de Gantt de un plan de trabajo tipo *job shop*. Corresponde a la solución de la instancia de problema la01. Valor óptimo de $C_{max} = 666$.

Tabla 3.3. Resultados obtenidos para problemas *Job Shop*. Las instancias corresponden a S. Lawrence. (*Lawrence's benchmark problems*).

Instancia	N	M	LB	Yamad	Appl	Laar	Mats	Adams	Bruck	Márq
la01	10	5	---	*666	*666	*666	---	*666	*666	*666
la02	10	5	---	*655	*655	*655	*655	669	*655	*655
la03	10	5	---	*597	*597	606	*597	605	*597	*597
la04	10	5	---	*590	*590	*590	*590	593	*590	*590
la05	10	5	---	*593	*593	*593	---	*593	*593	*593
la06	15	5	---	*926	*926	*926	---	*926	*926	*926
la07	15	5	---	*890	*890	*890	---	*890	*890	*890
la08	15	5	---	*863	*863	*863	*863	*863	*863	*863
la09	15	5	---	*951	*951	*951	---	*951	*951	*951
la10	15	5	---	*958	*958	*958	---	*958	*958	*958
la11	20	5	---	*1222	*1222	*1222	---	*1222	*1222	*1222
la12	20	5	---	*1039	*1039	*1039	---	1239	*1039	*1039
la13	20	5	---	*1150	*1150	*1150	---	*1150	*1150	*1150
la14	20	5	---	*1292	*1292	*1292	---	*1292	*1292	*1292
la15	20	5	---	*1207	*1207	*1207	---	*1207	*1207	*1207
la16	10	10	---	*945	*945	956	959	978	*945	*945
la17	10	10	---	*784	*784	*784	*784	787	*784	*784
la18	10	10	---	*848	*848	861	*848	859	*848	*848
la19	10	10	---	*842	*842	848	*842	860	*842	*842
la20	10	10	---	907	*902	*902	907	914	*902	907
la21	15	10	1046	1050	1053	1063	1071	1084	1059	1050
la22	15	10	---	935	*927	938	*927	944	*927	935
la23	15	10	---	*1032	*1032	*1032	*1032	*1032	*1032	*1032
la24	15	10	935	943	*935	952	973	976	*935	946
la25	15	10	977	985	*977	992	991	1017	977	978
la26	20	10	---	*1218	*1218	*1218	*1218	1224	*1218	*1218
la27	20	10	1235	1262	1269	1269	1274	1291	1270	1246
la28	20	10	---	*1216	*1216	1224	1216	1250	1276	1225
la29	20	10	1130	1188	1195	1203	1196	1239	1202	1189
la30	20	10	---	*1355	*1355	*1355	*1355	*1355	*1355	*1355
la31	30	10	---	*1784	*1784	*1784	---	*1784	*1784	*1784
la32	30	10	---	*1850	*1850	*1850	---	*1850	*1850	*1850
la33	30	10	---	*1719	*1719	*1719	---	*1719	*1719	*1719
la34	30	10	---	*1721	*1721	*1721	---	*1721	*1721	*1721
la35	30	10	---	*1888	*1888	*1888	---	*1888	*1888	*1888
la36	15	15	---	1291	*1268	1293	1292	1305	1268	1297
la37	15	15	---	1420	*1397	1433	1435	1423	1424	1425
la38	15	15	1196	1209	1209	1215	1231	1255	1232	1229
la39	15	15	---	1243	*1233	1248	1251	1273	1233	1263
la40	15	15	1222	1235	*1222	1234	1235	1269	1238	1241

Tabla 3.4. Resultados de *makespan* alcanzados utilizando los problemas de tipo *Job shop* pertenecientes a la clase ORB (*Applegate & Cook benchmark problems*).

Instancia	N	M	CI	CS	AGS + MBUTO
orb1	10	10	1059	1059	1059
orb2	10	10	888	888	889
orb3	10	10	1005	1005	1020
orb4	10	10	1005	1005	1011
orb5	10	10	887	887	889
orb6	10	10	1010	1010	1013
orb7	10	10	397	397	397
orb8	10	10	899	899	908
orb9	10	10	934	934	934
orb10	10	10	944	944	944

Tabla 3.5. Resultados de *makespan* alcanzados utilizando los problemas de tipo *Job shop* pertenecientes a la clase ABZ (*Adams-Balas-Zawack benchmark problems*).

Instancia	N	M	CI	CS	AGS + MBUTO
abz5	10	10	1234	1234	1238
abz6	10	10	943	943	947
abz7	20	15	656	656	683
abz8	20	15	646	665	700
abz9	20	15	662	679	712

Tabla 3.6. Resultados de *makespan* alcanzados utilizando los problemas de tipo *Job shop* pertenecientes a la clase YN (*Yamada & Nakano benchmark problems*).

Instancia	N	M	CI	CS	AGS + MBUTO
yn1	20	20	846	885	935
yn2	20	20	870	909	958
yn3	20	20	840	892	923
yn4	20	20	920	968	1012

Tabla 3.7. Resultados de *makespan* alcanzados utilizando los problemas de tipo *Job Shop* pertenecientes a la clase FT (*Fisher & Thompson benchmark problems*).

Instancia	N	M	CI	CS	AGS + MBUTO
ft06	6	6	55	55	*55
ft10	10	10	930	930	936
ft20	20	5	1165	1165	*1165

Tabla 3.8. Resultados de *makespan* alcanzados utilizando los problemas de tipo *Job shop* pertenecientes a la clase SWV (*Storer-Wu-Vaccari benchmark problems*).

Instancia	N	M	CI	CS	AGS + MBUTO
swv01	20	10	1407	1407	1489
swv02	20	10	1475	1475	1545
swv03	20	10	1398	1398	1486
swv04	20	10	1450	1474	1561
swv05	20	10	1424	1424	1531
swv06	20	15	1591	1673	1862
swv07	20	15	1447	1600	1751
swv08	20	15	1641	1759	1912
swv09	20	15	1605	1661	1830
swv10	20	15	1632	1761	1916
swv11	50	10	2983	2938	3347
swv12	50	10	2972	3003	3481
swv13	50	10	3104	3104	3524
swv14	50	10	2968	2968	3255
swv15	50	10	2885	2903	3286
swv16	50	10	2924	2924	2924
swv17	50	10	2794	2794	2794
swv18	50	10	2852	2852	2852
swv19	50	10	2843	2843	2843
swv20	50	10	2823	2823	2823

La tabla que continúa (Tabla 3.9) presenta un grupo seleccionado de problemas anteriores, lo cuales son considerados entre los diez de mayor dificultad (*Ten difficult benchmark job shop scheduling problems*) para encontrar soluciones óptimas o cercanas a ésta. La publicación de referencia a estas tablas es la Tesis doctoral del autor Takeshi Yamada, Universidad de Kyoto, 2003 (Yamada, 2003). Las columnas muestran resultados obtenidos por otros investigadores, tales como Nowicki (Nowicki y Smutnicki, 1996), Dell'Amico (Dell'Amico y Trubian, 1993), Aarts (Aarts, *et al.*, 1994), D.C. Mattfeld (Kopfer, *et al.*, 1994), D. Applegate (Applegate y Cook, 1991), P. van Laarhoven (Laarhoven, *et al.*, 1992), H. Matsuo (Matsuo, *et al.*, 1988), J. Adams (Adams, *et al.*, 1988) y P. Brucker (Brucker, *et al.*, 1994). La última columna ha sido adicionada con el propósito de mostrar los resultados alcanzados en la presente investigación por su autor (J. E. Márquez D.). El asterisco simple indica el valor óptimo o mejor valor mínimo conocido. En negritas se destacan los resultados que han sido superados o igualados. En la

sección de anexos (Anexo 7) se encuentra, en forma de gráficos de Gantt, la solución encontrada a cada uno de estos problemas.

Tabla 3.9. Resultados de *makespan* alcanzados en la presente investigación utilizando los problemas seleccionados de mayor dificultad.

Instancia	N	M	LB	Yamad	Nowi	Dell	Aarts	Matt	Appl	Laar	Mats	Adams	Bruck	Márq
abz7	20	15	655	665	---	667	668	672	668	---	---	710	---	683
abz8	20	15	638	675	---	678	670	683	687	---	---	716	---	700
abz9	20	15	656	686	---	692	691	703	707	---	---	735	---	712
la21	15	10	1046	*1046	1047	1048	1053	1053	1053	1063	1071	1084	1059	1050
la24	15	10	935	*935	939	941	*935	938	*935	952	973	976	935	955
la25	15	10	977	*977	*977	979	983	*977	*977	992	991	1017	977	978
la27	20	10	1235	*1235	1236	1242	1249	1236	1269	1269	1274	1291	1270	1246
la29	20	10	1130	1154	1160	1182	1185	1184	1195	1203	1196	1239	1202	1188
la38	15	15	1196	1198	1196	1203	1208	1201	1217	1215	1231	1255	1232	1229
la40	15	15	1222	1228	1229	1233	1225	1228	*1222	1234	1235	1269	1238	1241

3.3 Parámetros utilizados en el Algoritmo Genético Simple

Para las diferentes ejecuciones realizadas para el caso particular *PFSS*, el algoritmo genético fue regulado con los siguiente parámetros:

- tamaño de población: 100
- número de generaciones: 1000
- método de selección: ranking
- método de cruzamiento: 1 punto de cruce
- factor de cruzamiento: 0.6
- factor de mutación: 0.02
- elitismo: habilitado

Para la variante *JSS* los parámetros seleccionados para el algoritmo genético fueron los siguientes:

- tamaño de población: 250
- número de generaciones: 5000
- método de selección: ranking
- método de cruzamiento: 2 puntos de cruce
- factor de cruzamiento: 0.8
- factor de mutación: 0.01
- elitismo: habilitado

3.4. Estudio experimental. Materiales y métodos

Como parte de la validación de la propuesta se utilizó el problema *la15* (*Lawrence's benchmark problems*) cuyo valor óptimo de camino máximo es $C_{m\acute{a}x} = 1207$, (ver Tabla 3.3). El experimento fue realizado como se explica a continuación, con la finalidad de comparar la efectividad de la propuesta en cuanto al rendimiento y a la calidad de las soluciones encontradas:

- Se realizó la ejecución solo del Algoritmo Genético Simple (GAS).
- Se realizó la ejecución del Algoritmo Genético Simple más la Mejora Basada en el Uso del Tiempo Ocioso (GAS + MBUTO).

En ambos casos se efectuaron 100 repeticiones del experimento y los parámetros utilizados en el algoritmo genético fueron los siguientes:

- tamaño de población: 50
- método de selección: torneo
- método de cruzamiento: 2 puntos de cruce a nivel de gen
- factor de cruzamiento: 0.5
- factor de mutación: 0.01
- elitismo: habilitado

Para realizar la comparación entre AGS y AGS + MBUTO, se tuvieron en cuenta los siguientes indicadores:

- Valores promedios de C_{max} de cada ejecución.
- Valores máximos de C_{max} de cada ejecución.
- Valores mínimos de C_{max} de cada ejecución.
- Número de generaciones para encontrar la mejor solución de C_{max} .
- Tiempo empleado para encontrar la mejor solución de C_{max} .

3.4.1. Experimento No. 1

Para este primer experimento se fijó el número de generaciones en 100. Se registraron los valores mínimos (mejores), máximos (peores) y promedio, reportados en cada una de las ejecuciones realizadas, con el propósito de efectuar las correspondientes comparaciones. Los resultados de las ejecuciones que se muestran a continuación (Tabla 3.10), muestran que con los parámetros anteriores fue insuficiente para que el AGS encontrara el valor óptimo. En cambio al ser ayudado por la MBUTO en todos los casos se pudo encontrar dicho valor.

Tabla 3.10. Resultados de las ejecuciones realizadas. Valores máximos y mínimos de C_{max} .

Ejecución	AGS		AGS + MBUTO	
	Máximo	Mínimo	Máximo	Mínimo
1	1724	1298	1266	1207
2	1626	1293	1299	1207
3	1496	1326	1262	1207
4	1649	1316	1280	1207
5	1714	1300	1308	1207
6	1615	1300	1279	1207
7	1662	1345	1291	1207
8	1581	1314	1285	1207
9	1696	1305	1276	1207
10	1678	1333	1262	1207
11	1598	1304	1275	1207
12	1615	1392	1281	1207
13	1665	1316	1300	1207
14	1714	1281	1243	1207
15	1602	1323	1276	1207
16	1647	1311	1298	1207
17	1675	1324	1270	1207
18	1621	1313	1243	1207
19	1674	1302	1269	1207
20	1604	1350	1312	1207
21	1608	1270	1259	1207
22	1571	1320	1264	1207
23	1667	1323	1291	1207
24	1638	1341	1287	1207
25	1729	1308	1261	1207
26	1672	1384	1288	1207
27	1730	1301	1272	1207
28	1631	1373	1261	1207
29	1674	1314	1294	1207
30	1714	1291	1327	1207
31	1696	1300	1286	1207
32	1614	1283	1290	1207
33	1703	1310	1321	1207
34	1671	1326	1294	1207
35	1641	1364	1280	1207

Tabla 3.10 *Continuación.*

Ejecución	AGS		AGS + MBUTO	
	Máximo	Mínimo	Máximo	Mínimo
36	1604	1329	1267	1207
37	1658	1330	1232	1207
38	1580	1308	1282	1207
39	1751	1345	1250	1207
40	1746	1295	1273	1207
41	1721	1333	1287	1207
42	1612	1317	1313	1207
43	1565	1333	1277	1207
44	1708	1314	1272	1207
45	1670	1313	1264	1207
46	1693	1298	1301	1207
47	1651	1310	1297	1207
48	1531	1297	1300	1207
49	1621	1343	1262	1207
50	1575	1310	1304	1207
51	1660	1315	1270	1207
52	1615	1327	1257	1207
53	1613	1302	1326	1207
54	1617	1280	1305	1207
55	1722	1326	1279	1207
56	1705	1304	1259	1207
57	1646	1330	1296	1207
58	1582	1354	1301	1207
59	1664	1291	1320	1207
60	1610	1321	1267	1207
61	1694	1321	1264	1207
62	1607	1289	1306	1207
63	1607	1307	1279	1207
64	1669	1330	1280	1207
65	1705	1281	1292	1207
66	1646	1316	1310	1207
67	1585	1345	1278	1207
68	1669	1298	1262	1207
69	1603	1307	1274	1207
70	1664	1307	1285	1207

Tabla 3.10 *Continuación.*

Ejecución	AGS		AGS + MBUTO	
	Máximo	Mínimo	Máximo	Mínimo
71	1599	1312	1306	1207
72	1599	1316	1270	1207
73	1636	1286	1286	1207
74	1636	1286	1311	1207
75	1619	1305	1307	1207
76	1668	1331	1280	1207
77	1577	1280	1325	1207
78	1741	1307	1245	1207
79	1648	1263	1296	1207
80	1692	1329	1251	1207
81	1682	1341	1246	1207
82	1631	1302	1252	1207
83	1661	1319	1296	1207
84	1620	1322	1285	1207
85	1615	1336	1288	1207
86	1641	1367	1284	1207
87	1650	1341	1258	1207
88	1641	1265	1290	1207
89	1707	1298	1292	1207
90	1699	1304	1270	1207
91	1638	1311	1308	1207
92	1669	1326	1278	1207
93	1613	1313	1251	1207
94	1675	1334	1283	1207
95	1554	1317	1275	1207
96	1686	1341	1312	1207
97	1698	1313	1268	1207
98	1690	1312	1299	1207
99	1658	1325	1299	1207
100	1661	1311	1301	1207

Las siguientes figuras (Figuras 3.3 y 3.4) muestran en forma de gráficos de Gantt los mejores valores mínimos de camino máximo registrados en la tabla anterior.

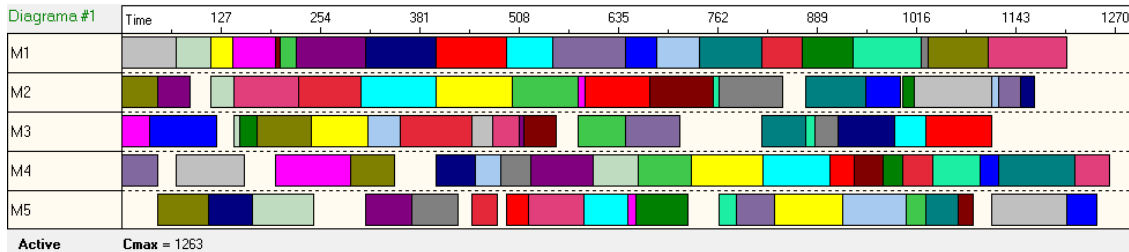


Figura 3.3. Gráfico de Gantt con la mejor solución encontrada por AGS. Valor de $C_{max} = 1263$. Corresponde a la ejecución número 79.

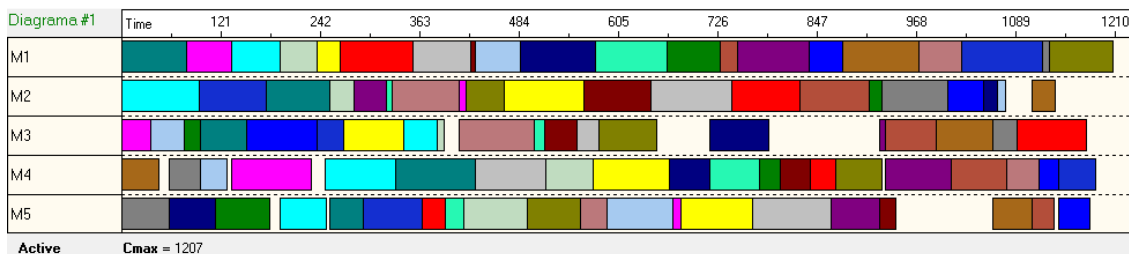


Figura 3.4. Gráfico de Gantt con la mejor solución encontrada por AGS + MBUTO. Valor óptimo de $C_{max} = 1207$.

El siguiente gráfico (Figura 3.5) ilustra a modo de comparación, los resultados obtenidos en este experimento. Se seleccionaron las cinco primeras ejecuciones en ambos casos, y se registró el valor de *makespan* obtenido en cada generación hasta alcanzar el número fijado de 100 iteraciones. Como se mencionó antes, AGS no consiguió en ningún caso alcanzar el valor óptimo para este problema, mientras que al ser ayudado (AGS + MBUTO) en todos los casos alcanzó este valor. Se puede destacar la mejoría que tiene lugar desde las primeras generaciones.

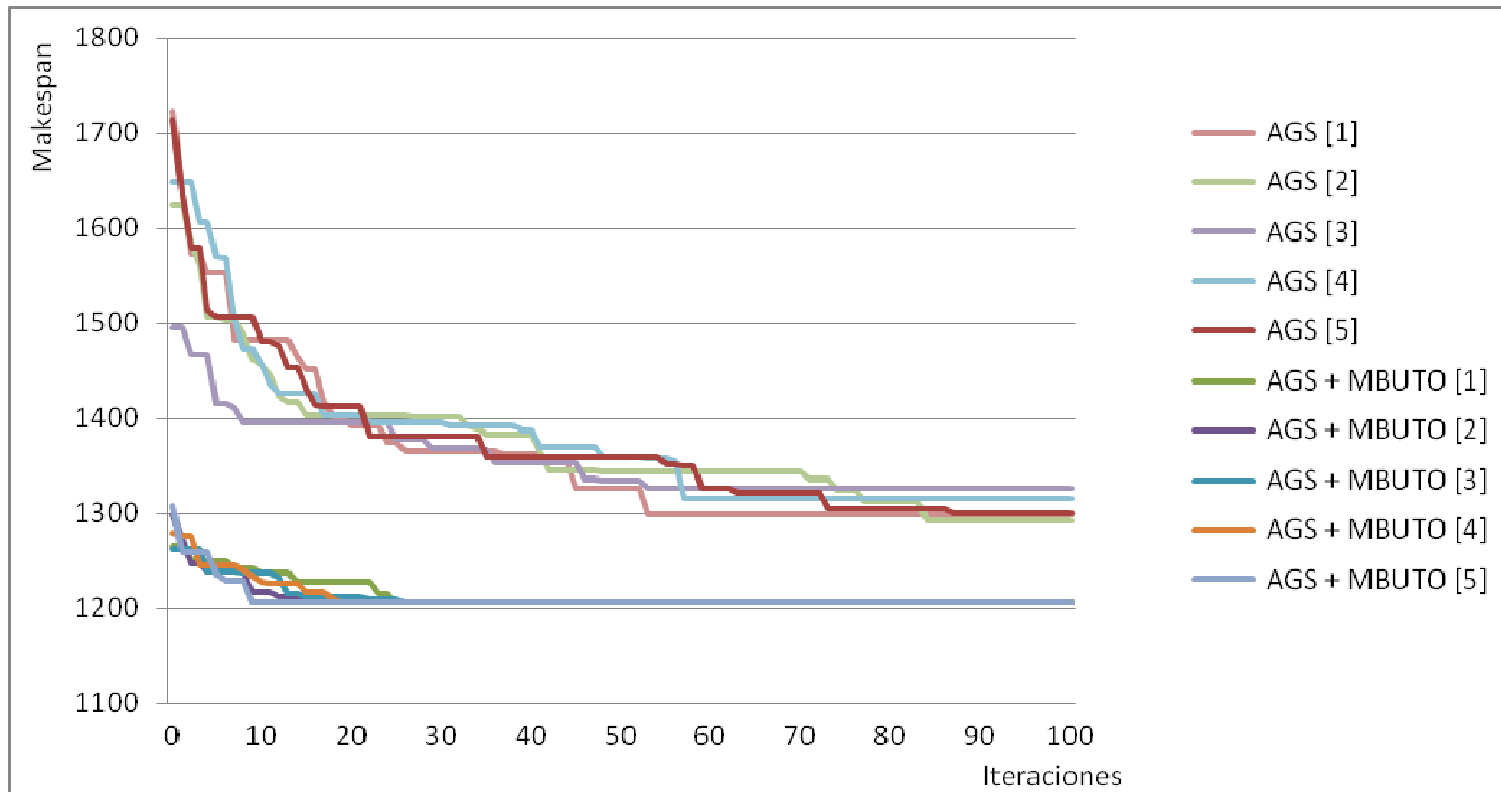


Figura 3.5. Gráfico comparativo del rendimiento entre AGS y AGS + MBUTO. Instancia de problema la15. Valor óptimo de $C_{max} = 1207$.

3.4.2. Experimento No. 2

Este consistió en realizar ejecuciones hasta alcanzar el valor óptimo para el problema seleccionado, o sea, sin limitar el número de generaciones necesarias por el AGS para encontrar dicha solución. De esta forma se pudo comparar:

- El número de generación necesario para encontrar el valor óptimo de C_{max}
- El tiempo empleado en encontrar este valor óptimo de C_{max} .

La siguiente tabla (Tabla 3.11) presenta los resultados de las ejecuciones realizadas. El tiempo fue medido en milisegundos. La primera columna significa el número de la ejecución. Las restantes se refieren a la generación en la cual se alcanzó dicho valor óptimo y el tiempo empleado, por el AGS primeramente, y a continuación por el AGS con la ayuda del método de Mejora MBUTO.

Tabla 3.11. Resultados de las ejecuciones realizadas: generación y tiempo.

Ejecución	GAS		GAS + MBUTO	
	Generación	Tiempo	Generación	Tiempo
1	775	12287	13	5708
2	5568	95207	33	14591
3	2369	44634	9	3565
4	2065	42772	12	4697
5	1691	37774	29	12137
6	1159	28191	19	7821
7	3247	85192	8	3325
8	1060	29483	3	951
9	787	22813	19	8462
10	1185	35711	11	4337
11	1780	58855	16	6560
12	1425	50062	13	5488
13	2421	88517	15	6330
14	1928	75979	13	7571
15	1531	62961	26	11927
16	1171	50032	24	11056
17	1882	78463	18	7912
18	2048	88357	41	17566
19	4085	185397	28	11547
20	498	24325	22	10195
21	2369	115946	26	12107
22	1124	56040	44	19579
23	2373	123057	14	6399
24	445	23644	15	6699
25	1799	97731	38	16794
26	5210	295725	9	3825
27	777	45265	41	18627
28	949	56932	20	8512
29	2203	139110	17	7911
30	411	27540	14	5868
31	1785	118020	34	14040
32	851	57472	21	9303
33	3396	232134	27	11747
34	1600	111481	9	4176
35	3105	221158	21	9583

Tabla 3.11 *Continuación.*

Ejecución	GAS		GAS + MBUTO	
	Generación	Tiempo	Generación	Tiempo
36	1295	95257	12	5578
37	1576	117599	8	3235
38	744	56862	11	4486
39	2673	229550	23	10685
40	876	81627	15	7250
41	2474	213777	22	9263
42	849	71292	14	6169
43	1617	137708	14	6699
44	1512	132421	8	3325
45	2438	217663	11	4196
46	1636	147362	16	6639
47	1551	143156	35	15943
48	616	57422	14	6259
49	3458	325989	12	5017
50	1548	150106	35	14401
51	1876	185056	35	15372
52	2077	207518	12	5138
53	816	82759	26	10916
54	1759	181340	69	29533
55	1325	138940	10	4196
56	685	72715	18	8312
57	1747	188281	32	14320
58	1325	145429	11	4466
59	1729	192497	16	6880
60	2299	266433	7	3235
61	1518	193979	11	4947
62	785	94496	21	9324
63	1443	169964	39	17686
64	1784	231567	23	10756
65	2465	298449	17	7411
66	2033	249139	17	7150
67	1934	240415	17	7251
68	844	106103	20	9073
69	1165	148524	6	2544
70	1449	187029	13	5768

Tabla 3.11 *Continuación.*

Ejecución	GAS		GAS + MBUTO	
	Generación	Tiempo	Generación	Tiempo
71	658	85713	22	10034
72	918	120593	34	15091
73	661	87817	27	11887
74	598	79905	17	7340
75	1481	200218	9	4066
76	741	101095	20	9083
77	1627	224583	21	8662
78	1484	207128	20	8332
79	562	78963	17	7741
80	1120	158949	20	9103
81	1090	156205	39	16203
82	2433	353548	14	6279
83	1255	184315	32	14551
84	1894	280964	65	27950
85	843	126011	12	5468
86	793	119372	47	20269
87	3357	510955	21	9274
88	6016	914735	17	7521
89	949	144959	11	4477
90	1079	166169	22	10526
91	2134	331917	11	4887
92	1694	265802	43	19968
93	730	115386	15	6139
94	1021	162814	24	10555
95	2581	415618	23	10665
96	1015	164817	7	2634
97	621	101566	29	13239
98	1845	305299	12	5548
99	1289	215971	15	6780
100	1890	318738	16	7170

3.5. Análisis estadístico

En cada uno de los indicadores se evaluaron los resultados obtenidos sin la aplicación de la mejora y con la aplicación de la misma, o sea, AGS y AGS + MBUTO respectivamente.

Para cada variante se determinaron varios estadígrafos descriptivos (Media, Mediana, valores mínimos y máximos, rangos interpercentil e intercuartil, desviación estándar y curtosis). Antes de seleccionar la prueba estadística para comparar los resultados se valoró en cada serie el cumplimiento del supuesto teórico del ajuste a la distribución normal, invariablemente en todos los casos no se cumplió (Anexo 8). Por lo cual se utilizó la prueba de distribución libre de *Wald-Wolfowitz* y se estableció el nivel de significación para: $p \leq 0.05$.

3.5.1. Resultados

La evaluación de la eficacia del uso del procedimiento AGS + MBUTO, se desarrolló mediante el cálculo de los valores descriptivos y diferencia entre promedios, tanto de los valores promedios, máximos, mínimos de C_{max} y número de generación y tiempo para encontrar la mejor solución.

Al observar los valores descriptivos de las variables analizadas (Tabla 3.12), sugieren que cuando se utiliza el método propuesto, mejora sustancialmente la variabilidad, por lo que se puede apreciar que el rango de búsqueda disminuye, tal es así que para los valores mínimos de C_{max} la variabilidad es cero, obsérvese además que para todos los casos los valores descriptivos son menores cuando se emplea AGS + MBUTO. En las tres primeras variables, promedios, valores máximos, y valores mínimos de C_{max} , la variabilidad es baja en ambos casos, el coeficiente de variación (CV) para el AGS fluctúa entre 1,5688 a 2,9893 y para el AGS + MBUTO entre 0 y 1,6047. Sin embargo el número de generaciones para encontrar la mejor solución y el tiempo dedicado a ello, son mucho más variables en el AGS: 60,9641 y 78,7640 respectivamente y en AGS + MBUTO: 56,5814 y 56,5882 respectivamente. Los valores obtenidos de curtosis para todas las variables muestran el no ajuste de estas series a la distribución normal, ya que esto ocurre solo cuando la curtosis es igual a 3, lo cual corrobora los resultados obtenidos al aplicar las pruebas estadísticas de *Kolmogorov y Smirnov (K-S)* y la de *Lilliefors* (Anexo 8), las cuales en todos los casos muestran diferencias significativas entre la curva

esperada de la distribución de probabilidad normal y la observada para $p < 0.01$. Este elemento sirvió de criterio para utilizar una prueba de distribución libre para calcular las diferencias entre los promedios de las medidas estudiadas.

La tabla siguiente muestra los resultados del análisis estadístico realizado.

Tabla 3.12. Valores descriptivos de los promedios de C_{max} .

Valores promedios de C_{max}			
ESTADIGRAFOS		AGS	AGS + MBUTO
Media		1372,200	1212,251
Valor	mínimo	1320,900	1207,750
	máximo	1435,310	1219,240
±Desviación típica		21,5276	2,27601
Coeficiente de Variación		1,5688	0,1877
Curtosis		0,3827	1,0157
Valores máximos de C_{max}			
Media		1648,880	1282,530
Valor	mínimo	1496,000	1232,000
	máximo	1751,000	1327,000
±Desviación típica		49,29119	20,58118
Coeficiente de Variación		2,9893	1,6047
Curtosis		-0,004701	-0,446968
Valores mínimos de C_{max}			
Media		1315,930	1207,000
Valor	mínimo	1263,000	1207,000
	máximo	1392,000	1207,000
±Desviación típica		23,25930	0,00000
Coeficiente de Variación		1,7675	0,00000
Curtosis		1,202714	-
Valores del número generaciones en encontrar la mejor solución			
Media		1673,720	20,740
Valor	mínimo	411,0000	3,0000
	máximo	6016,000	69,000
±Desviación típica		1020,369	11,735
Coeficiente de Variación		60,9641	56,5814
Curtosis		5,254417	3,529166
Valores de tiempo para encontrar la mejor solución			
Media		153768,2	9117,8
Valor	mínimo	12287,00	951,00
	máximo	914735,0	29533,0
±Desviación típica		121114,1	5159,6
Coeficiente de Variación		78,7640	56,5882
Curtosis		15,2084	2,9196

La eficacia del uso del AGS + MBUTO se muestra en la tabla siguiente (Tabla 3.13), donde se han comparado los promedios de las variables seleccionadas para el estudio a través de una prueba de distribución libre (*Wald-Wolfowitz*). En todas las variables los promedios para la propuesta (AGS + MBUTO) son menores y difieren significativamente de los promedios del AGS para $p < 0.001$, lo cual constituye para este caso una regularidad estadística. Con el método AGS + MBUTO se encuentran promedios, valores máximos y mínimos de C_{max} , número de generaciones para encontrar la mejor solución y tiempo en encontrar la misma, menores que cuando se usa solo el AGS.

Tabla 3.13. Eficacia del uso del AGS + MBUTO.

Estadígrafos	Variables									
	Promedios de Cmax		Máximo valor de Cmax		Mínimo valor de Cmax		Generaciones para encontrar mejor solución		Tiempo en encontrar mejor solución	
	AGS	AGS + MBUTO	AGS	AGS + MBUTO	AGS	AGS + MBUTO	AGS	AGS + MBUTO	AGS	AGS + MBUTO
Promedios	1372,175	1212,251	1648,88	1282,53	1315,93	1207	1673,72	20,74	153768,2	9117,810
Z	-14,0360		-14,0360		-14,0360		-14,0360		-13,1854	
p	0,001		0,001		0,001		0,001		0,001	
Z ajustada	13,96514		13,96514		13,96514		13,96514		13,11447	
p	0,001		0,001		0,001		0,001		0,001	

3.5. Aplicación computacional realizada

Finalmente, derivado de la investigación, se realizó una aplicación (software para computadora), la cual ha sido utilizada para mostrar los resultados alcanzados en este trabajo. Para los experimentos y ejecuciones se utilizó un ordenador con 512 MB de RAM y un microprocesador Intel Pentium a 2.66 GH. El código fue escrito en el lenguaje de programación C++ y se compiló para la plataforma Windows.

3.6. Conclusiones del capítulo

En este capítulo se han realizado comprobaciones con problemas seleccionados pertenecientes a la biblioteca de investigación de operaciones (*OR- Library*), con el propósito de comparar el rendimiento de la metaheurística Algoritmo Genético Simple (AGS), y cuando se le añade el método que aquí se propone de Mejora Basada en el Uso del Tiempo Ocioso (AGS + MBUTO), y se ha concluido que:

1. La aplicación del método (AGS + MBUTO), permitió obtener mejores resultados en el proceso de optimización para el objetivo del camino máximo, al obtenerse:
 - menores valores promedios de C_{max} .
 - menores valores máximos de C_{max} .
 - menores valores mínimos de C_{max} .
 - menor número de generaciones para encontrar la mejor solución.
2. Con los parámetros seleccionados para el algoritmo genético, nunca se obtuvo el valor óptimo de C_{max} para la instancia de problema objeto del experimento, sin embargo, siempre se consiguió cuando se aplicó el método propuesto.
3. El tiempo empleado para encontrar la mejor solución para este problema (valor óptimo de C_{max}) fue mayor para el algoritmo genético en comparación al tiempo empleado al aplicarse el método propuesto.

CONCLUSIONES GENERALES

La realización de esta investigación permitió obtener las siguientes conclusiones:

1. La representación del problema mediante un grafo disyuntivo dirigido demostró ser una adecuada estructura de datos para tratar problemas de *scheduling*.
2. La forma de cálculo planteada para acotar la dimensión necesaria y suficiente para cada variante de problema, permite obtener todas las soluciones factibles, donde es posible encontrar los valores óptimos globales para los diferentes objetivos.
3. Se creó un modelo matemático para determinar el consumo de energía de las máquinas que intervienen en la ejecución de un plan de trabajo.
4. La utilización de una metaheurística basada en un algoritmo genético simple (AGS), como propuesta de solución, ofrece buenos resultados al problema planteado, en un tiempo de cómputo razonable.
5. El método desarrollado para el aprovechamiento del tiempo ocioso en las máquinas (MBUTO), permitió mejorar las soluciones obtenidas por el algoritmo genético.
6. Se obtuvo un modelo para la optimización capaz de calcular varios objetivos objetivos (8 en total) agrupados en tres categorías: basados en los instantes de finalización (3 objetivos), en las fechas de compromisos (4 objetivos) y uno basado en el consumo de energía.
7. Se demostró, a través de la comparación con otros problemas clásicos, la calidad de las soluciones obtenidas para el objetivo del camino máximo.
8. Mediante análisis estadístico se comprobó el rendimiento de la metaheurística con la adición del método que se propone de mejora basada en el uso del tiempo ocioso.

RECOMENDACIONES

1. Incluir en la modelación del problema otros elementos tales como: el tiempo de configuración y cambio de herramientas, la existencia de máquinas en paralelo, la recirculación de los trabajos, el posible conocimiento incompleto de los tiempos de procesamiento (incertidumbre), entre otros, que le otorgarían a la modelación del problema un mayor acercamiento a la realidad de un taller de mecanizado.
2. Utilizar otra metaheurística como solución para este problema conjuntamente con el método propuesto en esta investigación para comparar los resultados que se obtienen.
3. Adicionar el costo de fabricación de las piezas como otro objetivo a medir en un plan de trabajo.

BIBLIOGRAFÍA

- Aarts, E. H. L., P. J. M. v. Laarhoven y J. K. Lenstra (1994). A computational study of local search algorithms for job shop scheduling. *ORSA Journal on Computing*. 6 (2). pp. 118-125.
- Adams, J., E. Balas y D. Zawack (1988). The Shifting Bottleneck Procedure for Job Shop Scheduling. *Management Science*. 34 (3). pp. 391-401.
- Adibi, M. A., M. Zandieh y M. Amiri (2010). Multi-objective scheduling of dynamic job shop using variable neighborhood search. *Expert Systems with Applications*. 37 (2010). pp. 282-287.
- Akers, S. B. y J. Friedman (1955). A non-numerical approach to production scheduling problems. *Operations Research*. 3 pp. 429 - 442.
- Akyol, D. E. (2004). Application of neural networks to heuristic scheduling algorithms. *Computers & Industrial Engineering*. 46 pp. 679-696.
- Allahverdi, A. y F. Al-Anzi (2006). A branch and bound algorithm for three-machine flowshop scheduling problem to minimize total completion time with separate setup times. *European Journal of Operational Research*. 169 pp. 767-780.
- Applegate, D. y W. Cook (1991). A computational study of the job-shop scheduling problem. *ORSA Journal of computing*. 3 (2). pp. 149-156.
- Artigues, C. y D. Feillet (2008). A branch and bound method for the job-shop problem with sequence dependent of setup times. *Annals of Operations Research*. 159 (1). pp. 135-159.
- Artigues, C., M. Gendreau, L.-M. Rousseau y A. Vergnaud (2009). Solving an integrated employee timetabling and job-shop scheduling problem via hybrid branch-and-bound. *Computers & Operations Research*. 36 (2009). pp. 2330-2340.
- Balas, E. (1969). Machine sequencing via disjunctive graphs: An implicit enumeration algorithm. *Operations Research*. 17 pp. 1-10.
- Banzhaf, W., P. Nordin, R. Keller y F. D. Francone (1998). *Genetic Programming. An Introduction*, Morgan Kaufmann Publisher. 512 p.

- Bean, J. (1994). Genetics and Random Keys for Sequencing and Optimization. *ORSA Journal on Computing*. 6 pp. 154-160.
- Beasley, J. E. (1990). Or-library. <http://people.brunel.ac.uk/mastijb/jeb/info.html>.
- Beasley, J. E. (1990). Or-library: Distributing Test Problems by Electronic Mail. *Journal of the Operational Research Society*. 41 (11). pp. 1069-1072.
- Beasley, J. E. (1993). Lagrangian relaxation. *Modern heuristic techniques for combinatorial problems*. C. R. Reeves. Oxford, Blackwell Scientific Publications. pp. 243-303.
- Bierwirth, C. (1995). A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms. *OR Spektrum*. 17 pp. 87-92.
- Bierwirth, C. y D. C. Mattfeld (1999). Production Scheduling and Rescheduling with Genetic Algorithms. *Evolutionary Computation*. 7 (1). pp. 1-17.
- Blazewicz, J., K. H. Ecker, E. Pesch, G. Schmidt y J. Weglarz (2007). *Handbook on Scheduling from Theory to applications*. Berlin, Springer-Verlag Berlin Heidelberg. 647 p.
- Brito, R. A., G. Mejía Delgadillo y J. P. Caballero Villalobos (2006). Combined approach of the Shifting Bottleneck and Tabu Search Heuristics for Minimizing Total Weighted Tardiness in Job Shop Scheduling Problems. *Third International Conference on Production Research - Americas' Region 2006 (ICPR-AM06)*.
- Brucker, P. (2007). *Scheduling Algorithms*. Berlin, Springer-Verlag. 371 p.
- Brucker, P., B. Jurisch y B. Sievers (1994). A Branch and bound algorithm for the job-shop scheduling problem. *Discrete Applied Mathematics*. 49 (1994). pp. 107-127.
- Carlier, J. y E. Pinson (1989). An Algorithm for solving the job-shop problem. *Management Science*. 35 (2). pp. 164-176.
- Conway, R. W., W. L. Maxwell y L. W. Miller (1967). *Theory of Scheduling*. Reading, Massachusetts, Addison-Wesley Pub. Co.,. 294 p.
- Chen, R. y M. Gen (1996). A tutorial survey of job-shop scheduling problems using genetic algorithms. *Representation Computers ind. Engng*. 30 (4). pp. 983-997.
- Davis, L. (1987). *Genetic Algorithm and Simulated Annealing*. London, Pitman Publishing. 216 p.
- Davis, L. (1990). *Handbook of Genetic Algorithms*. New York, Van Nostrand Reinhold. 385 p.
- Dell'Amico, M. y M. Trubian (1993). Applying tabu search to the job shop scheduling problem. *Annals of Operations Research*. 41 pp. 231-252.

- Desrochers, A. A. y R. Y. Al-Jaar (1995). Applications of Petri Nets in Manufacturing Systems: Modeling, Control and Performance Analysis. Piscataway, NJ, IEEE Press. 384 p.
- Dorigo, M. y T. Stützle (2003). The ant colony optimization metaheuristic: Algorithms applications, and advancesp.
- Dorndorf, U., E. Pesch y T. Phan-Huy (2002). Constraint propagation and problem descomposition: A preproceessing procedure for the job shop problem. Annals of Operations Research. 115 (125-142).
- Fang, H., P. Ross y D. Corne (1993). A Promising Genetic Algorithm Approach to Job-Shop Scheduling, Rescheduling and Open-Shop Scheduling Problems. Fifth International Conference on Genetic Algorithms, San Mateo, Morgan Kaufmann. pp. 375-382.
- Farber, G., S. Salhi y A. M. C. Moreno (2007). Sequencing in a non-permutation flowshop with constrained buffers: Applicability of genetic algorithms versus constraint logic programming. XI Congreso de Ingeniería de Organización. International Conference on Industrial Engineering and Industrial Management - CIO 2007, Madrid, España. pp. 69-78.
- Figlali, N., C. Özkale, O. Engin y A. Figlali (2009). Investigation of Ant System parameter interactions by using design of experiments for job-shop scheduling problems. Computers & Industrial Engineering. 56 (2009). pp. 538-559.
- Fisher, H. y G. L. Thompson (1963). Probabilistic learning combinations of local job-shop scheduling rules. Industrial Scheduling, Englewood Cliffs, New Jersey, Prentice Hall. pp. 225-251.
- Gao, J., M. Gen, L. Sun y X. Zhao (2007). A hybrid of genetic algorithm and bottleneck shifting for multiobjective flexible job shop scheduling problems. Computers & Industrial Engineering. 53 (2007). pp. 149-162.
- Gao, J., L. Sun y M. Gen (2008). A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. Computers & Operations Research. 35 (2008). pp. 2892-2907.
- Garey, M. R., D. S. Johnson y R. Sethi (1976). The Complexity of Flowshop and Jobshop Scheduling. Mathematics of Operations Research. 1 pp. 117-129.
- Giffler, B. y G. Thompson (1960). Algorithm for solving production scheduling problems. Operations Research. 8 pp. 487-503.
- Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. Computers & Operations Research. 5 pp. 533-549.
- Goldberg, D. E. (1989). Genetic algorithms in search, optimization and machine learning. Massachusetts, USA, Addison-Wesley Longman Publishing Co., Inc., Boston. 432 p.

- Graham, R. L., E. L. Lawler, J. K. Lenstra y A. H. G. R. Kan (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*. 5 pp. 287-326.
- Guinard, M. (2002). Lagrangian relaxation. *Handbook of Applied Optimization* Oxford University Press. P. M. Pardalos and M. G. C. Resende. Oxford, Oxford University Press. pp. 465-474.
- Hentenryck, H. y M. Simonis (1992). Constraint satisfaction using constraint logic programming. *Artificial Intelligence*. 58 pp. 113-159.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems: an introductory analysis with applications to biology, control, and artificial intelligence*. Michigan, USA, University of Michigan Press, Ann Arbor. 183 p.
- Howard, L. M. y D. J. D'Angelo (1995). The GA-P: A Genetic Algorithm and Genetic Programming Hybrid. *IEEE Expert*. pp. 11-15.
- Jackson, J. R. (1955). Scheduling a production line to minimize maximum tardiness. Research Report 43, Management Science Research Project.
- Jackson, J. R. (1956). An Extension of Johnson Result on Job Lot Scheduling. *Naval Research Logistic Quarterly*. 3 (3). pp. 201-203.
- Jain, A. S. y S. Meeran (1998). Deterministic job shop scheduling: Past, present and future. *European Journal of Operational Research*. 113 pp. 390-434.
- Johnson, S. M. (1954). Optimal two and three stage production schedules with setup includes. *Naval Research Logistic Quarterly*. 1 pp. 61-68.
- Kacem, I., S. Hammadi y P. Borne (2002). Pareto-optimality approach for flexible job-shop scheduling problem: hybridization of evolutionary algorithms and fuzzy logic. *Mathematical and Computer in Simulation*. 60 (2002). pp. 245-276.
- Kirkpatrick, S. (1983). Optimization by Simulated Annealing. *Science*. 220 (4598). pp. 671-680.
- Kolonko, M. (1999). Some new results on simulated annealing applied to the job shop scheduling problem. *European Journal of Operational Research*. 111 pp. 509-517.
- Kopfer, H., D. C. Mattfeld y C. Bierwirth (1994). Control of parallel population dynamics by social-like behavior of ga-individuals. *Parallel Problem Solving from Nature*. 3.
- Laarhoven, P. J. M. V., E. H. L. Aarts y J. K. Lenstra (1992). Job shop scheduling by simulated annealing. *Operations Research*. 40 (1). pp. 113-125.

- Laguna, M., F. Glover y R. Martí (2003). Scatter search and path relinking: Advances and applications. Handbook of Metaheuristics. F. Glover and G. Kochenberger. Boston, Kluwer Academic Publishers. pp. 1-32.
- Larrañaga, P. y J. A. Lozano (2002). Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation, Kluwer Academic. 416 p.
- Larrañaga, P., J. A. Lozano y H. Mühlenbein (2003). Algoritmos de estimación de distribuciones en problemas de optimización combinatoria. Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial. 19 pp. 149-168.
- Lawler, E. L., J. K. Lenstra y A. H. G. R. Kan (1993). Sequencing and scheduling: algorithms and complexity. Handbooks in Operations Research and Management Science. S. C. Graves, A. H. G. R. Kan and P. H. Zipkin, North Holland. 4. pp. 445-522.
- Lawler, E. L., J. K. Lenstra, A. H. G. R. Kan y D. B. Shmoys (1982). Recent developments in deterministic sequencing and scheduling: A survey. In Lenstra Dempster & Kan Rinnooy (Eds.), Deterministic and Stochastic Scheduling, Dordrecht, Netherlands: D. Reidel Publishing Company. pp. 35-74.
- Lin, S., E. Goodman y W. Punch (1997). A genetic algorithm approach to dynamic job shop scheduling problems. Proceedings of the Seventh International Conference on Genetic Algorithms, San Mateo, California, Morgan Kaufmann. pp. 481-489.
- Looi, C. K. (1992). Neural network method in combinatorial optimization. Computers & Operations Research. 19 pp. 191-208.
- MacCarthy, B. L. y J. Liu (1993). Addressing the gap in scheduling research: a review of optimization and heuristic methods in production scheduling. International Journal of Production Research. 31 (1). pp. 59-79.
- Márquez, J. E., R. L. Ávila, M. A. Gómez-Elvira y C. R. Herrera (2012). Algoritmo Genético aplicado al problema de programación en procesos tecnológicos de maquinado con ambiente Flow Shop. Revista Ciencias Técnicas Agropecuarias. 21 (2). pp. 70-75.
- Martí, R. y M. Laguna (2003). Scatter search: Diseño básico y estrategias avanzadas. Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial. 19 pp. 46-60.
- Matsuo, H., C. J. Suh y R. S. Sullivan (1988). A controlled search simulated annealing method for the general jobshop scheduling problem. Department of Management, The University of Texas at Austin, Vol. Working Paper, 03-04-88.
- Michalewicz, Z. (1996). Genetic Algorithms + DataStructures = Evolution Programs, Springer-Verlag. 408 p.

- Montoya Torres, J. R., C. D. Paternina Arboleda y F. Yannick (2002). Minimización del tiempo total de flujo de tareas en una sola máquina: estado del arte. *Ingeniería & Desarrollo*. 12 pp. 118-129.
- Murata, T. (1989). Petri Nets: Properties, Analysis and Applications. *Proceedings of the IEEE*. 77 (4). pp. 541-580.
- Muth, J. F. (1963). *Industrial Scheduling*. N.J., Prentice-Hall. 387 p.
- Nakano, R. y T. Yamada (1991). Conventional Genetic Algorithms for Job-Shop Problems. 4th International Conference on Genetic Algorithms (ICGA-91), San Mateo, CA, Morgan Kaufman Publishers. pp. 447-479.
- Nowicki, E. y C. Smutnicki (1996). A fast tabu search algorithm for the job-shop scheduling problem. *Management Science*. 42 pp. 797-813.
- Papadimitriou, C. H. (1994). *Computational complexity*. Reading, Mass., Addison-Wesley.
- Pezzella, F. y E. Merelli (2000). A tabu search method guided by shifting bottleneck for the job shop scheduling problem. *European Journal of Operational Research*. 120 (2000). pp. 297-310.
- Pinedo, M. L. (2005). *Planning and Scheduling in Manufacturing and Services*. New York, Springer. 506 p.
- Pinedo, M. L. (2008). *Scheduling: Theory, Algorithms and Systems*. New York, Springer.
- Ponnambalam, S. G., N. Jawahar y P. Aravindan (1999). A simulated annealing algorithm for Job Shop Scheduling. *Production Planning and Control*. 10 (767-777).
- Rajendran, C. y H. Zeigler (2001). A performance Analysis of Dispatching rules and heuristic in static flowsops with missing operations of jobs. *European Journal of Operational Research*. 131 pp. 622-634.
- Reeves, C. (1993). *Modern Heuristic Techniques for Combinatorial Problems*. New York, NY, USA, John Wiley & Sons, Inc. 320 p.
- Resende, M. y J. L. González-Velarde (2003). Grasp: Procedimientos de búsqueda miopes aleatorizados y adaptativos. *Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial*. 19 pp. 61-76.
- Resende, M. G. C. y C. C. Ribeiro (2003). Greedy randomized adaptive search procedures. *Handbook of Metaheuristics*. F. Glover and G. G. Kochenberger, Kluwer Academic Publishers. pp. 219-249.
- Rondón, R. L. A., A. d. S. Carvalho y G. I. Hernández (2008). Neural network modeling and simulation of the scheduling. *Innovation in Manufacturing Networks*. 266 pp. 231-238.

- Roshanaei, V., B. Naderi, F. Jolai y M. Khalili (2009). A variable neighborhood search for job shop scheduling with set-up times to minimize makespan. *Future Generation Computer Systems*. 25 (2009). pp. 654-661.
- Roy, B. y B. Sussman (1964). *Les Problemes d'Ordonnancement avec Contraintes Disjonctives*. Rapport technique, Technical Report Note DS No.9 bis. SEMA, Montrouge, Paris.
- Russell, S. J. y P. Norvig (1995). *Artificial Intelligence a moder approach*. Englewood Cliffs New Jersey 07632, Prentice Hall. 932 p.
- Sadeh, N. y M. S. Fox (1996). Variable and Value ordering heuristics for the job shop scheduling constraint satisfaction problem. *Artificial Intelligence*. 86 pp. 1-41.
- Sadeh, N. y Y. Nakakuki (1996). Focused Simulated Annealing Search - An Application to Job-Shop Scheduling. *Annals of Operations Research*. 63 pp. 77-103.
- Schutten, J. M. J. (1998). Practical job shop scheduling. *Annals of Operations Research*. 83 pp. 161-178.
- Storer, R. y F. Talbot (1992). New search spaces for sequencing problems with application to job shop scheduling. *Management Science*. 38 pp. 1494-1509.
- Syswerda, G. (1991). *Scheduling Optimization using genetic algorithms*, New York, Van Nostrand Reinhold. pp. 332-349.
- Tsai, C.-F. y F. C. Lin (2003). A New Hybrid Heuristic Technique for Solving Job-Shop Scheduling Problem. *IEEE International Workshop on Intelligent Data Acquisition and Advance Computing Systems: Technology and Applications*, Lviv, Ukraine.
- Vaessens, R. J. M., E. H. L. Aarts y J. K. Lenstra (1994). Job shop scheduling by local search. T. Report, Eindhoven University of Technology, Dpto. of Math. and CS.
- Vignier, A., J. C. Billaut y C. Proust (1999). Scheduling problems of hybrid flowshop type: state of art. *RAIRO Operations Research*. 33 (2). pp. 117-183.
- Vilcot, G. y J.-C. Billaut (2008). A Tabu search and genetic algorithm for solving a bicriteria general job shop scheduling problem. *European Journal of Operational Research*. 190 (2008). pp. 398-411.
- Wang, S. y J. Yu (2010). An effective heuristic for the flexible job shop scheduling problem with maintenance activities. *Computers & Industrial Engineering*. 59 (2010). pp. 436-447.

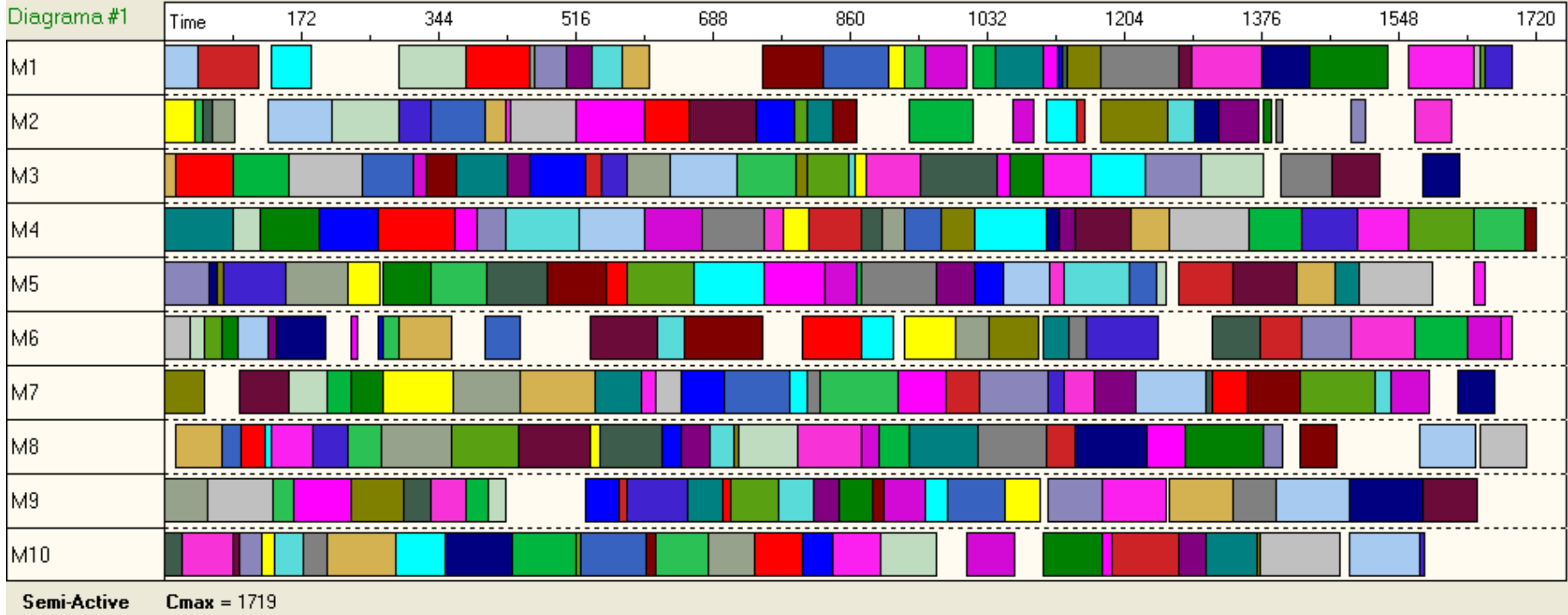
- Wenqi, H. y Y. Aihua (2004). An improved shifting bottleneck procedure for the job shop scheduling problem. *Computers & Operations Research*. 31 (2004). pp. 2093-2110.
- Xing, L.-N., Y.-W. Chen, P. Wang, Q.-S. Zhao y J. Xiong (2010). A Knowledge-Based Ant Colony Optimization for Flexible Job Shop Scheduling Problems. *Applied Soft Computing*. 10 (2010). pp. 888-896.
- Yamada, T. (2003). Studies on Metaheuristics for Jobshop and Flowshop Scheduling Problems. vol. Doctor of Informatics Kyoto, Kyoto University. 120 p.
- Yamada, T. y R. Nakano (1995). A Genetic Algorithm with Multi-Step Crossover for Job-Shop Scheduling Problems. First IEE/IEEE International Conference on Genetic Algorithms in Engineering Systems (GALESIA-95), Sheffield, UK, Innovations and Applications. pp. 146-151.
- Yamada, T. y R. Nakano (1996). Job-Shop scheduling by Simulated Annealing Combined with Deterministic Local Search. *Meta-heuristics: Theory and applications*. 37 (4). pp. 237-248.
- Yamada, T. y R. Nakano (1997). Genetic Algorithms for Job-Shop Scheduling Problems. *Proceedings of Modern Heuristic for decision support, UNICOM Seminar, London*. pp. 67-81.
- Yamada, T. y C. R. Reeves (1997). Permutation Flowshop Scheduling by Genetic Local Search. 2nd IEE/IEEE International Conference on Genetic Algorithms in Engineering Systems (GALESIA'97). pp. 232-238.
- Yamada, T., B. E. Rosen y R. Nakano (1994). A simulated annealing approach to job shop scheduling using critical block transition operators. *Proceedings of IEEE International Conference on Neural Networks, Orlando, Florida*. pp. 4687-4692.
- Yang, J.-h., L. Sun, H. P. Lee, Y. Qian y Y.-c. Liang (2008). Clonal Selection Based Memetic Algorithm for job shop scheduling problems. *Journal of Bionic Engineering*. 5 (2008). pp. 111-119.
- Yazdani, M., M. Amiri y M. Zandieh (2010). Flexible job-shop scheduling with parallel variable neighborhood search algorithm. *Expert Systems with Applications*. 37 (2010). pp. 678-687.
- Yun, Y. S. (2002). Genetic algorithm with fuzzy logic controller for preemptive and non-preemptive job-shop scheduling problems. *Computers & Industrial Engineering*. 43 (2002). pp. 623-644.
- Zhang, C. Y., P. Li, Y. Rao y Z. Guan (2008). A very fast TS/SA algorithm for the job shop scheduling problem. *Computers & Operations Research*. 35 (2008). pp. 282-294.

- Zhang, R. y C. Wu (2010). A hybrid immune simulated annealing algorithm for the job shop scheduling problem. *Applied Soft Computing*. 10 (2010). pp. 79-89.
- Zweben, M., E. Davis, B. Daun, E. Drascher, M. Deale y M. Eskey (1992). Learning to improve constraint-based scheduling. *Artificial Intelligence*. 58 pp. 271-296.

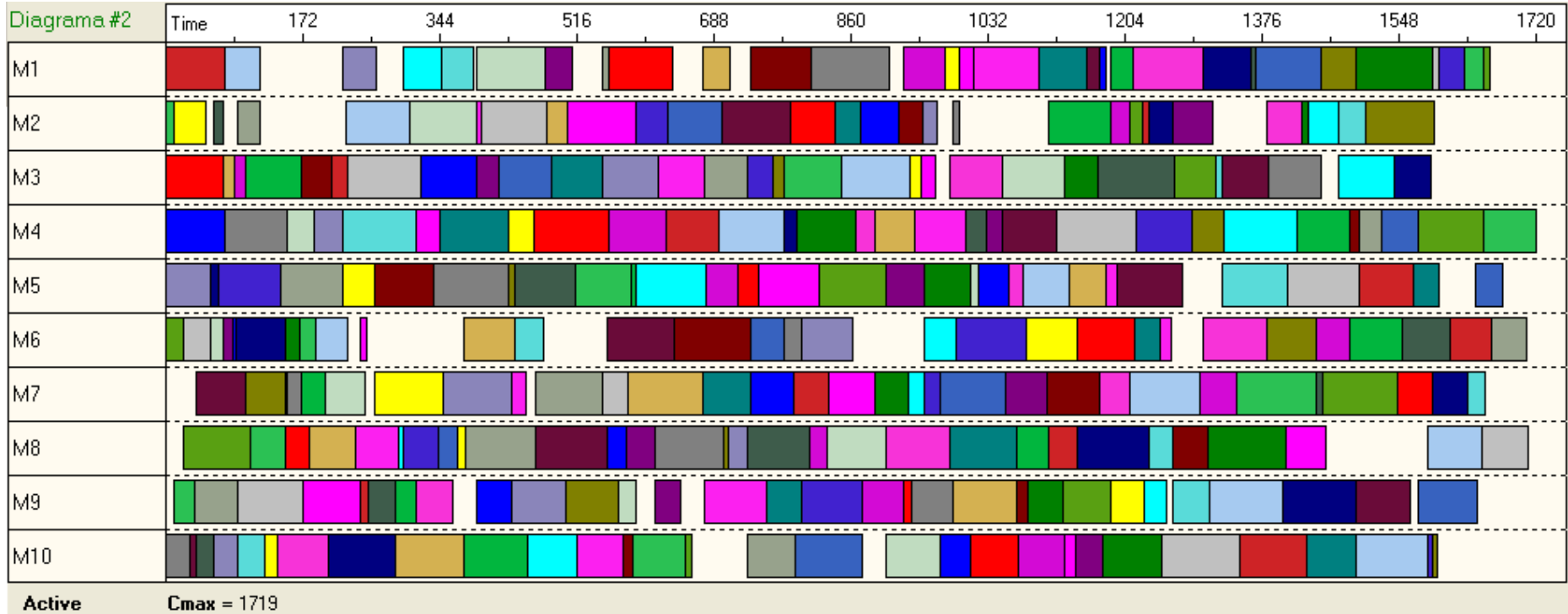
ANEXOS

Anexo 1. Instancia de problema la33 (30 trabajos en 10 máquinas) con solución óptima de camino máximo ($C_{max} = 1719$) en planes de trabajo: semi-activo, activo y sin retardo. Variante *Job Shop Scheduling*.

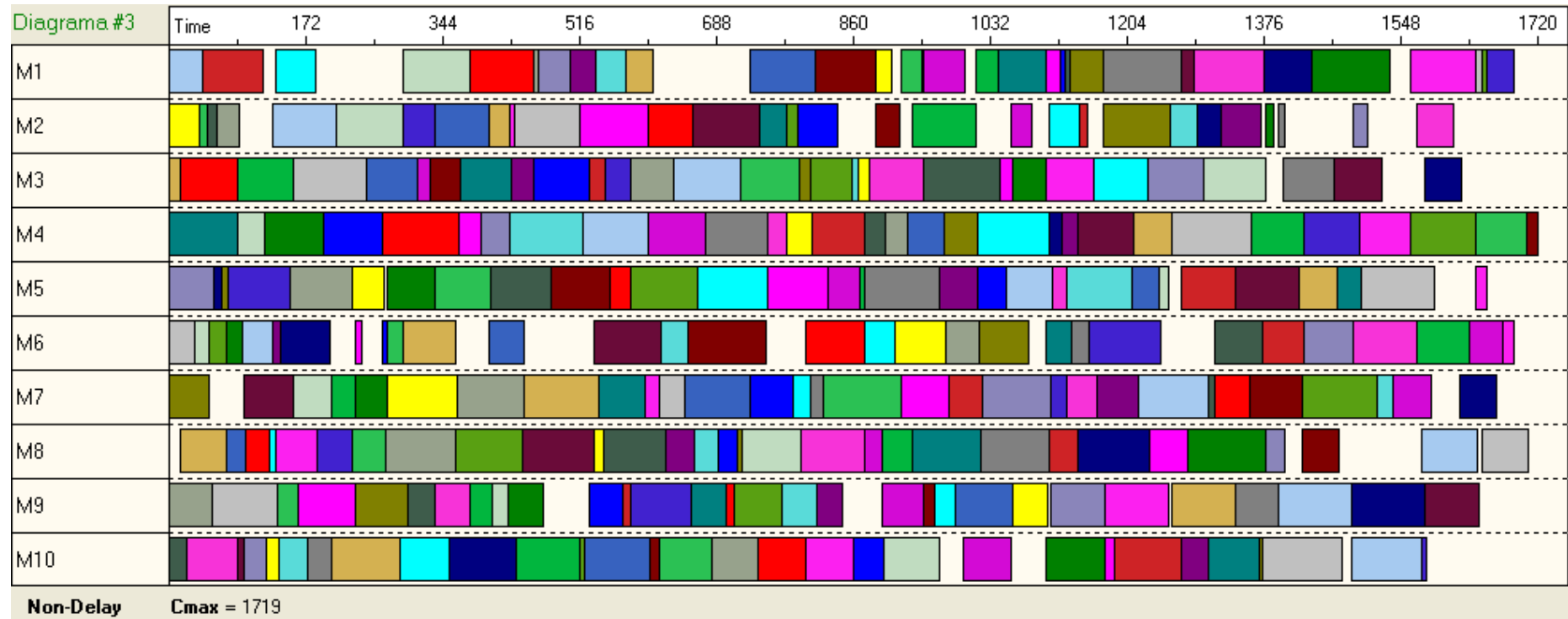
Anexo 1.1. Gráfico de Gantt con plan de trabajo semi-activo (semi-active).



Anexo 1.2. Gráfico de Gantt con plan de trabajo activo (active).

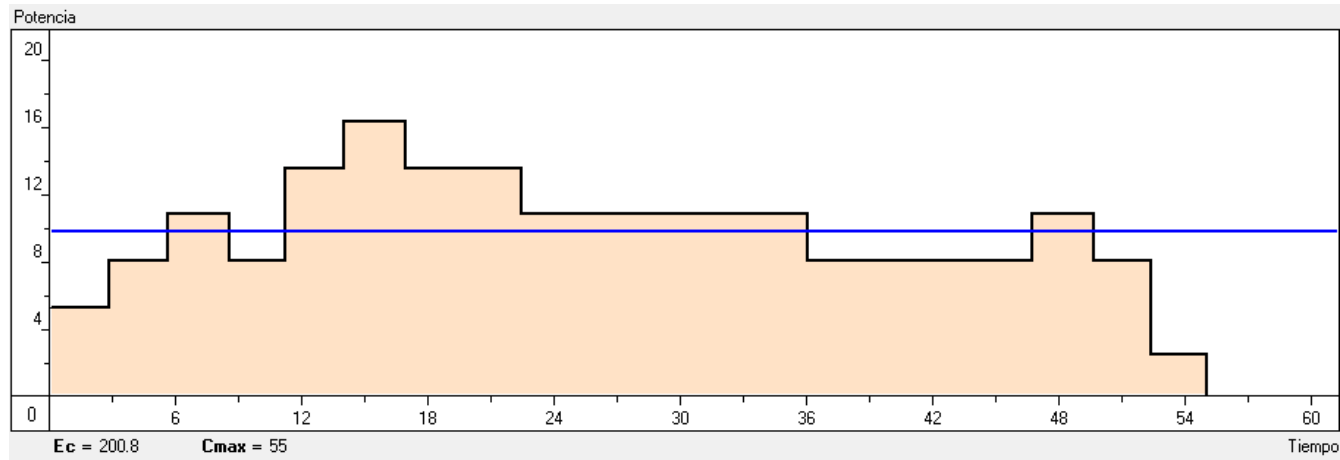
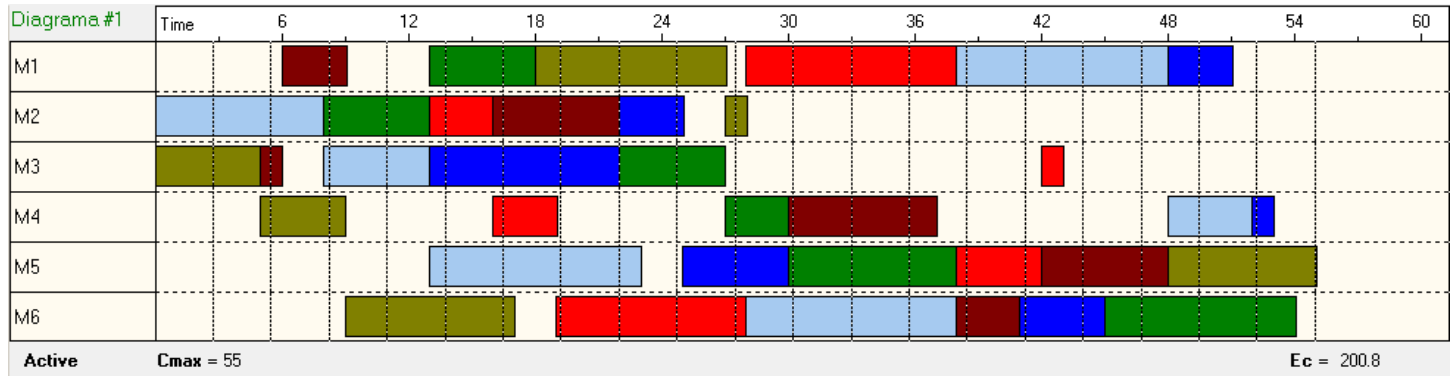


Anexo 1.3. Gráfico de Gantt con plan de trabajo sin retardo (*non-delay*).

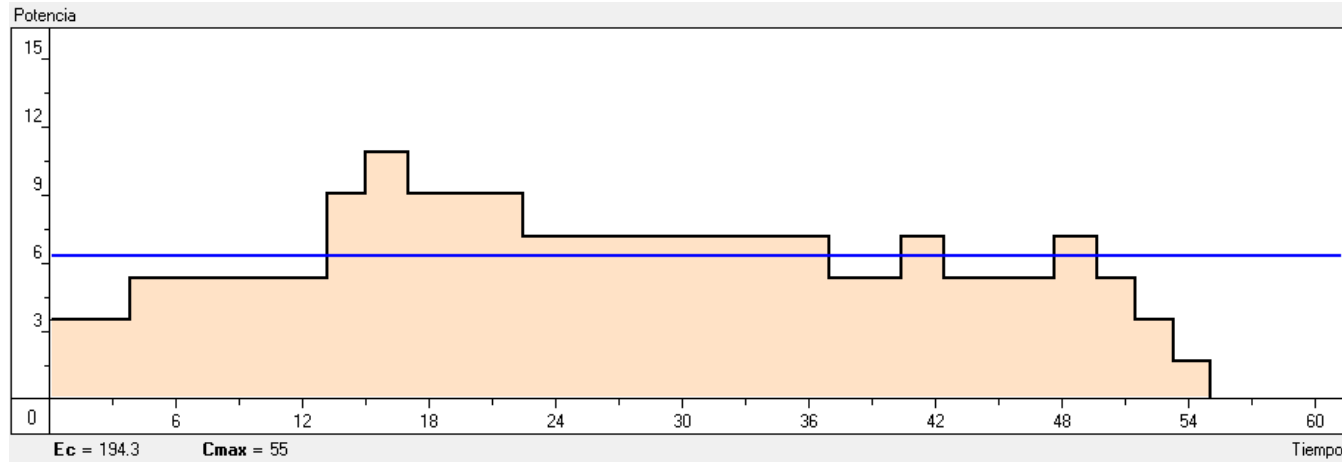
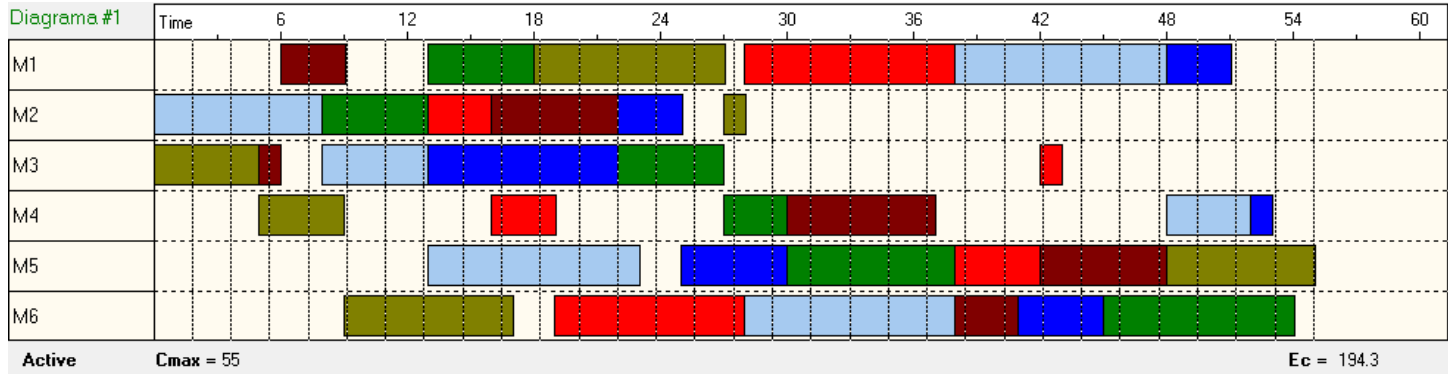


Anexo 2. Gráficos de Gantt y gráficos de carga con valores de consumo de energía en las máquinas.

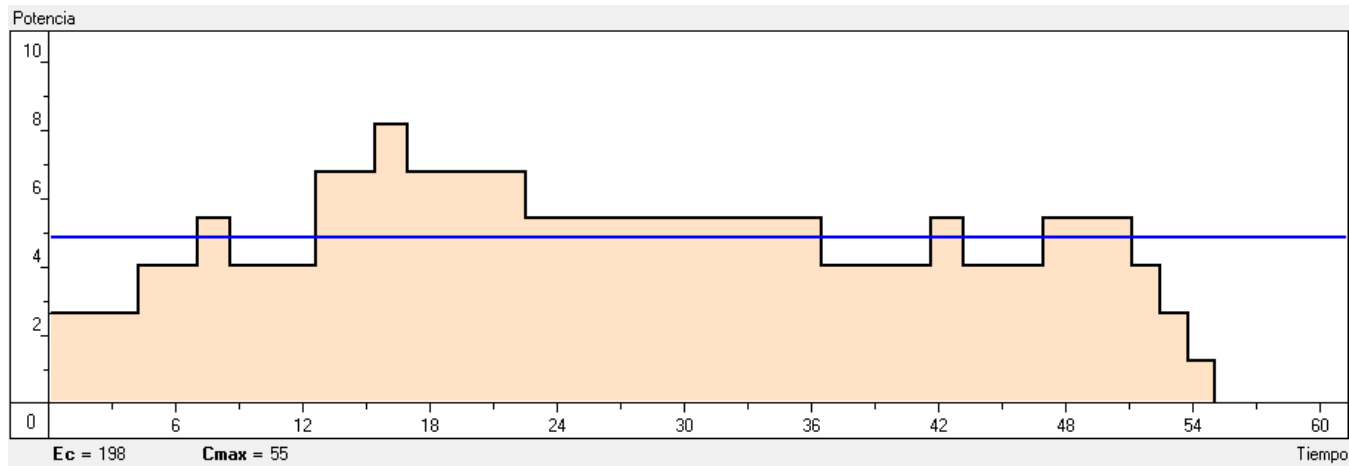
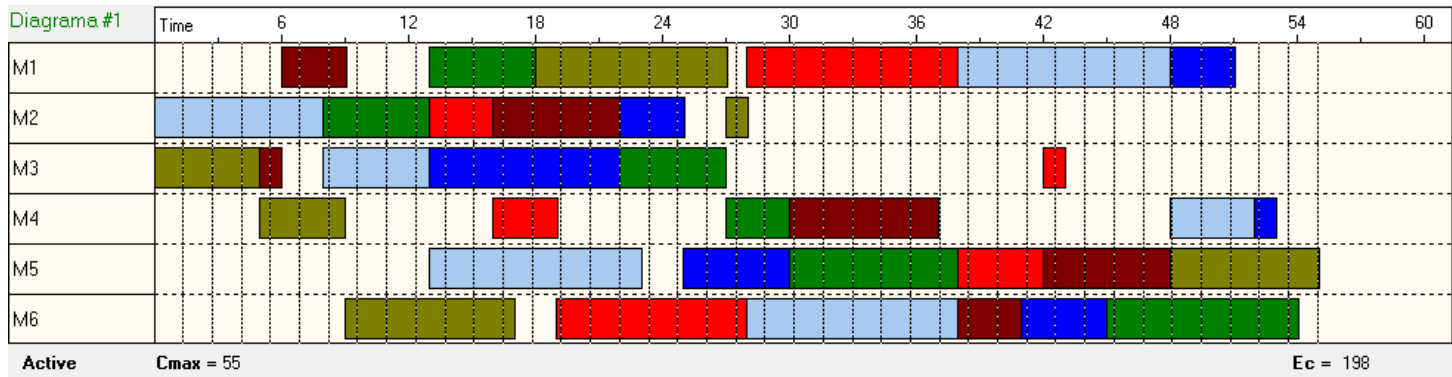
Anexo 2.1. Gráfico de Gantt con 20 mediciones realizadas para calcular el consumo de energía y gráfico de carga correspondiente. Consumo total: 208 unidades.



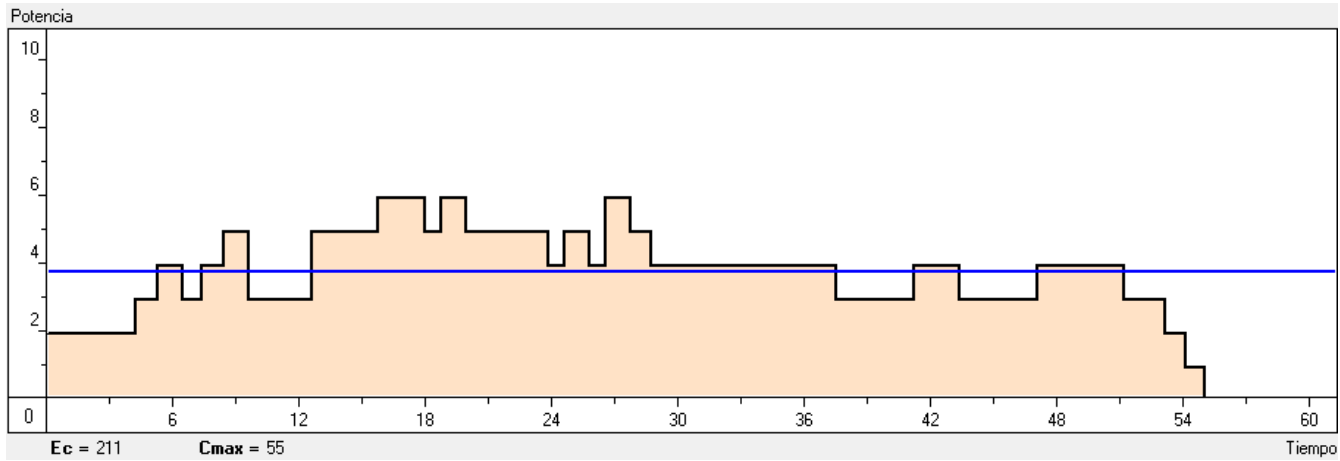
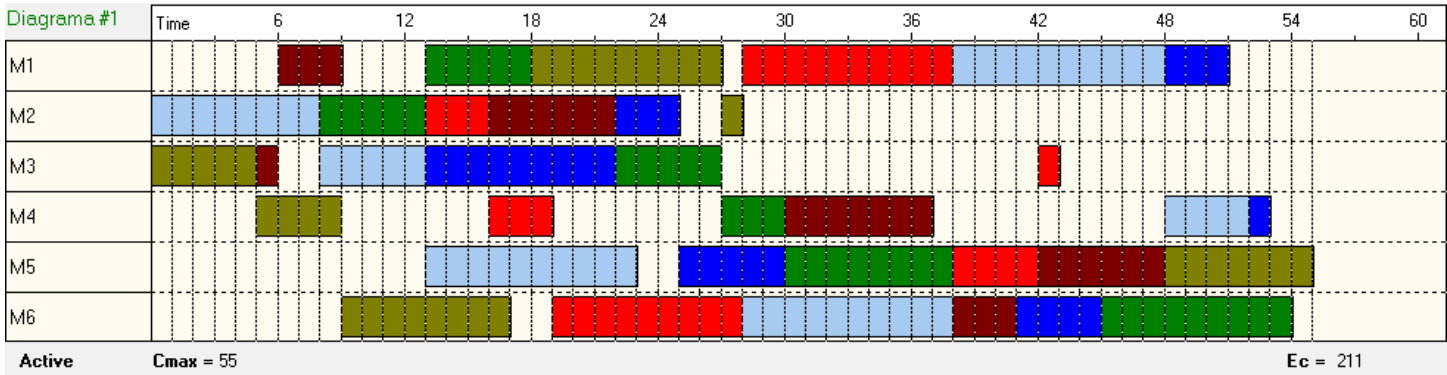
Anexo 2.2. Gráfico de Gantt con 30 mediciones realizadas para calcular el consumo de energía y gráfico de carga correspondiente. Consumo total: 194.3 unidades.



Anexo 2.3. Gráfico de Gantt con 40 mediciones realizadas para calcular el consumo de energía y gráfico de carga correspondiente. Consumo total: 198 unidades.

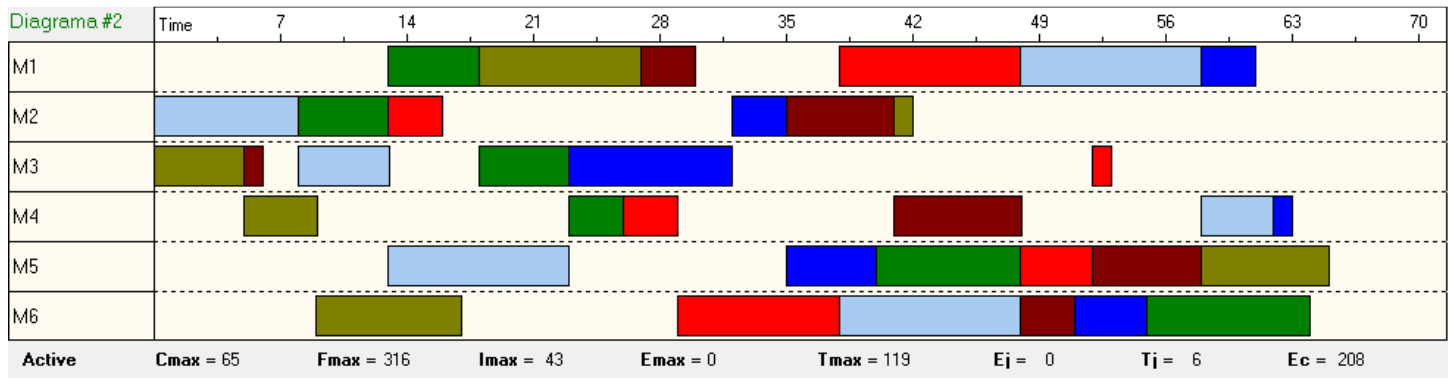
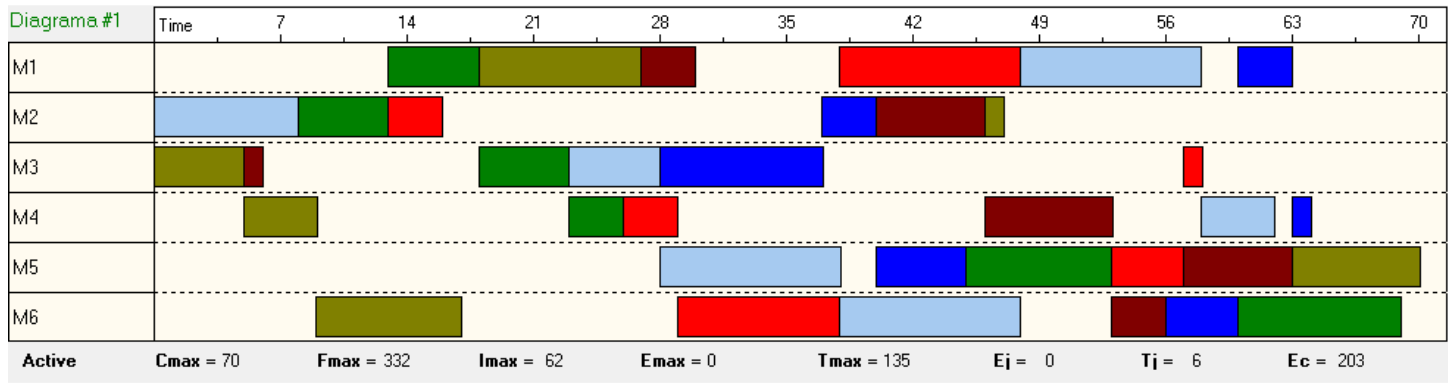


Anexo 2.4. Gráfico de Gantt con 55 mediciones realizadas para calcular el consumo de energía y gráfico de carga correspondiente. Consumo total: 211 unidades.

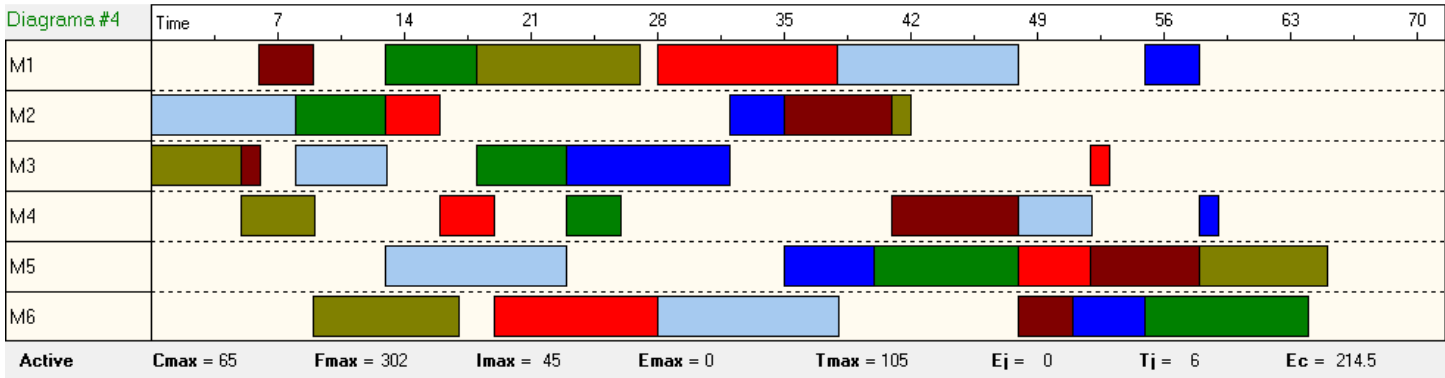
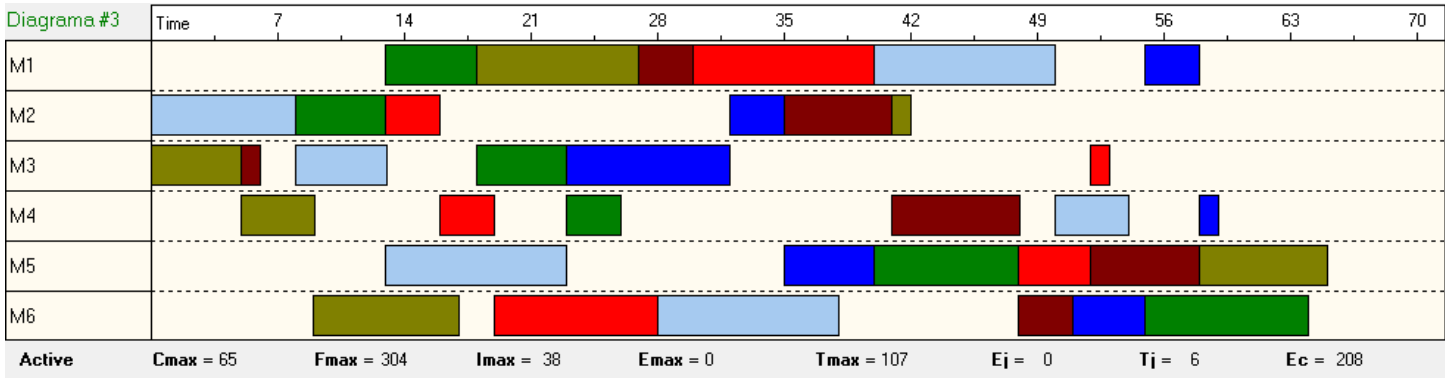


Anexo 3. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas (MBUTO).

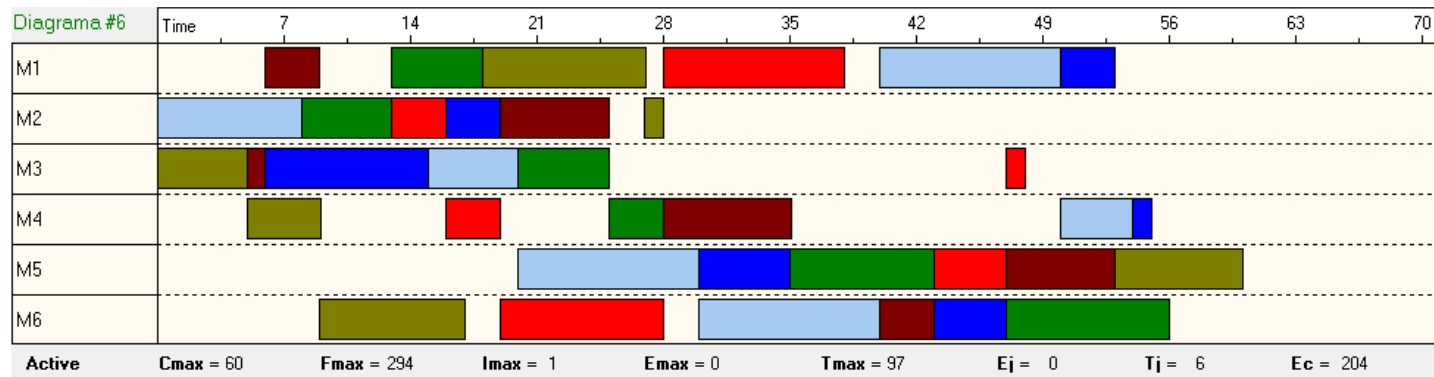
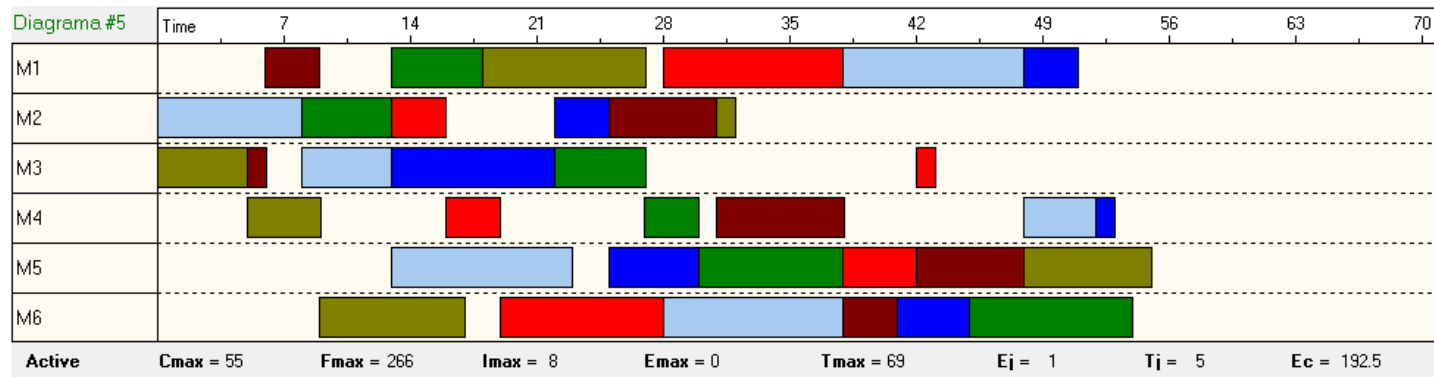
Anexo 3.1. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas.



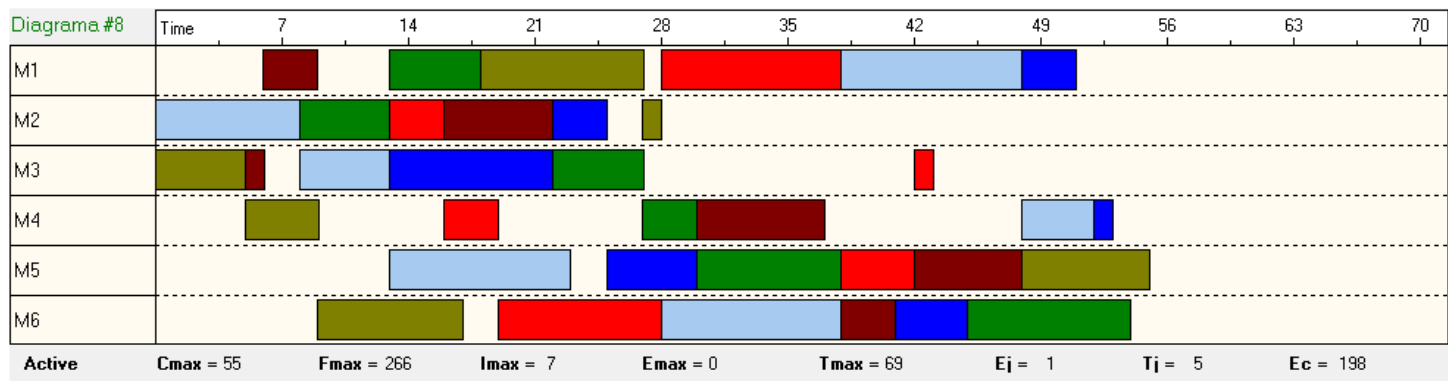
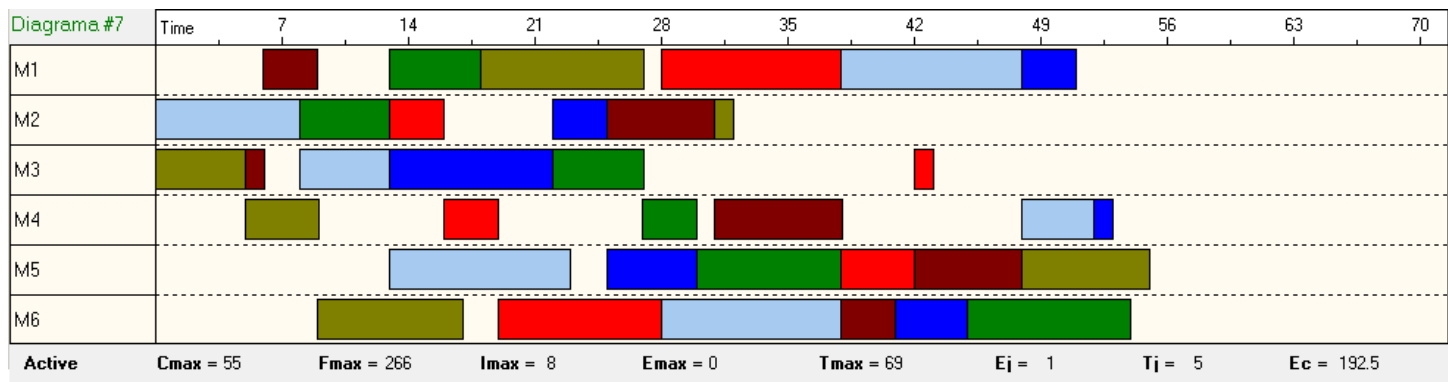
Anexo 3.2. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas. (Continuación).



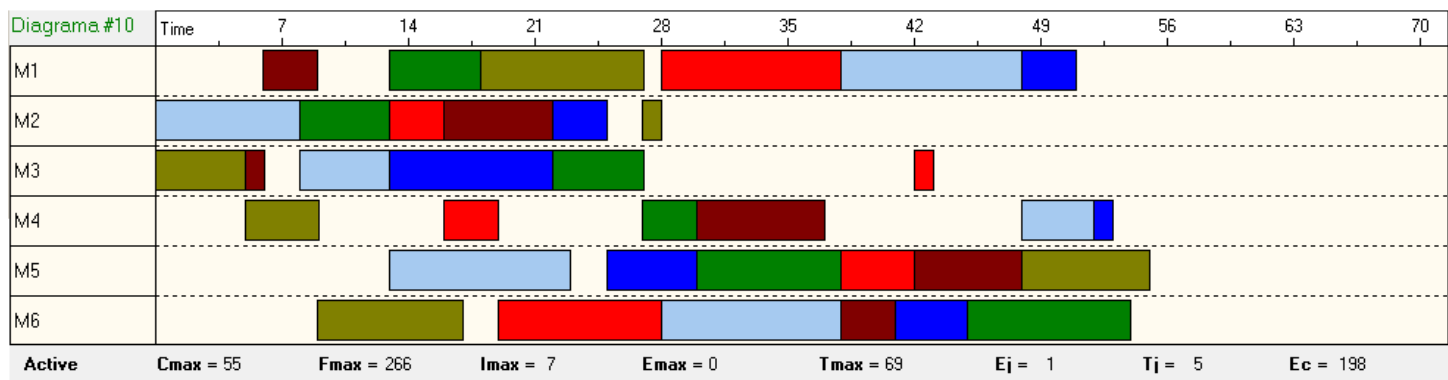
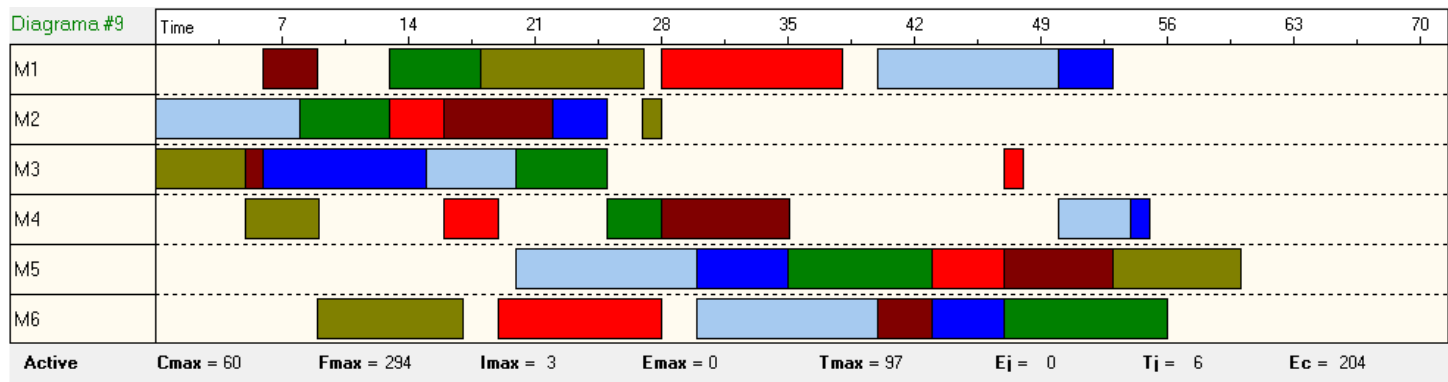
Anexo 3.3. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas.
(Continuación).



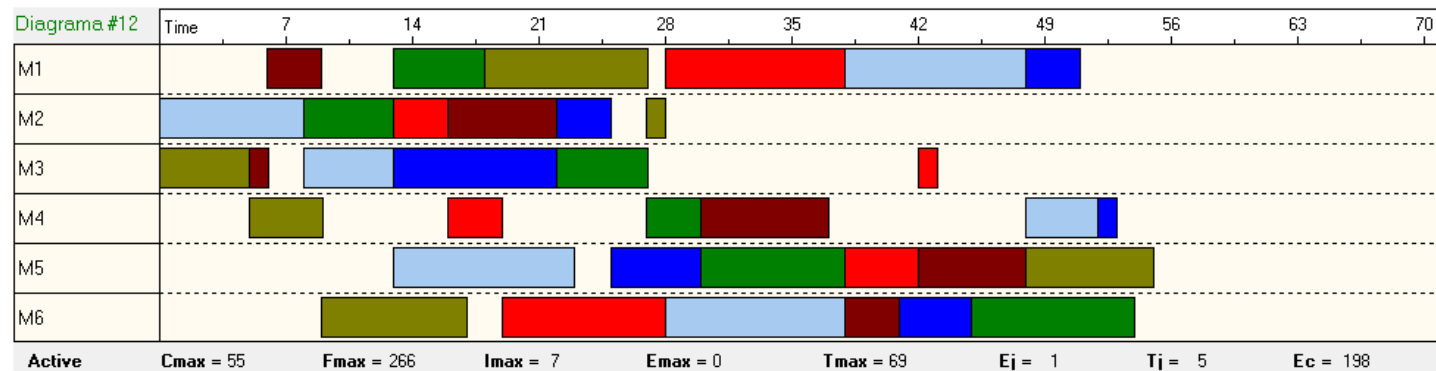
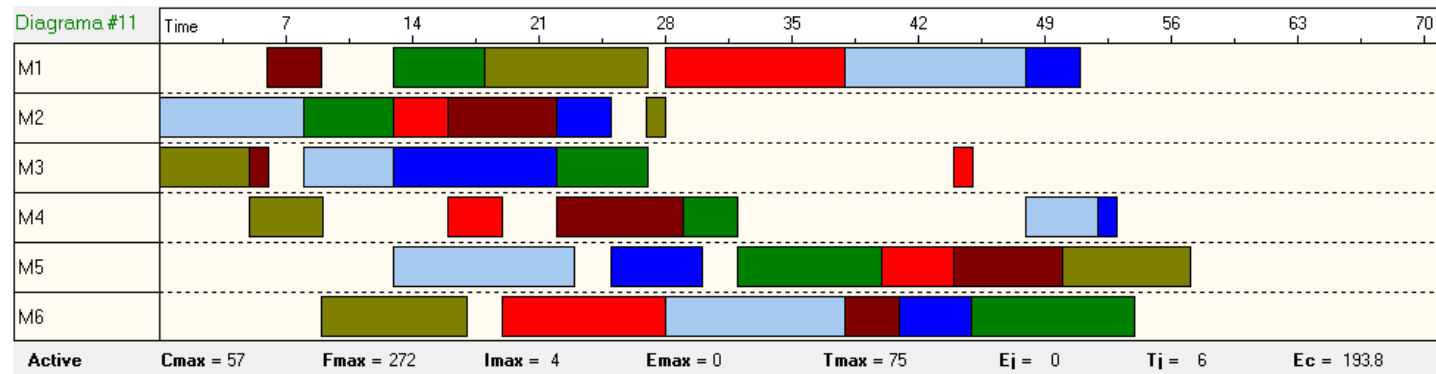
Anexo 3.4. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas. (Continuación).



Anexo 3.5. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas. (Continuación).



Anexo 3.6. Gráficos de Gantt con la transformación de un plan de trabajo por medio del aprovechamiento del tiempo ocioso en las máquinas.
(Continuación).



Anexo 4. Conjunto de instancias de problema de referencia (*benchmark problems*) pertenecientes a la biblioteca de investigación de operaciones (*OR-Library*).

Anexo 4.1 Conjunto de instancias de problema de la variante *flow shop*.

Instancia de problema	N	M	Número de instancias	Propuesto por
car	11	5	1	J. Carlier. Ordonnancements a contraintes disjonctives, R.A.I.R.O. Recherche operationelle. Operations Research 12, 333-351. 1978.
	13	4	1	
	12	5	1	
	14	4	1	
	10	6	1	
	8	9	1	
	7	7	1	
	8	8	1	
hel	100	10	1	J. Heller. Some numerical experiments for an MxJ flow shop and its decision theoretical aspects. Operations Research 8, 178-184. 1960.
	20	10	1	
reC	20	5	3	C.R. Reeves. A genetic algorithm for flowshop sequencing. Computer Ops. Res. 22, 5-13. 1995.
	20	10	3	
	20	15	3	
	30	10	3	
	30	15	3	
	50	10	3	
	75	20	3	
ta	20	5	10	E. D. Taillard. Benchmarks for basic scheduling problems. European Journal of Operational Research, 64, 278-285, 1993.
	20	10	10	
	20	20	10	
	50	5	10	
	50	10	10	
	50	20	10	
	100	5	10	
	100	10	10	
	100	20	10	
	200	10	10	
	200	20	10	
	500	20	10	

Anexo 4.2. Conjunto de instancias de problemas de la variante *job shop*.

Instancia de problema	N	M	Número de instancias	Propuesto por
abz	10	10	2	J. Adams, E. Balas and D. Zawack. The shifting bottleneck procedure for job shop scheduling. Management Science, 34 (3) : 391-401, 1988.
	20	15	3	
ft	6	6	1	H. Fisher and G.L. Thompson. Probabilistic learning combinations of local job shop scheduling rules. Prentice Hall. Englewood Cliffs, New Jersey, 225-251, Industrial
	10 x 10	10	1	
	20	5	1	
la	10	5	5	S. Lawrence. Resource constrained Project scheduling: an experimental investigation of heuristic scheduling techniques (supplement). Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, Pennsylvania, 1984.
	15	5	5	
	20	5	5	
	10	10	5	
	15	10	5	
	20	10	5	
	30	10	5	
orb	15	15	5	D. Applegate and W. Cook. A computational study of the job-shop scheduling instance. ORSA Journal on Computing 3, 149-156, 1991.
	15	15	5	
swv	20	10	5	R. H. Storer, S. D. Wu and R. Vaccari. New search spaces for sequencing instances with application to job shop scheduling. Management Science 38, 1495-1509, 1992.
	20	15	5	
	50	10	5	
	50	10	5	
ta	15	15	10	E. D. Taillard. Benchmarks for basic scheduling problems. European Journal of Operational Research, 64, 278-285, 1993.
	20	15	10	
	20	20	10	
	30	15	10	
	30	20	10	
	50	15	10	
	50	20	10	
yn	100	20	10	T. Yamada and R. Nakano. A genetic algorithm applicable to large-scale job-shop instances. R. Manner, B. Manderick (eds.), Parallel instance solving from nature 2, North-Holland, Amsterdam, 281-290, 1992.
	20	20	4	

Anexo 4.3. Resumen de las instancias de problema. Variante *flow shop*.

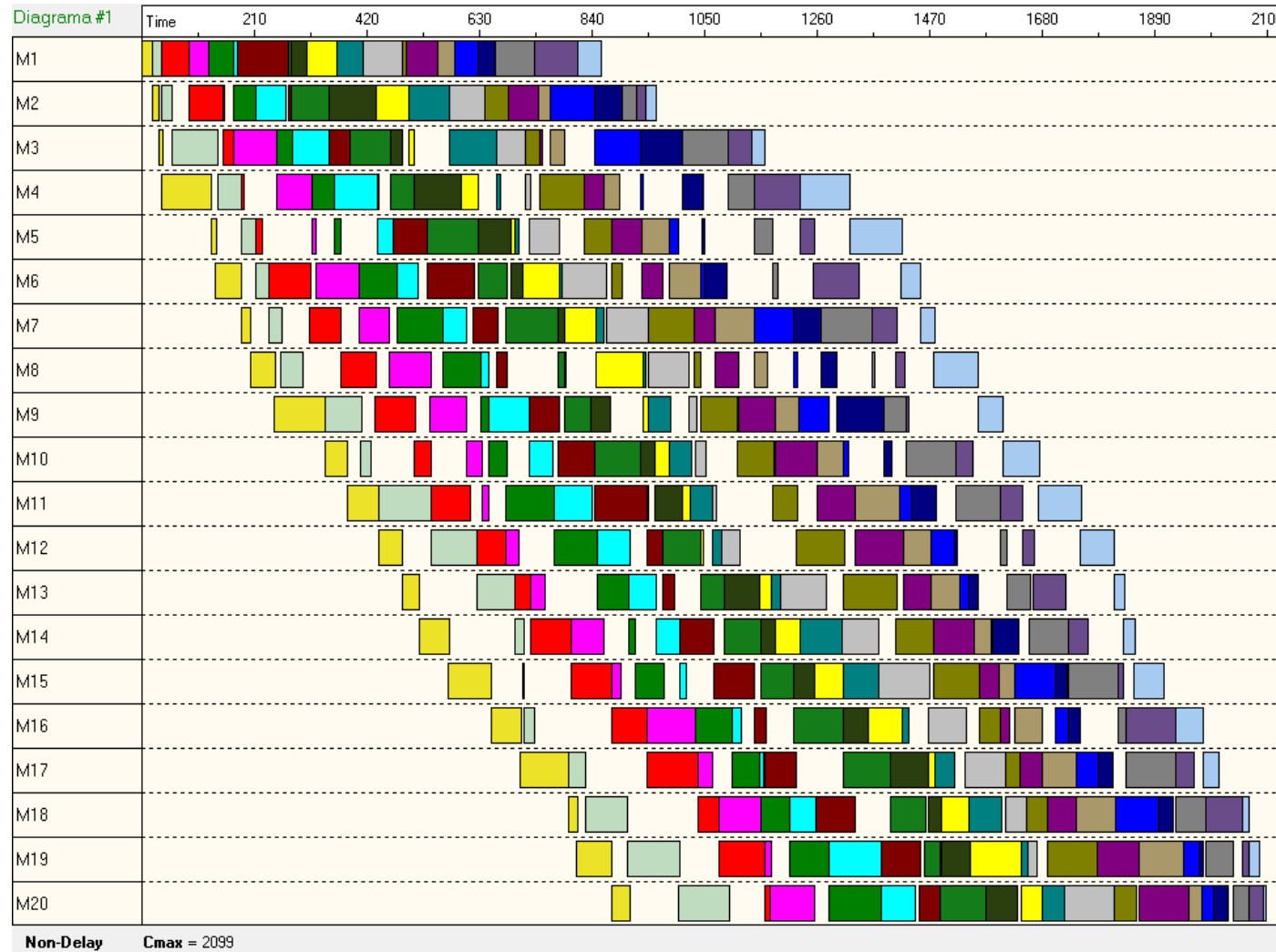
Instancia de problema	Identificadores de las instancias	Total de Instancias
car	car1 – car8	8
hel	hel1 – hel2	2
reC	reC01, reC03, reC05, reC07, reC09, reC11, reC13, reC15, reC17, reC19, reC21, reC23, reC25, reC27, reC29, reC31, reC33, reC35, reC37, reC39, reC41	21
ta	ta001 – ta120	120
Total		151

Anexo 4.4. Resumen de las instancias de problema. Variante *job shop*.

Instancia de problema	Identificadores de las instancias	Total de Instancias
abz	abz5 – abz9	5
ft	ft06, ft10, ft20	3
la	la01 – la40	40
orb	orb01 – orb10	10
swv	swv01 – swv20	20
ta	ta01 – ta80	80
yn	yn1 – yn4	4
Total		162

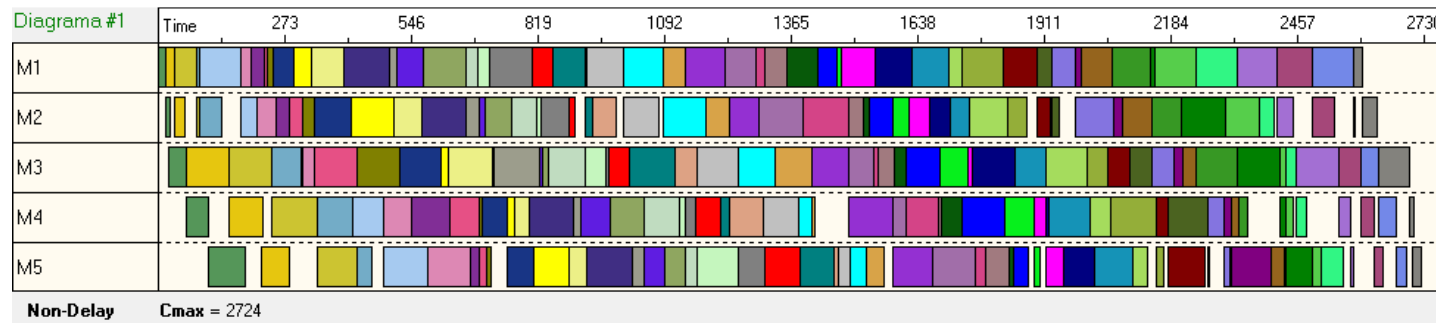
Anexo 5. Gráficos de Gantt con soluciones encontradas a problemas de la variante *flow shop*.

Anexo 5.1. Gráfico de Gantt con solución de la instancia de problema ta022 (20 trabajos en 20 máquinas).

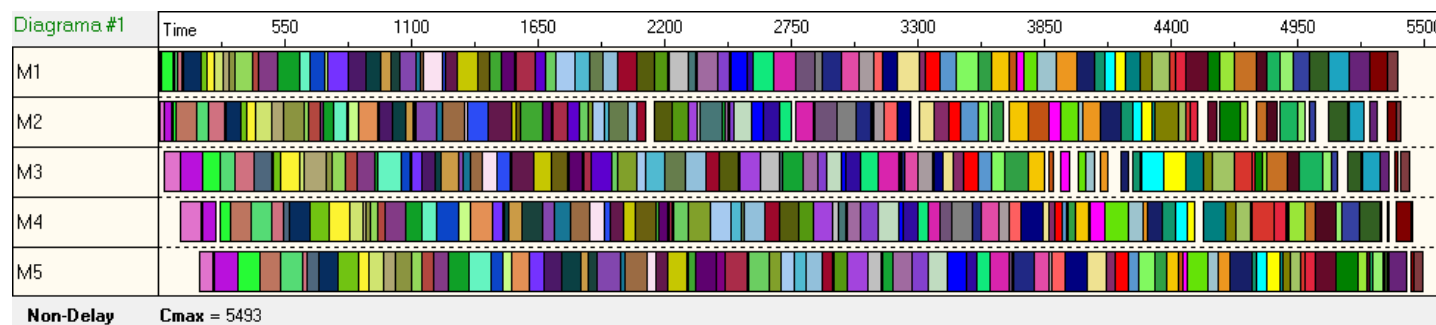


Anexo 5.2. Gráficos de Gantt con soluciones a problemas.

a) Instancia de problema ta031 (50 trabajos en 5 máquinas).

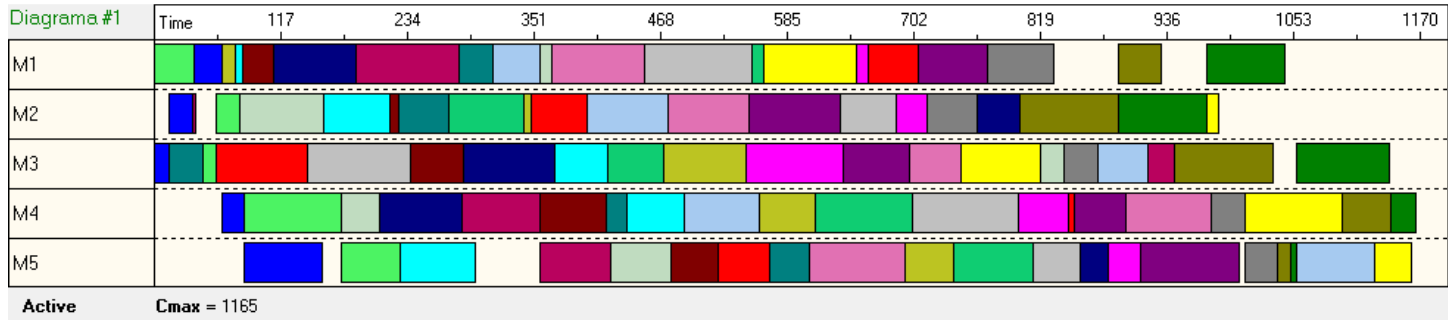


b) Instancia de problema ta061 (100 trabajos en 5 máquinas).

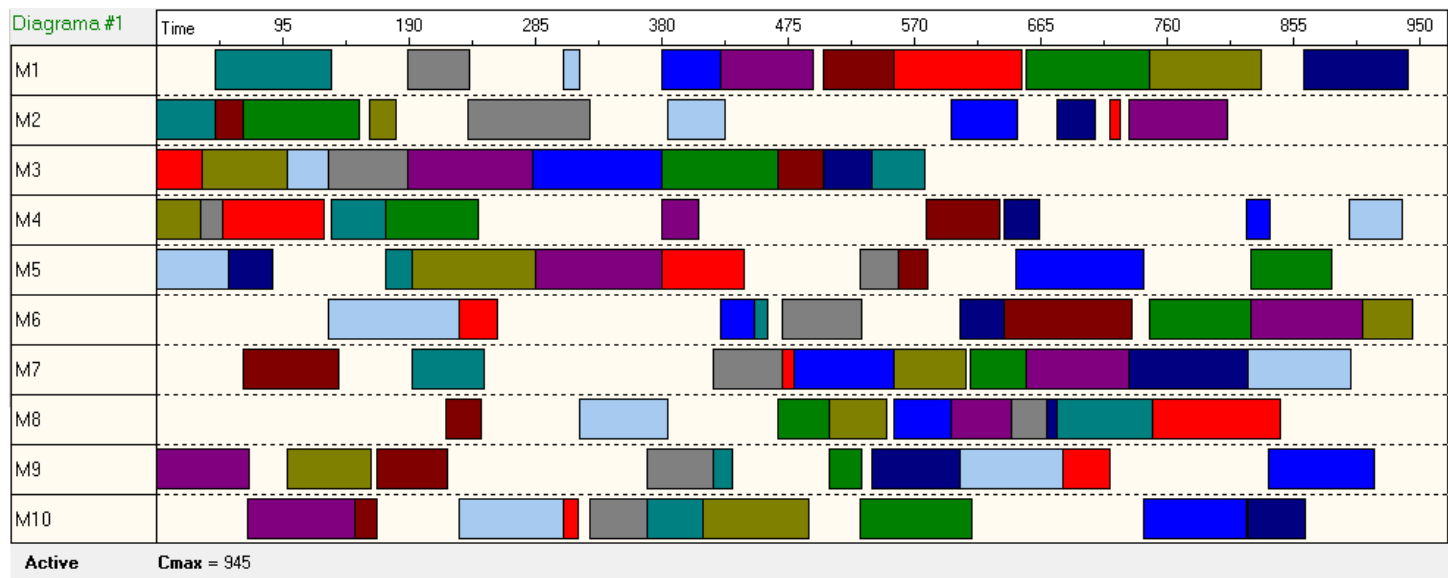


Anexo 6. Gráficos de Gantt con soluciones encontradas a problemas de la variante *job shop*.

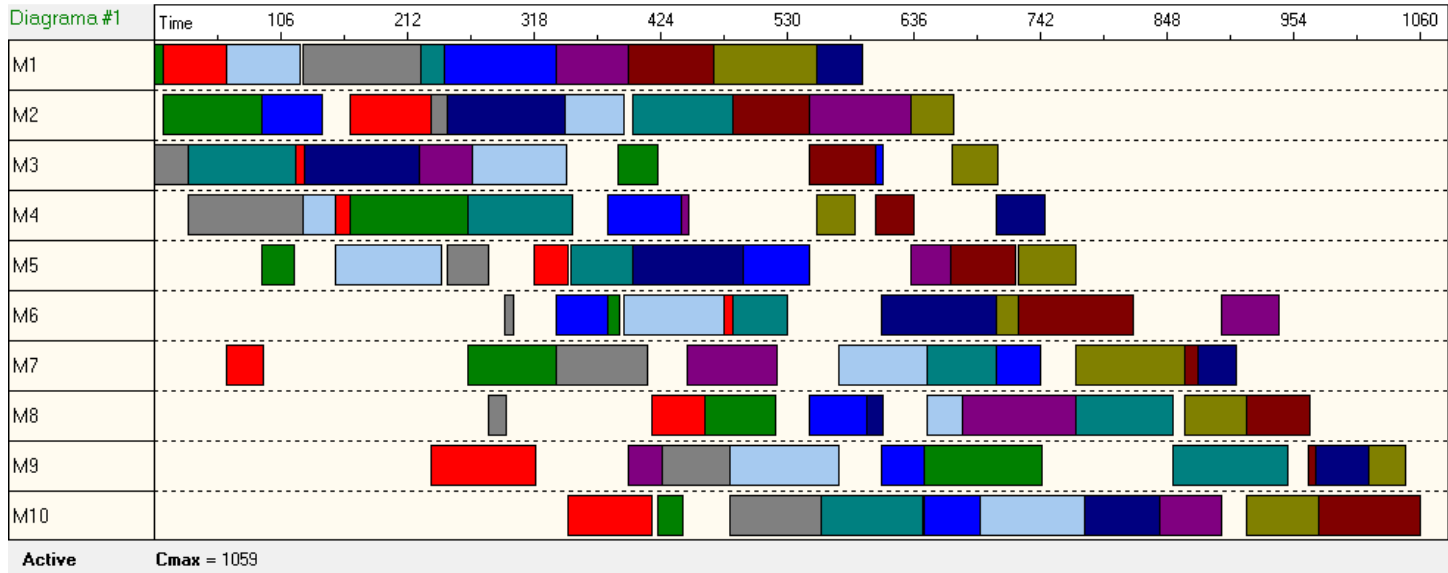
Anexo 6.1. Gráfico de Gantt con solución de la instancia de problema ft20 (20 trabajos en 5 máquinas).



Anexo 6.2. Gráfico de Gantt con solución de la instancia de problema la16 (10 trabajos en 10 máquinas).

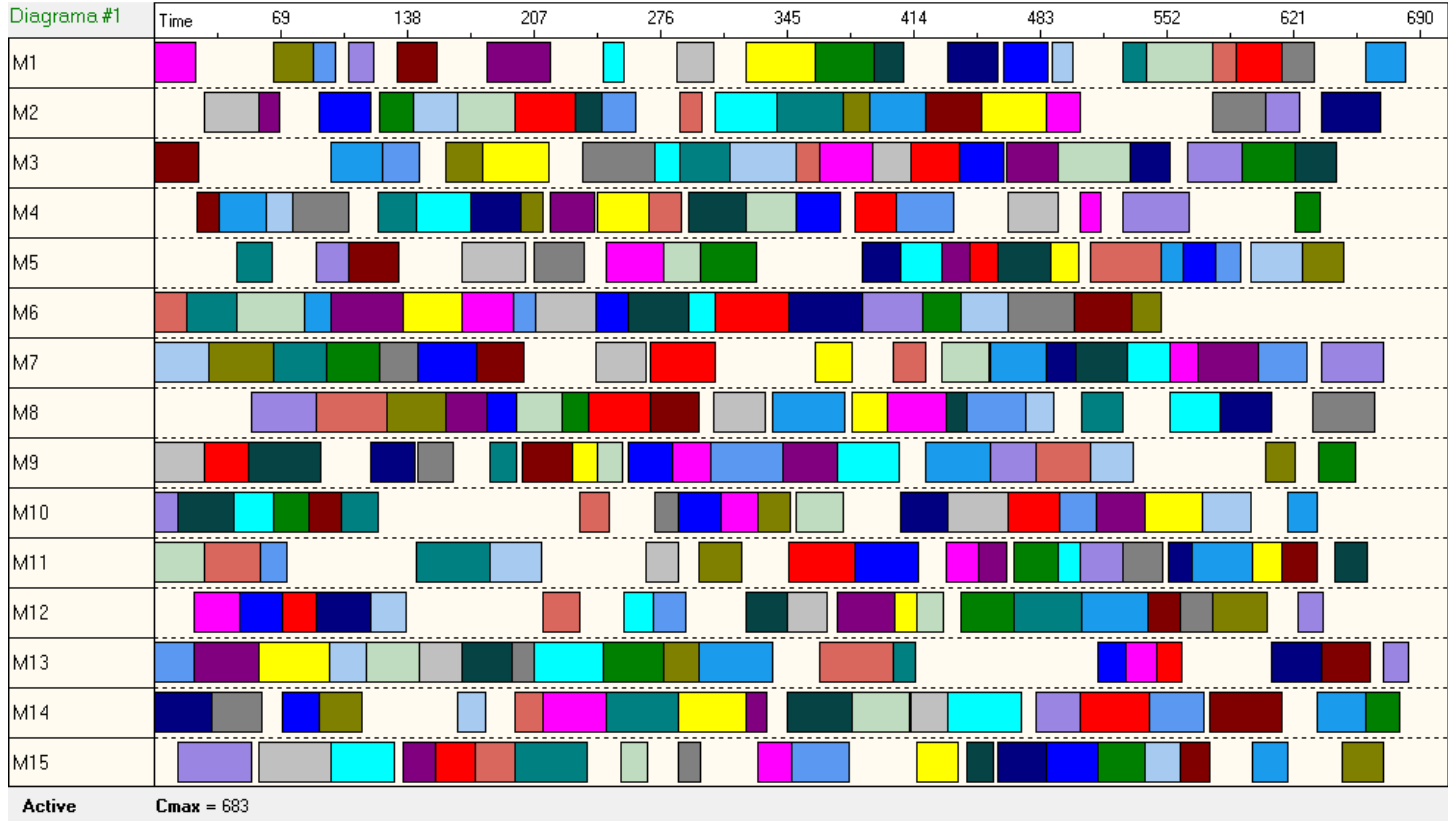


Anexo 6.3. Gráfico de Gantt con solución de la instancia de problema orb01 (10 trabajos en 10 máquinas).

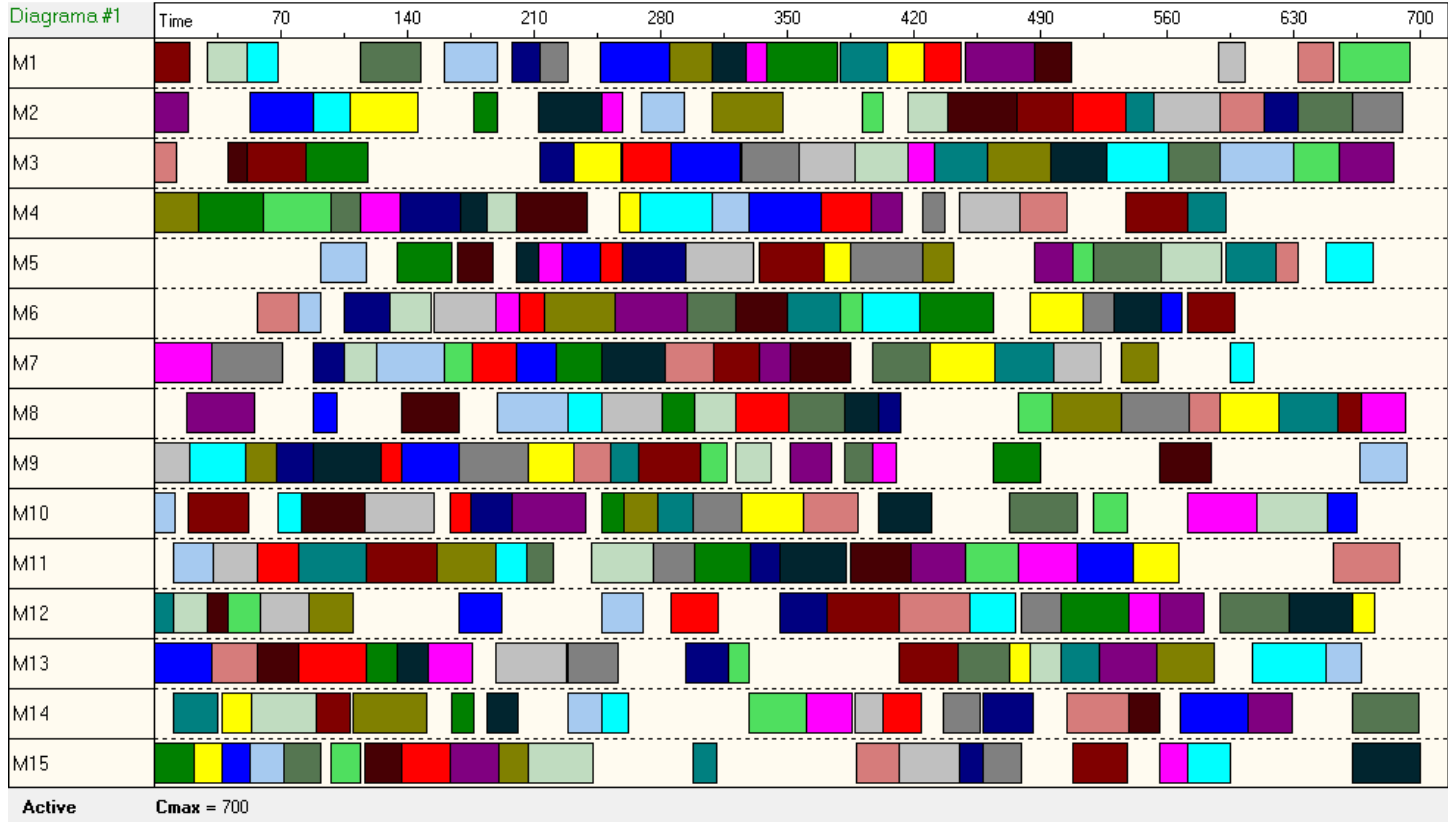


Anexo 7. Gráficos de Gantt con las soluciones encontradas a la selección de problemas nombrados los “10 problemas de mayor dificultad” (*Ten difficult benchmark job shop scheduling problems*).

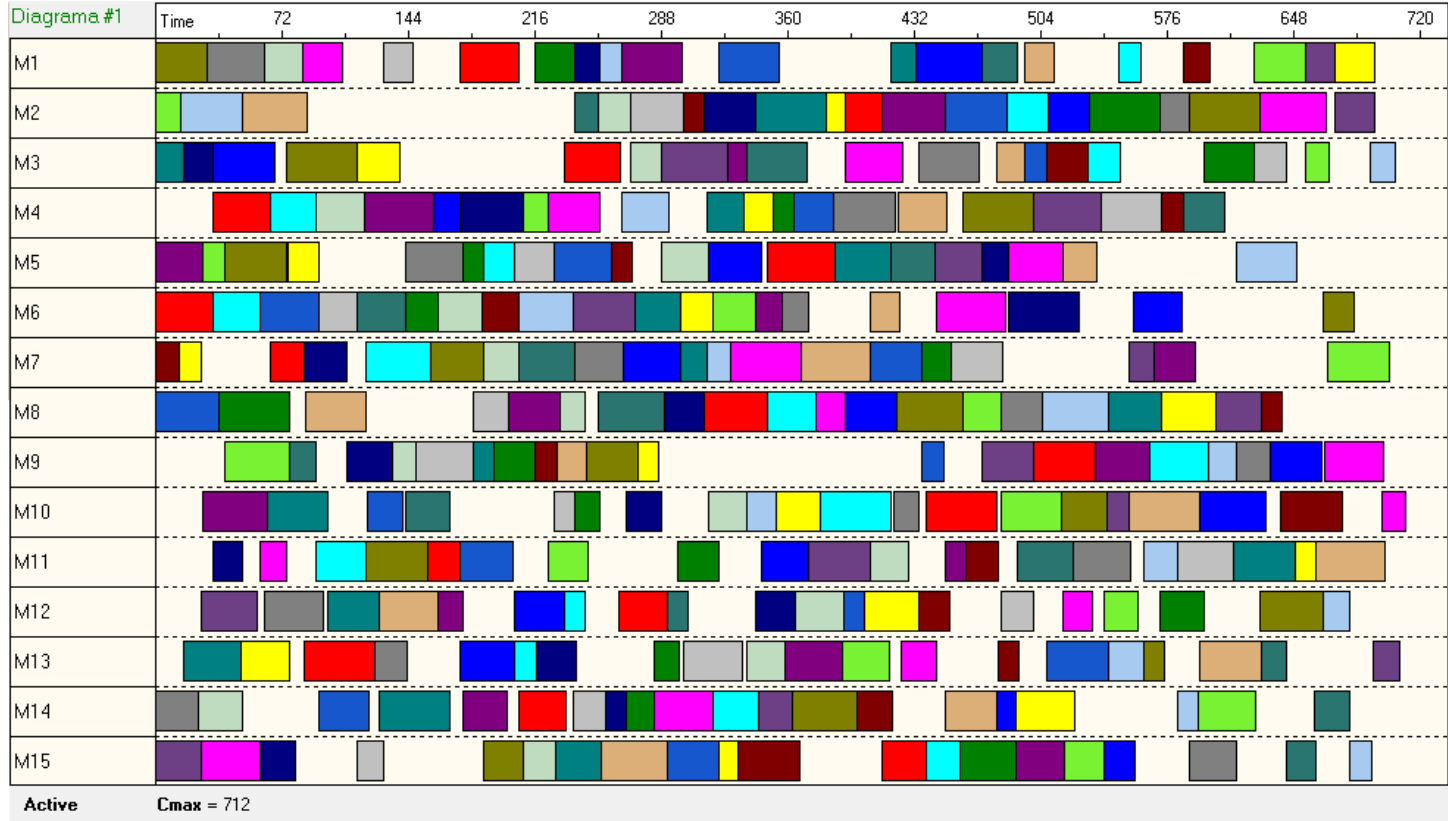
Anexo 7.1. Gráfico de Gantt con solución de la instancia de problema abz7 (20 trabajos en 15 máquinas).



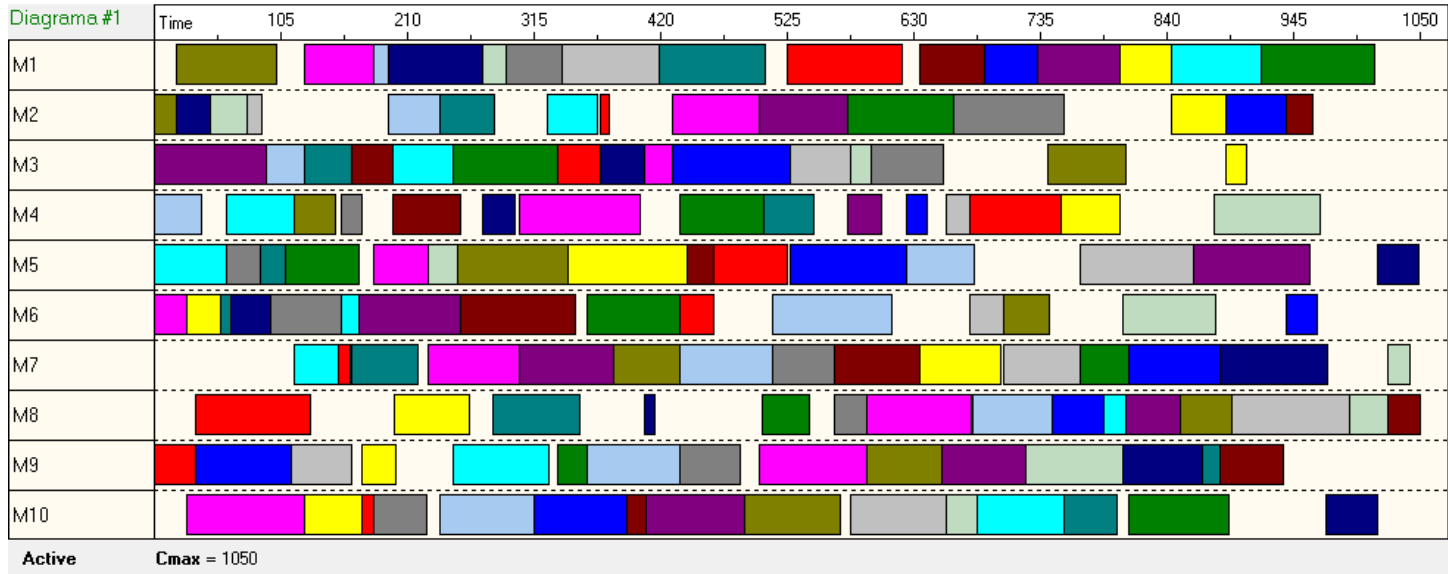
Anexo 7.2. Gráfico de Gantt con solución de la instancia de problema abz8 (20 trabajos en 15 máquinas).



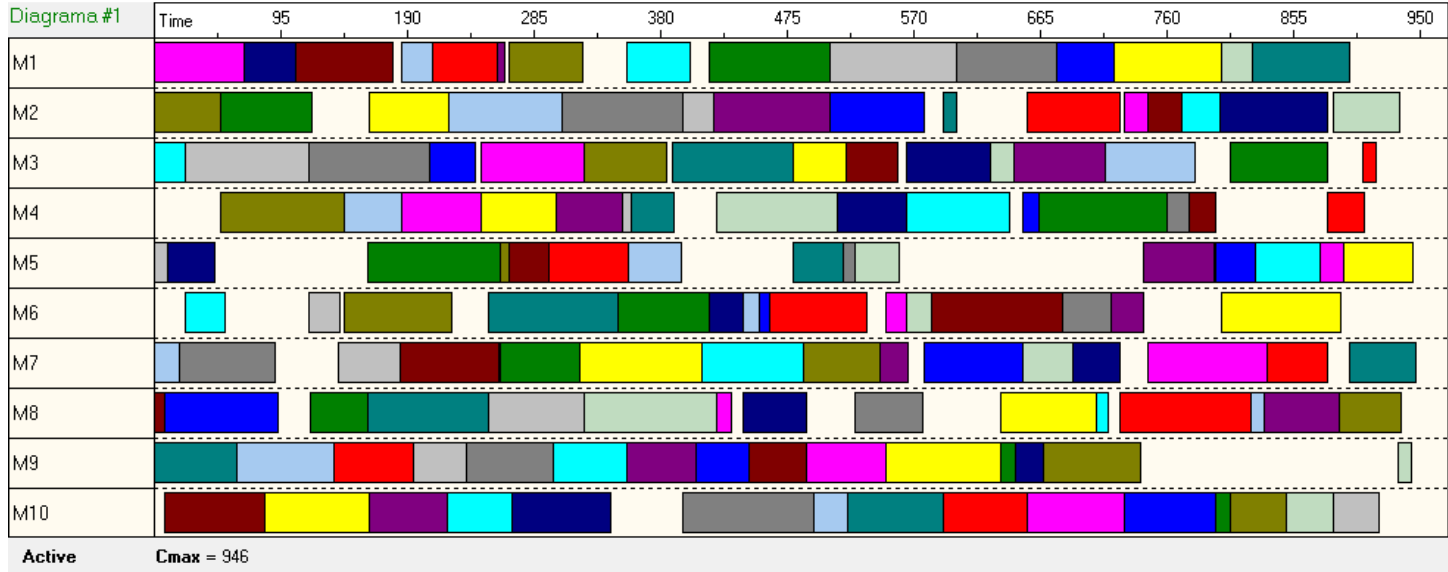
Anexo 7.3. Gráfico de Gantt con solución de la instancia de problema abz9 (20 trabajos en 15 máquinas).



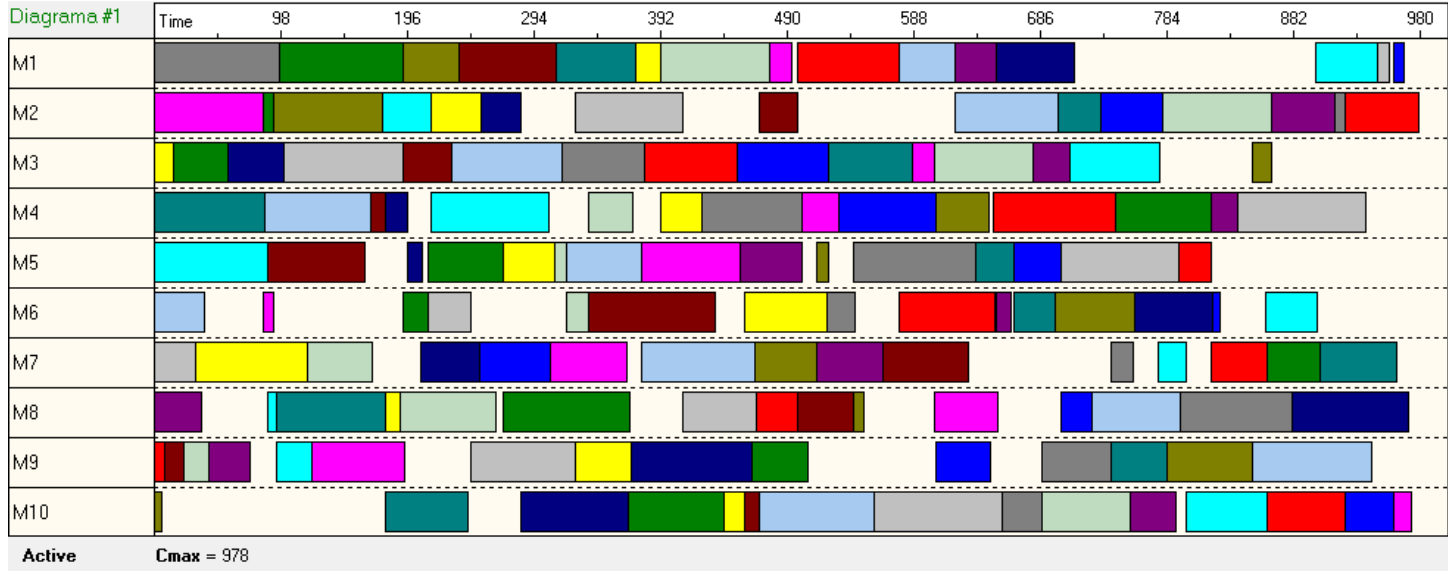
Anexo 7.4. Gráfico de Gantt con solución de la instancia de problema la21 (15 trabajos en 10 máquinas).



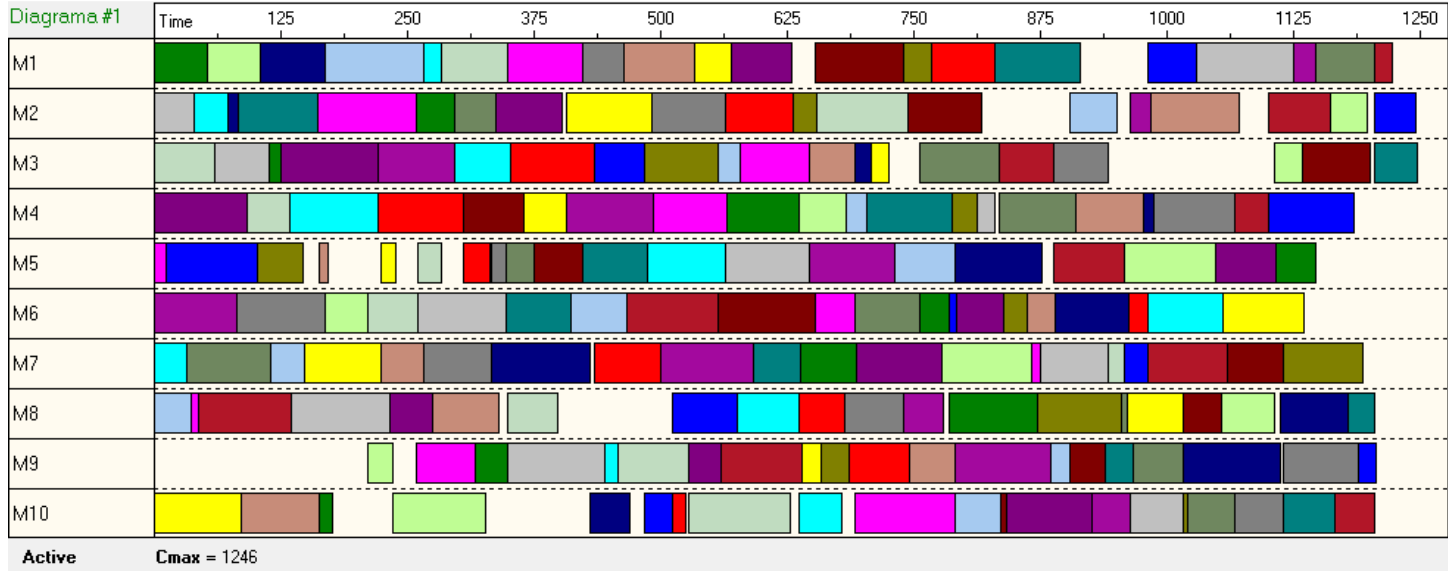
Anexo 7.5. Gráfico de Gantt con solución de la instancia de problema la24 (15 trabajos en 10 máquinas).



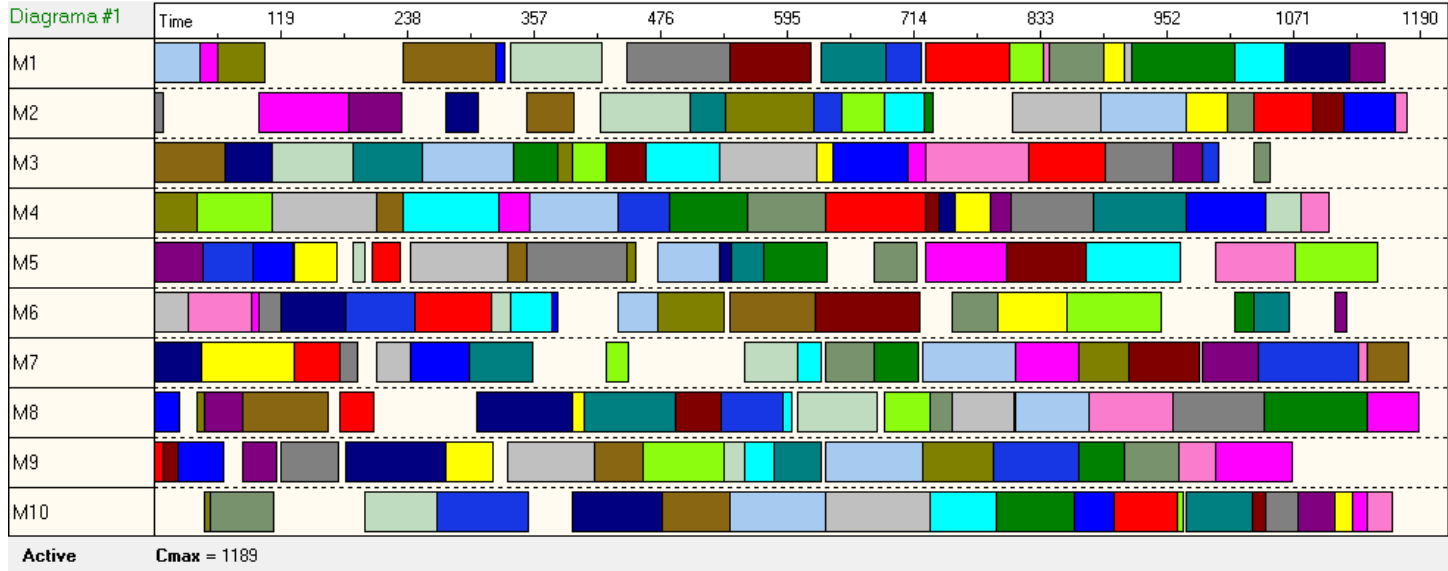
Anexo 7.6. Gráfico de Gantt con solución de la instancia de problema la25 (15 trabajos en 10 máquinas).



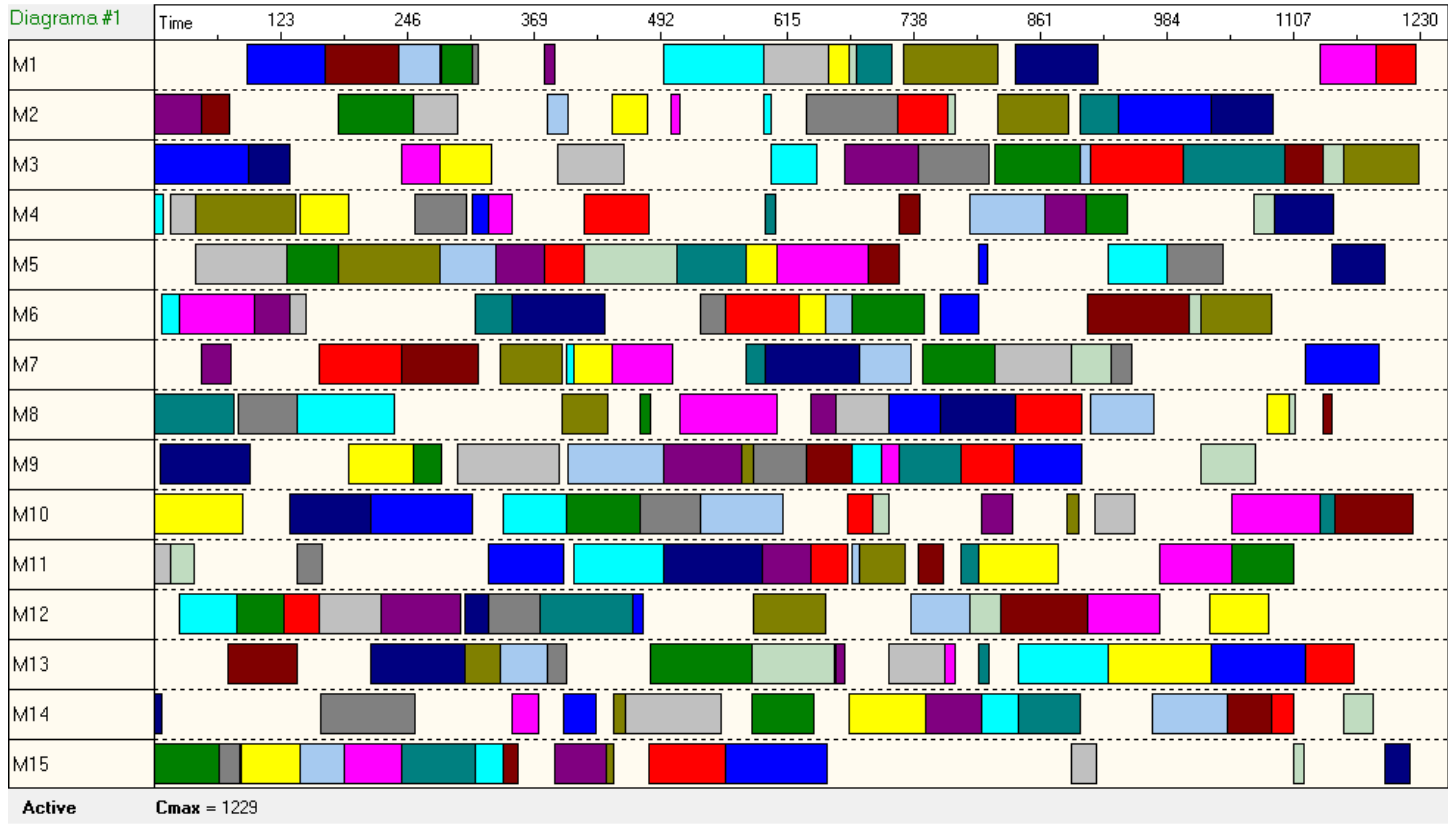
Anexo 7.7. Gráfico de Gantt con solución de la instancia de problema la27 (20 trabajos en 10 máquinas).



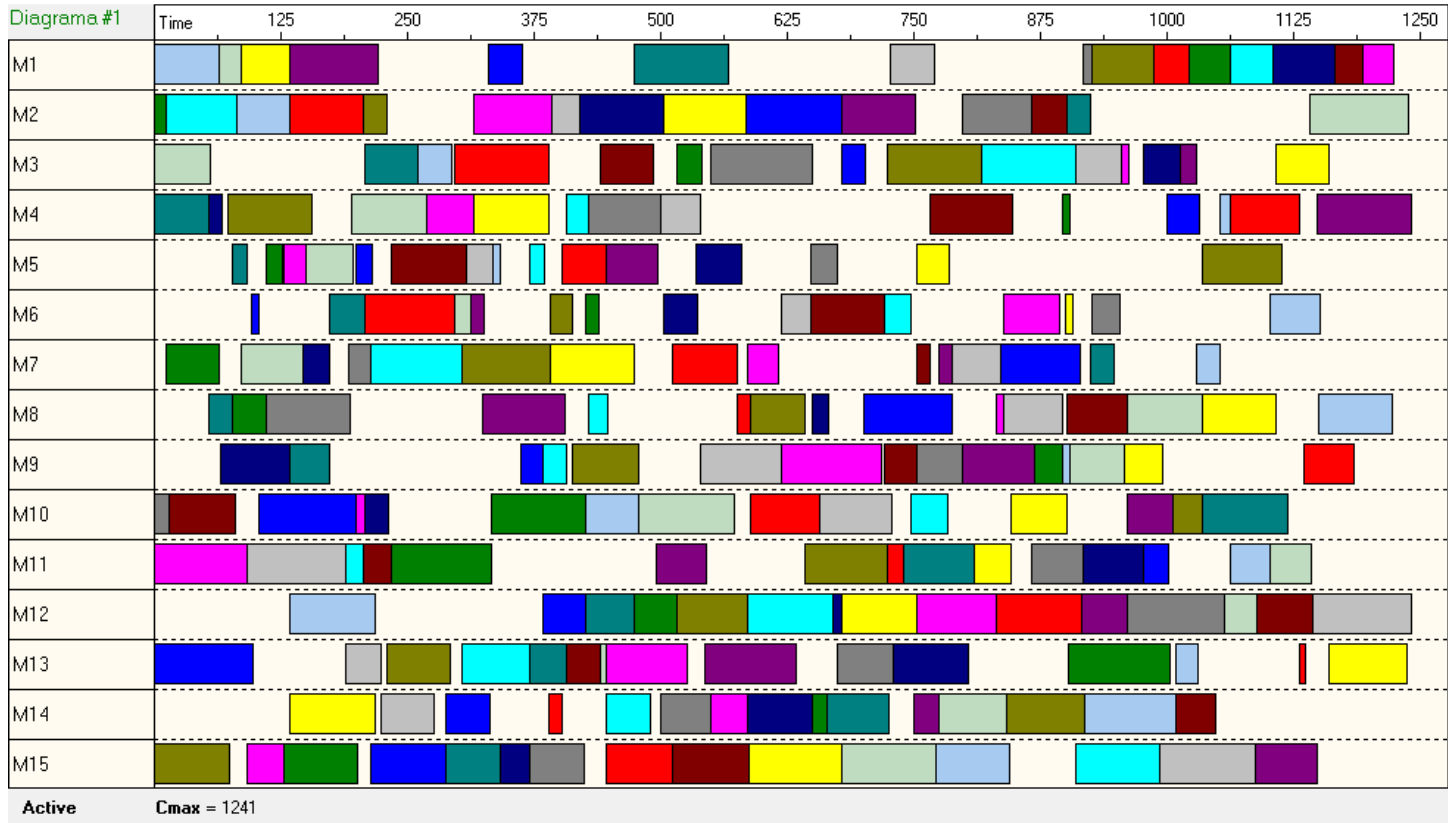
Anexo 7.8. Gráfico de Gantt con solución de la instancia de problema la29 (20 trabajos en 10 máquinas).



Anexo 7.9. Gráfico de Gantt con solución de la instancia de problema la38 (15 trabajos en 15 máquinas).

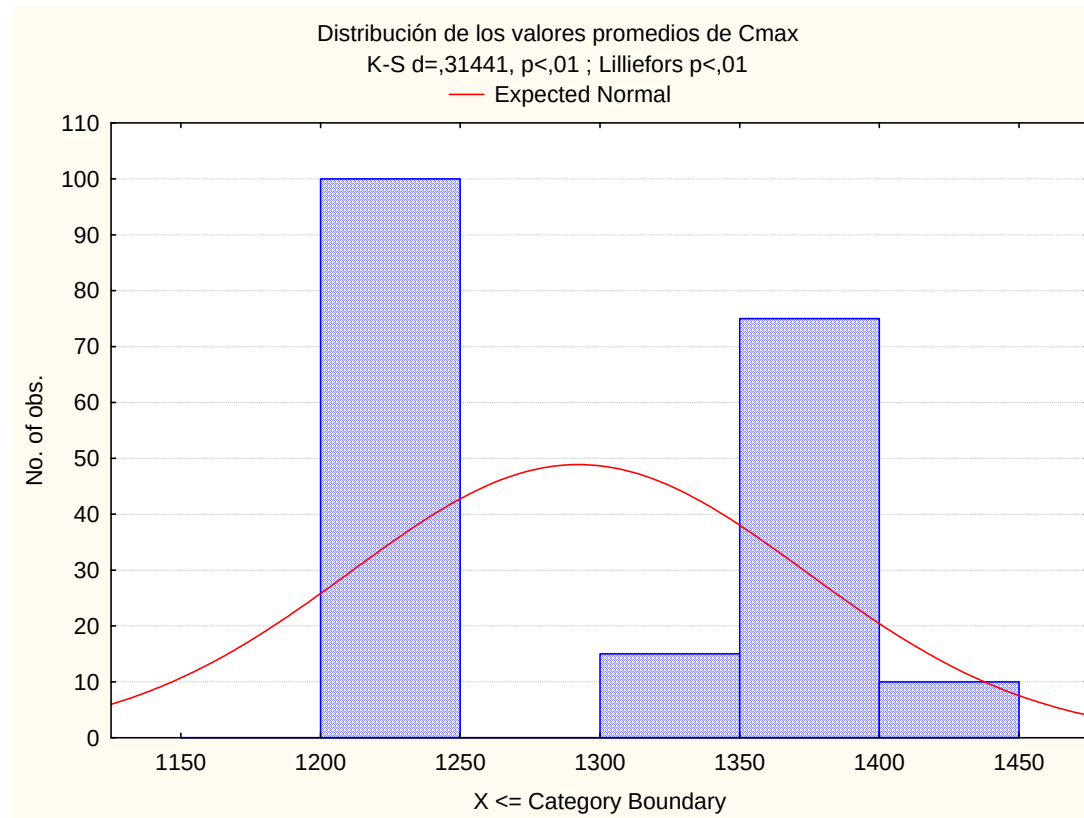


Anexo 7.10. Gráfico de Gantt con solución de la instancia de problema la40 (15 trabajos en 15 máquinas).

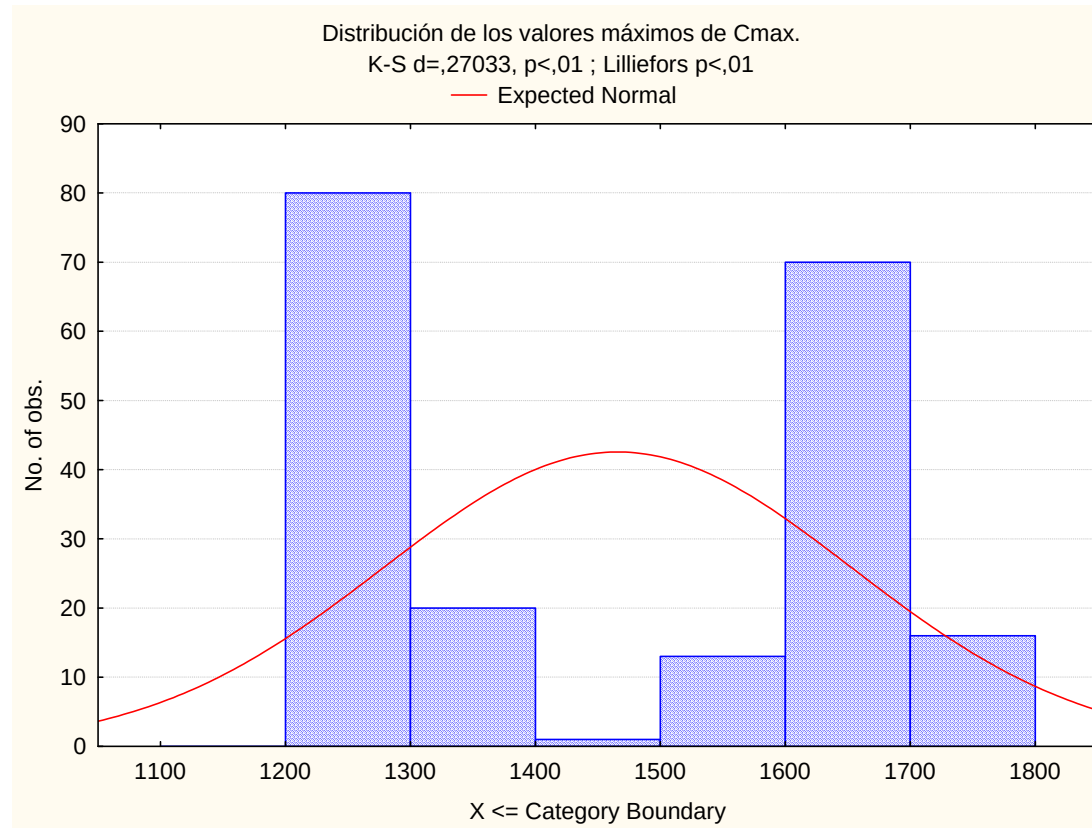


Anexo 8. Gráficos estadísticos resultado del diseño de experimentos.

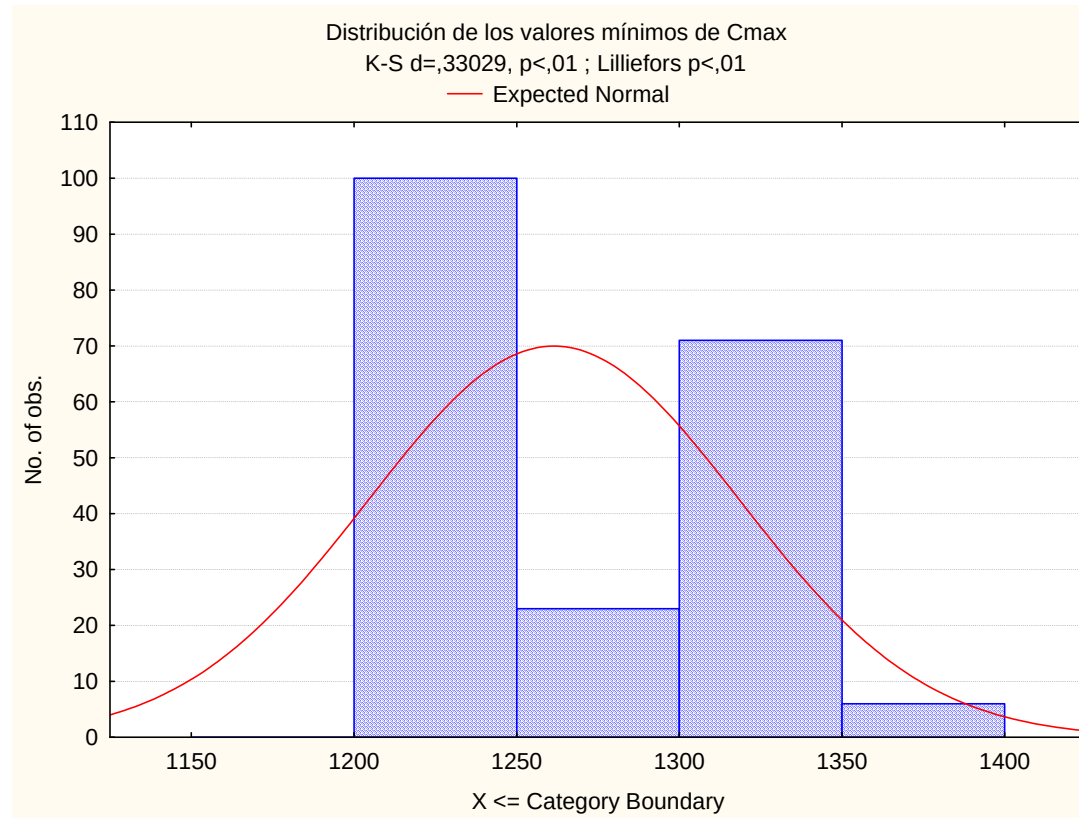
Anexo 8.1. Distribución de los valores promedios de C_{max} .



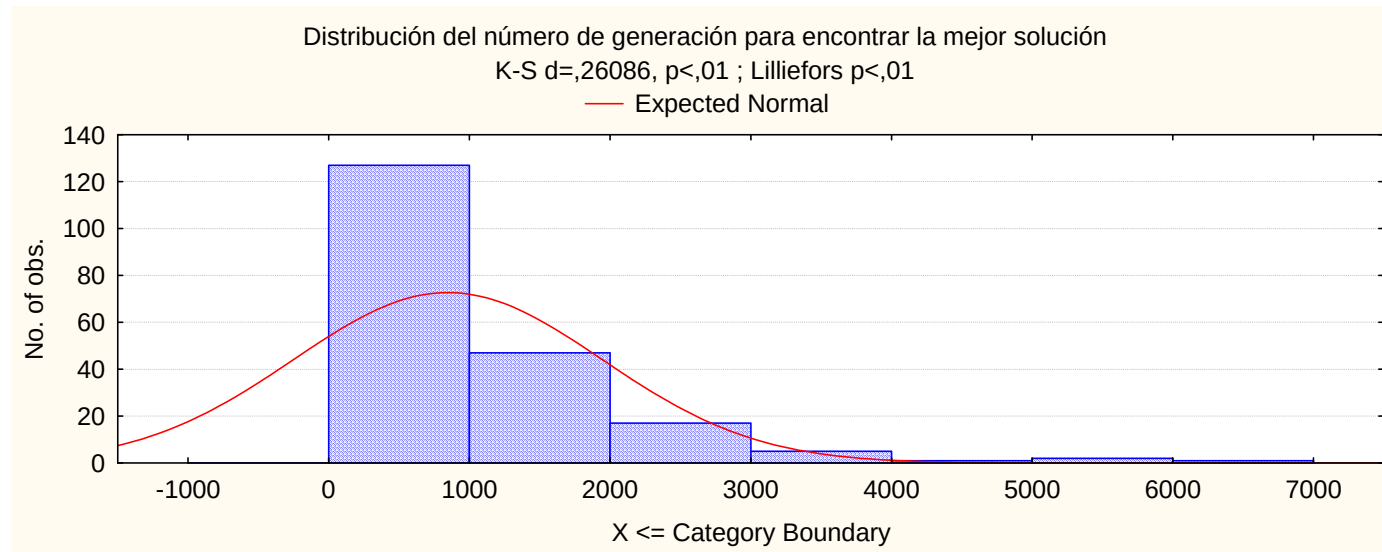
Anexo 8.2. Distribución de los valores máximos de C_{max} .



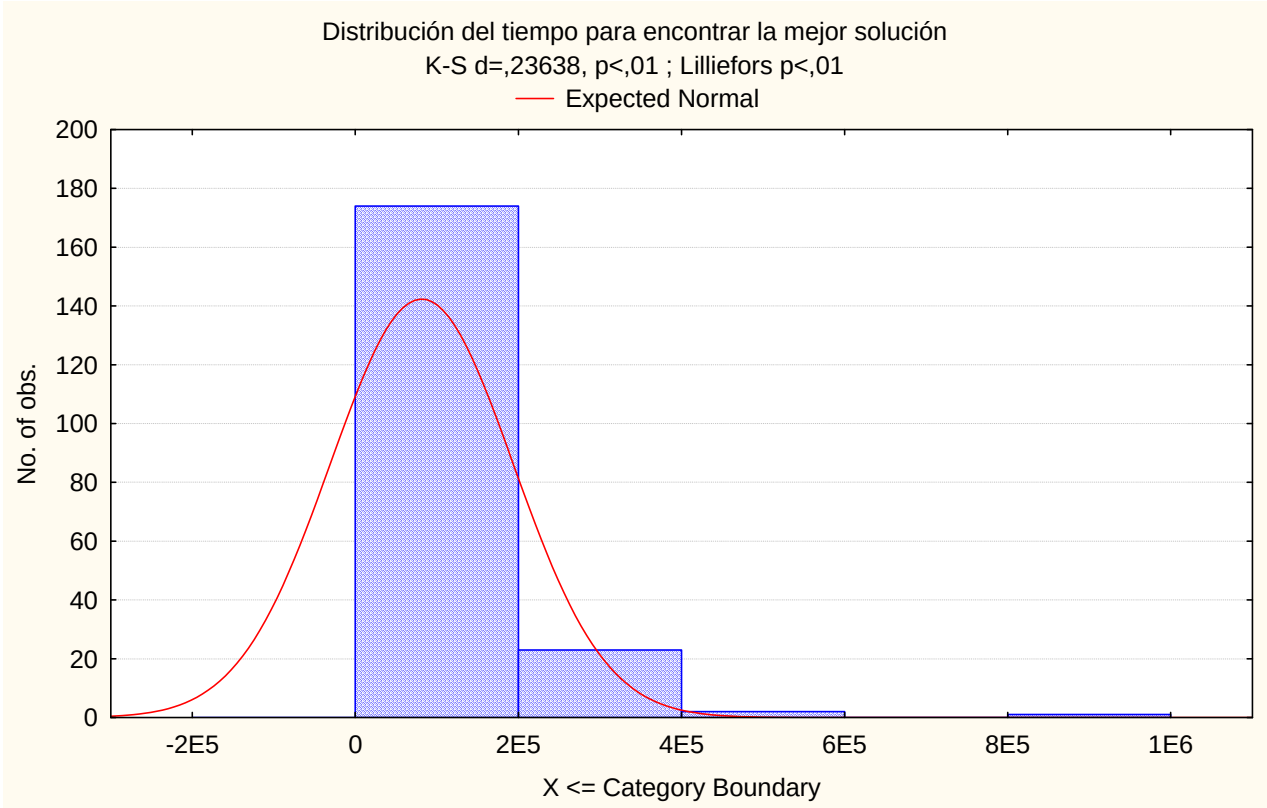
Anexo 8.3. Distribución de los valores mínimos de C_{max} .



Anexo 8.4. Distribución del número de generaciones para encontrar la mejor solución.



Anexo 8.5. Distribución del tiempo para encontrar la mejor solución.



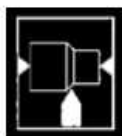
LISTA DE TRABAJOS DEL AUTOR

PUBLICACIONES:

- Márquez, J. E., R. L. Ávila, M. A. Gómez-Elvira y C. R. Herrera (2012). *Algoritmo Genético aplicado al problema de programación en procesos tecnológicos de maquinado con ambiente Flow Shop*. **Revista Ciencias Técnicas Agropecuarias**. 21 (2). pp. 70-75. ISSN 2071-0054.
<http://scielo.sld.cu/pdf/rcta/v21n2/rcta12212.pdf>
- Márquez, J. E., R. L. Ávila, M. A. Gómez-Elvira y C. R. Herrera (2012). *Algoritmo genético aplicado a la programación en talleres de maquinado*. **Revista Ingeniería Mecánica**. 15 (3). pp. 201-212. ISSN 1815-5944.
<http://www.ingenieriamecanica.cujae.edu.cu/index.php/revistaim/article/view/430/787>.

EVENTOS INTERNACIONALES:

- Márquez, J. E., R. L. Ávila, M. A. Gómez-Elvira y C. R. Herrera. *Optimización de secuencias en procesos tecnológicos de maquinado utilizando Algoritmos Genéticos*. **VI SELASI, 15 Convención Científica de Ingeniería y Arquitectura**. Instituto Superior Politécnico “José Antonio Echeverría”. Ciudad de La Habana del 29 de noviembre al 3 de diciembre de 2010.
- Márquez, J. E., R. L. Ávila, M. A. Gómez-Elvira y C. R. Herrera. *Algoritmo genético para la solución de problemas de planificación tipo flow shop y job shop*. **UCIENCIA 2012. VI edición**. Universidad de las Ciencias Informáticas. Ciudad de La Habana del 20 y el 22 de febrero de 2012.



REPARACIÓN Y TECNOLOGÍA MECÁNICA REPAIRING AND MECHANICAL TECHNOLOGICAL

ARTÍCULO ORIGINAL

Algoritmo Genético aplicado al problema de programación en procesos tecnológicos de maquinado con ambiente Flow Shop

Genetic algorithm applied to the scheduling problem in technological processes of having schemed with ambient Flow Shop

José Eduardo Márquez Delgado¹, Ricardo Lorenzo Ávila Rondón², Miguel Ángel Gómez-Elvira González³
y Carlos Rafael Herrera Márquez⁴

RESUMEN. Debido a las limitaciones de las técnicas de optimización convencionales, en el siguiente trabajo se presenta una metaheurística basada en un algoritmo genético (AG), para resolver problemas de programación de tipo flow shop, con el objetivo de minimizar el tiempo de finalización de todos los trabajos, más conocido como makespan. Este problema, considerado de difícil solución, es típico de la optimización combinatoria y se presenta en talleres con tecnología de maquinado, donde existen máquinas-herramientas convencionales y se fabrican diferentes tipos de piezas que tienen en común una misma ruta tecnológica (orden del proceso). La solución propuesta se probó con problemas clásicos publicados por otros autores, obteniéndose resultados satisfactorios en cuanto a la calidad de las soluciones encontradas y el tiempo de cómputo empleado.

Palabras clave: algoritmo genético, flow shop, programación, makespan.

ABSTRACT. Due to the limitations of the conventional techniques of optimization, in the following work a metaheuristic one is presented based on a genetic algorithm (GA), to solve scheduling problems of type flow shop, with the objective of minimizing the time of culmination of all the works, good known as makespan. This problem, considered to be a difficult solution is typical in combinatory optimization and it is presented in workshops with technology of having schemed, where conventional machine-tools exist and different types of pieces are manufactured that have in common oneseft technological route (process order). The proposed solution it was proven with classic problems published by other authors, obtaining you satisfactory results as for the quality of the opposing solutions and the time of used computation.

Keywords: genetic algorithm, flow shop, scheduling, makespan.

INTRODUCCIÓN

La planificación y control de la producción se reconoce como un problema complejo en el mundo empresarial, y en particular la secuenciación (scheduling), realiza un rol muy importante dentro del proceso de toma de decisiones en una organización y es usada en muchas áreas de la misma, tales

como: producción, distribución, transporte, en procesos de información y comunicación, etc. El desarrollo actual de las computadoras, y la aparición de nuevas técnicas de simulación y optimización heurística que aprovechan plenamente las disponibilidades de cálculo intensivo que estas proporcionan, han abierto una nueva vía para abordar los problemas de secuenciación o problemas de scheduling como también se le conocen,

Recibido 10/12/10, aprobado 30/01/12, trabajo 27/12, artículo original.

¹ Prof., Departamento de Informática, Facultad de Ciencias Técnicas, Universidad de Granma (UDG), Carretera Bayamo-Manzanillo km. 17 ½, Granma, CP 85100, Cuba, E-✉: jmarquezd@udg.co.cu

² Prof., Centro de Estudios CAD/CAM, Facultad de Ingeniería, Universidad de Holguín (UHo), Ave. XX Aniversario Holguín. CP: 80900, Cuba.

³ Prof., Escuela Técnica Superior de Ingenieros (E.T.S.I.) Agrónomos, Universidad Politécnica de Madrid (UPM), España.

⁴ Ing., Empresa de Acumuladores "XX Aniversario", Zona Industrial. Manzanillo, CP 87510 Granma, Cuba.

Algoritmo genético aplicado a la programación en talleres de maquinado

Genetic algorithm applied to scheduling in machine shops

José Eduardo Márquez-Delgado ^I, Ricardo Lorenzo Ávila-Rondón ^{II},
Miguel Ángel Gómez-Elvira-González ^{III}, Carlos Rafael Herrera-Márquez ^{IV}

I. Universidad de Granma. Facultad de Ciencias Técnicas. Granma. Cuba

Correo electrónico: jmarquezd@udg.co.cu

II. Universidad de Holguín. Facultad de Ingeniería. Holguín. Cuba.

III. Universidad Politécnica de Madrid. Escuela Técnica Superior de Ingenieros Agrónomos. Madrid. España.

IV. Empresa de Acumuladores XX Aniversario. Manzanillo, Granma. Cuba.

Recibido: 28 de diciembre de 2011 Aceptado: 20 de julio de 2012

Resumen

En este trabajo se utiliza la metaheurística nombrada algoritmo genético, para dos variantes típicas de problemas de planificación presentes en un taller de maquinado de piezas: las variantes flujo general y flujo regular, y se ha seleccionado la minimización del tiempo de finalización de todos los trabajos o camino máximo, como objetivo a optimizar en un plan de trabajo. Este problema es considerado de difícil solución y es típico de la optimización combinatoria. Los resultados demuestran la calidad de las soluciones encontradas en correspondencia con el tiempo de cómputo empleado, al ser comparados con problemas clásicos reportados por otros autores. La representación propuesta de cada cromosoma genera el universo completo de soluciones factibles, donde es posible encontrar valores óptimos globales de solución y cumple con las restricciones del problema.

Palabras claves: algoritmo genético, cromosomas, flujo general, flujo regular, planificación, camino máximo.

Abstract

In this paper we use the metaheuristic named genetic algorithm, for two typical variants of problems of scheduling present in a machine shop parts: the variant job shop and flow shop, and the minimization of the time of finalization of all the works has been selected, good known as makespan, as objective to optimize in a work schedule. This problem is considered to be a difficult solution and is typical in combinatorial optimization. The results demonstrate the quality of the solutions found in correspondence with the time of used computation, when being compared with classic problems reported by other authors. The proposed representation of each chromosome generates the complete universe of feasible solutions, where it is possible to find global good values of solution and it fulfills the restrictions of the problem.

Key words: genetic algorithm, chromosomes, flow shop, job shop, scheduling, makespan.